14 haftalık yol haritasında 14. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

# HAFTA 14

# Sentez

## Algoritma Tasarım Stratejileri, Hesaplanabilirliğin Sınırları ve Final Projesi

MODÜL V · OPTİMİZASYON VE SENTEZ · H13–H14

> **HAFTANIN BÜYÜK SORUSU**
>
> *Her problem hızlı bir algoritmayla çözülebilir mi ve çözülemiyorsa bir mühendis ne yapar?*

**Dr. Öğr. Üyesi Mehmet Sevri**
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

# İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları ..... 5
1.1 Öğrenme çıktıları ..... 5
2. Geçen Haftadan ve Dönemden Bağlantı ..... 6
3. Kavramsal Anlatım: Strateji, İndirgeme ve Hesaplanabilirliğin Sınırları ..... 6
3.1 Dönemin haritası: strateji aileleri ..... 6
3.2 Strateji seçimi için karar soruları ..... 7
3.3 Problem indirgeme ve modelleme: “Bunu daha önce gördüm mü?” ..... 8
3.4 P, NP ve NP-tamlık (sezgisel düzeyde) ..... 9
3.5 Hesaplanabilirliğin sınırı: durma problemi ..... 11
3.6 NP-zor problemlerle başa çıkmak ..... 11
3.7 Performans mühendisliği: önce ölç ..... 11
3.8 Kaputun Altında ..... 11
4. Algoritmalar: Gezgin Satıcı İçin Dört Yöntem ..... 12
4.1 Kaba kuvvet: bütün turlar (Örnek 2) ..... 12
4.2 En yakın komşu sezgiseli (Örnek 3) ..... 15
4.3 2-opt yerel arama (Örnek 4) ..... 18
4.4 Held–Karp: bit maskesi DP'si (Örnek 5, merak) ..... 20
4.5 Dört yöntemin karşılaştırması ..... 23
5. Çözümlü Örnekler ve Uygulamalar ..... 23
5.1 Problem–strateji eşleme: on iskelet (Örnek 1) ..... 24
5.2 Doğrulama ile bulma (Örnek 6) ..... 24
5.3 İndirgeme örnekleri: sudoku, labirent, ders planı (Örnek 7) ..... 25
5.4 Profil çıkarma: darboğazı bulmak (Örnek 8) ..... 26
5.5 Dönem kıyaslama programı (Örnek 9) ..... 27
5.6 Final projesi entegrasyonu: Mini-OBS çekirdeği (Örnek 10) ..... 28
6. Haftanın Deneyi: Dönem Kıyaslama Raporu ..... 29
6.1 Soru ve ölçüm düzeni ..... 29
6.2 (a) İlk ↔ son sürüm: Mini-OBS 2'nin dört işlemi ..... 29
6.3 (b) Algoritmik ↔ derleyici ↔ dil kazancı ..... 30
6.4 (c) Dönemin sentezi: her haftadan bir bulgu ..... 30
6.5 Yorum ve sınırlılıklar ..... 30
7. Hata Avcısı ..... 31
8. Dil Penceresi: C · C++ · Python ..... 33
9. Mühendis Gözüyle: Önce Doğru, Sonra Ölç ..... 34

10. Final Projesi: Mini-OBS 2 v3.0 ve Seçenek Havuzu ..... 35
10.1 Yönerge özeti ve takvim ..... 35
10.2 Varsayılan proje: Mini-OBS 2 v3.0 ..... 35
10.3 Seçenek havuzu ve ortak asgari gereksinimler ..... 37
10.4 Teknik kurallar ve teslim edilecekler ..... 38
10.5 Kabul ölçütleri ve test senaryoları ..... 38
10.6 Rubrik (yol haritası Tablo 10.3) ..... 39
10.7 Ölçüm raporu, rapor şablonu ve sunum ..... 39
11. Alıştırmalar ve Atölyeler ..... 40
11.1 İzleme ve çıktı tahmini (A) ..... 40
11.2 Karmaşıklık (B) ..... 41
11.3 Değişmez yazma (C) ..... 41
11.4 Hata bulma (D) ..... 41
11.5 Parsons (E) ..... 42
11.6 Kod yazma (F) ..... 42
11.7 Diller arası okuma (G) ..... 43
11.8 Atölye 1 · Problem–strateji eşleme turnuvası (takımlar, ≈ 30 dk) ..... 44
11.9 Atölye 2 · Gezgin satıcı yarışması: 50 şehirde en kısa tur (≈ 40 dk) ..... 44
11.10 Atölye 3 · Final projesi sunumları: canlı demo, ölçüm raporu ve kod incelemesi ..... 45
12. Kendini Değerlendirme ve Final Hazırlığı ..... 45
12.1 H1–H14 konu haritası ..... 46
12.2 Soru türleri ve soru bankası ..... 46
12.3 Çalışma planı (iki hafta) ..... 46
13. İleri Bakış ve Merak Kutusu ..... 47
13.1 Merak Kutusu: Held–Karp neden 16–18 şehirde durur? ..... 47
13.2 Merak Kutusu: derleyicim neden “bu döngü sonsuz” demiyor? ..... 47
14. Özet, Kavram Haritası ve Terim Sözlüğü ..... 47
15. Kaynaklar ..... 49
Ek A. Dil Karşılıkları ..... 51
A.1 Örnek 1 · Problem–strateji eşleme (P8: DP ↔ açgözlü) ..... 51
A.2 Örnek 2 · Gezgin satıcı: kaba kuvvet ..... 52
A.3 Örnek 3 · En yakın komşu ..... 54
A.4 Örnek 4 · 2-opt yerel arama ..... 55
A.5 Örnek 5 · Held–Karp ..... 56
A.6 Örnek 6 · Doğrulama ile bulma ..... 58
A.7 Örnek 7 · İndirgeme: labirent → BFS ..... 60

A.8 Örnek 8 · Profil çıkarma: darboğaz.....61
A.9 Örnek 9 · Dönem kıyaslaması: karma (denetim toplamı).....62
A.10 Örnek 10 · Mini-OBS çekirdeği: düzenleme uzaklığı.....62
Ek B. Başvuru Tabloları.....64
B.1 Strateji kataloğu (Bölüm 3.1).....64
B.2 Dönem kıyaslamasının ayrıntı tabloları (Bölüm 6).....64
B.3 Final projesi: gereksinimler, teslim, kabul ölçütleri ve test senaryoları (Bölüm 10.2–10.5).....66
B.4 Final projesi: rubrik düzeyleri ve sunum yönergesi (Bölüm 10.6–10.7).....68
B.5 Final hazırlığı: konu haritası, çalışma planı ve örnek sorular (Bölüm 12).....70

# 1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Her problem hızlı bir algoritmayla çözülebilir mi ve çözülemiyorsa bir mühendis ne yapar?

On üç hafta boyunca problemleri birer algoritma ailesiyle hızlandırdık: aramayı Θ(n)'den Θ(log n)'e ve beklenen Θ(1)'e, yinelenen tespitini $\Theta(n^2)$'den Θ(n)'e, ders seçimini üstelden sözde polinoma indirdik. **Yeni bir problem için hangi stratejiyi seçeceğiz?** Dönemi bir **strateji kataloğunda** topluyor, karar sorularıyla ve **indirgemeyle** (reduction) eşleştiriyoruz. **Her problemin hızlı bir algoritması var mı?** Gezgin satıcı problemi (traveling salesman problem) bu sorunun simgesidir: 5 şehirde 12 farklı tur vardır, 20 şehirde $6{,}082 \times 10^{16}$; saniyede $10^9$ tur değerlendiren bir makine bile bunları 1,93 yılda tarar (K01). Yanıtın kuramsal yarısı **P, NP ve NP-tamlık**tır: çözümü **doğrulamak** kolay olduğu hâlde **bulmanın** hızlı yolu bilinmeyen büyük bir problem ailesi vardır; durma problemi ise bazı soruların hiçbir algoritmayla yanıtlanamayacağını gösterir. Mühendislik yarısı: n küçükse kaba kuvvet ya da Held–Karp, büyükse **sezgisel** (heuristic) tur ve **yerel arama** (local search) — ve her durumda **önce ölç**. Dönemi **final projesinde** (Mini-OBS 2 v3.0) birleştiriyor, final sınavına hazırlanıyoruz.

> **GERÇEK DÜNYADAN · NP-tamlık, bir milyon dolarlık soru ve 85 900 şehirlik tur**
>
> Stephen Cook, Mayıs 1971'de Ohio'daki ACM Hesaplama Kuramı Sempozyumunda (STOC '71) belirlenimci olmayan bir makinenin polinom zamanda tanıdığı her problemin önermeler mantığındaki bir formül sorusuna polinom zamanda indirgenebildiğini gösterdi (Cook, 1971): formül sağlanabilirliği (SAT) bugün **NP-tam** dediğimiz ilk problem oldu (Cormen vd., 2022). Karp (1972) aynı teknikle 21 problemin — aralarında 0–1 tamsayı programlama, klik, Hamilton devresi, kromatik sayı (graf boyama) ve sırt çantası — aynı zorlukta olduğunu gösterdi; gezgin satıcının karar sürümü Hamilton devresinden kolayca indirgenir.
>
> Clay Matematik Enstitüsü yedi Milenyum Problemi'ni 24 Mayıs 2000'de Paris'te, Collège de France'ta duyurdu ve her birine bir milyon dolar ödül koydu; P ↔ NP bunlardan biridir ve hâlâ çözülmemiştir (Clay Mathematics Institute, t.y.).
>
> Mühendislik ise büyük örnekleri çözdü: kesin çözülen ilk büyük örnek 1954'te Dantzig, Fulkerson ve Johnson'ın 49 şehirlik turuydu (University of Waterloo, t.y.-a); Applegate, Bixby, Chvátal, Cook ve Helsgaun Mayıs 2004'te İsveç'in 24 978 şehrinin en kısa turunu kanıtlayarak Nisan 2001'deki 15 112 şehirlik Almanya rekorunu geçti (University of Waterloo, t.y.-c). Çözülmüş en büyük örnek pla85900, Bell Laboratuvarları'nda 1980'lerin sonunda ortaya çıkan bir yonga tasarımı (VLSI) uygulamasının 85 900 noktasıdır; Concorde ile 2005/06'da çözüldü (Applegate vd., 2006; University of Waterloo, t.y.-b). Bu çözücüler kaba kuvvet değil, budama ve alt sınır fikirlerinin (Bölüm 3.6) ileri biçimleridir.

## 1.1 Öğrenme çıktıları

- **ÖÇ 14.1** Bir problemi uygun algoritma tasarım stratejisiyle (kaba kuvvet, böl ve fethet, açgözlü, DP, graf algoritmaları, karma) eşleştirir ve seçimini gerekçelendirir. (Bölüm 3.1–3.2, 5.1, Atölye 1)
- **ÖÇ 14.2** Problemleri bilinen problemlere indirger (ör. labirent → BFS, ders planı → topolojik sıralama, sınav programı → graf boyama). (Bölüm 3.3, 5.3, Alıştırma 14.15)
- **ÖÇ 14.3** P, NP ve NP-tam kavramlarını sezgisel düzeyde açıklar; NP-zor problemlerde sezgisel ve yaklaşık yöntemlerin rolünü tartışır. (Bölüm 3.4–3.6, 4)
- **ÖÇ 14.4** Bir uygulamanın performansını ölçer, profil çıkararak darboğazı bulur ve algoritmik iyileştirmeyi deneysel olarak belgeler. (Bölüm 3.7–3.8, 5.4, 6, 9)
- **ÖÇ 14.5** Dönem boyunca geliştirilen algoritmik modülleri bütünleştiren final projesini tasarlar, test eder, belgeler ve sunar. (Bölüm 5.6, 10, Atölye 3)

# 2. Geçen Haftadan ve Dönemden Bağlantı

Bu hafta bütün dönemi kullanıyoruz; en çok dört haftaya yaslanıyoruz: **H3** (permütasyon üretme ve geri izleme = gezgin satıcının kaba kuvveti), **H11–H12** (graflar ve açgözlü algoritmalar = indirgemenin en sık hedefi), **H13** (DP = Held–Karp; alt küme toplamı = sözde polinom zaman) ve **H1–H2** (ölçüm ve büyüme sınıfları = strateji seçiminin dili).

> TAHMİN ET · **Geri çağırma: dört soru (yanıtlar aşağıda)**
>
> **S1 (H13).** 0/1 sırt çantası DP'si n = 100 ve W = $10^9$'da kaç hücre doldurur? Θ(n·W) neden "polinom" sayılmaz? **S2 (H3).** 8-vezirde budamasız ve budamalı arama ağaçlarında kaç düğüm vardır?
>
> **S3 (H11–H12).** Ağırlıksız bir labirentte en kısa yol hangi algoritmayla bulunur; ağırlıklar negatif değilse ne değişir? **S4 (H13).** Fibonacci'de n = 35 için saf özyineleme ve memoization kaç çağrı yapar?

**Yanıtlar.** S1: (n + 1)(W + 1) ≈ $10^{11}$ hücre; W'yi yazmak 30 bit sürer ama iş $2^{30}$ ile orantılıdır: süre girdinin **bit uzunluğunda üsteldir** — sözde polinom (Hata Avcısı 2). S2: 19 173 961 ↔ 2 057 düğüm, 9 321 kat az (H3 deneyi). S3: BFS (katman = kenar sayısı); negatif olmayan ağırlıklarda Dijkstra. Bu hafta labirenti BFS'ye **indirgeyeceğiz** (Örnek 7). S4: 29 860 703 ↔ 69 çağrı (süre katı 199 725). Held–Karp aynı fikri gezgin satıcıya uygular: (n − 1)! tur yerine n·$2^n$ durum.

# 3. Kavramsal Anlatım: Strateji, İndirgeme ve Hesaplanabilirliğin Sınırları

## 3.1 Dönemin haritası: strateji aileleri

Bir **algoritma tasarım stratejisi** (design strategy), birçok algoritmayı üreten bir düşünme kalıbıdır: kaba kuvvet "hepsini dene", böl ve fethet "bağımsız parçalara ayır", açgözlü "her adımda en iyisini al", DP "her alt problemi bir kez çöz" der. Dönemin 14 haftasındaki **74 kimlik kartını** strateji alanlarına göre programla sınıflandırdık; her kart tam bir aileye düştü (Şekil 14.1; aile başına kart: kaba kuvvet ve üretme 8, geri izleme ve budama 2, azalt ve fethet 5, artımlı yöntem 4, böl ve fethet 7, veri yapısıyla çözüm 17, karma 10, graf modelleme ve dolaşma 5, açgözlü 7, dinamik programlama 7, sezgisel ve yerel arama 2). Her ailenin "nasıl tanınır?" **işareti** (problem tanımında aranacak ipucu) ve dönemdeki örnekleri başvuru tablosu olarak Ek B'dedir (Tablo 14.19). Strateji bir **kanıt borcu** da getirir: açgözlü değiş tokuş argümanı (H12), DP en iyi alt yapı (H13) ister; sezgisel garanti vermez, ölçümle değerlendirilir.

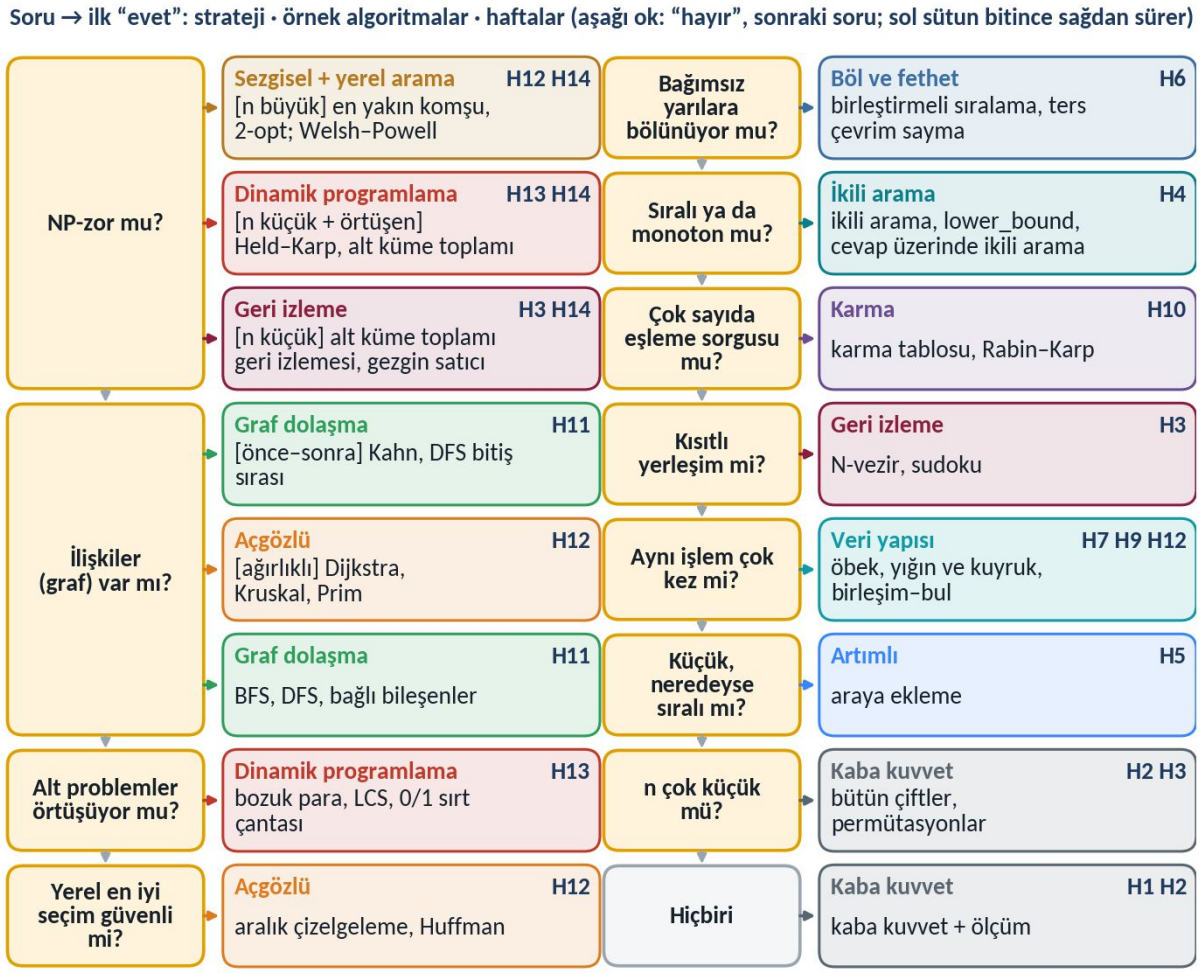
**Şekil 14.1**
*Strateji karşılaştırma matrisi: on bir aile, en iyiyi garanti edip etmediği, tipik karmaşıklık, gerekli özellik ve dönemdeki kart sayısı (74 kart, programla sınıflandırıldı)*

| Strateji | En iyiyi garanti | Tipik karmaşıklık | Gerekli özellik | Kart | Haftalar |
|---|---|---|---|---|---|
| Kaba kuvvet | ✓ evet | $\Theta(n^2)$, $\Theta(2^n)$, $\Theta(n!)$ | yok (yalnız sonlu aday kümesi) | 8 | H1–H4, H10, H14 |
| Geri izleme | ✓ evet | en kötü üstel | kısmi çözümde denetlenebilir kısıt | 2 | H3 |
| İkili arama | ✓ evet | $\Theta(\log n)$ yoklama | sıralılık ya da monotonluk | 5 | H1, H4 |
| Artımlı | ✓ evet | çoğunlukla $\Theta(n^2)$ | önek değişmezi | 4 | H5 |
| Böl ve fethet | ✓ evet | çoğunlukla $\Theta(n \log n)$ | bağımsız (örtüşmeyen) alt problemler | 7 | H2, H6, H8 |
| Veri yapısı | ✓ evet | işlem başına $O(1)$ ya da $O(\log n)$ | uygun soyut veri tipi | 17 | H7–H9, H12 |
| Karma | ✓ evet | beklenen $\Theta(1)$ / sorgu | iyi bir karma fonksiyonu ya da küçük değer aralığı | 10 | H2, H7, H10 |
| Graf dolaşma | ✓ evet | $\Theta(V + E)$ | problemi graf olarak modellemek | 5 | H11 |
| Açgözlü | ± koşullu | çoğunlukla $\Theta(n \log n)$ ya da $O((V + E) \log V)$ | açgözlü seçim özelliği + en iyi alt yapı | 7 | H12 |
| Dinamik programlama | ✓ evet | durum sayısı × durum başına geçiş | örtüşen alt problemler + en iyi alt yapı | 7 | H2, H13, H14 |
| Sezgisel + yerel arama | ✗ hayır | polinom | ölçü: yaklaşım oranı, alt sınır | 2 | H14 |

## 3.2 Strateji seçimi için karar soruları

Seçim iki bilgiyle başlar: **girdi boyutu** ve **süre bütçesi**. H2'nin uygulanabilirlik tablosu (saniyede $10^8$ işlem, 1 s) en büyük n'yi verir: n! için 11, $2^n$ için 26, $n^3$ için 464, $n^2$ için 10 000, n log n için 4 523 071, n için 100 000 000. Tersinden: $n \le 10$ permütasyonlara, $n \approx 20$ alt kümelere, $n \approx 10^4$ karesel bir algoritmaya izin verir; $n \approx 10^6$ n log n ya da daha iyisini ister. Kalan sorular problemin **yapısı** hakkındadır (Şekil 14.2). Ağaç kesin bir kural değil, bir **sıralama** önerir: 8 şehirde kısa tur hem “n küçük” hem “NP-zor” sorusuna uyar; ağaç, n büyüyecekse sezgiseli, küçük kalacaksa kesin yöntemi önerir. Örnek 1 on kısa problemi bu mantıkla çözer (Bölüm 5.1).

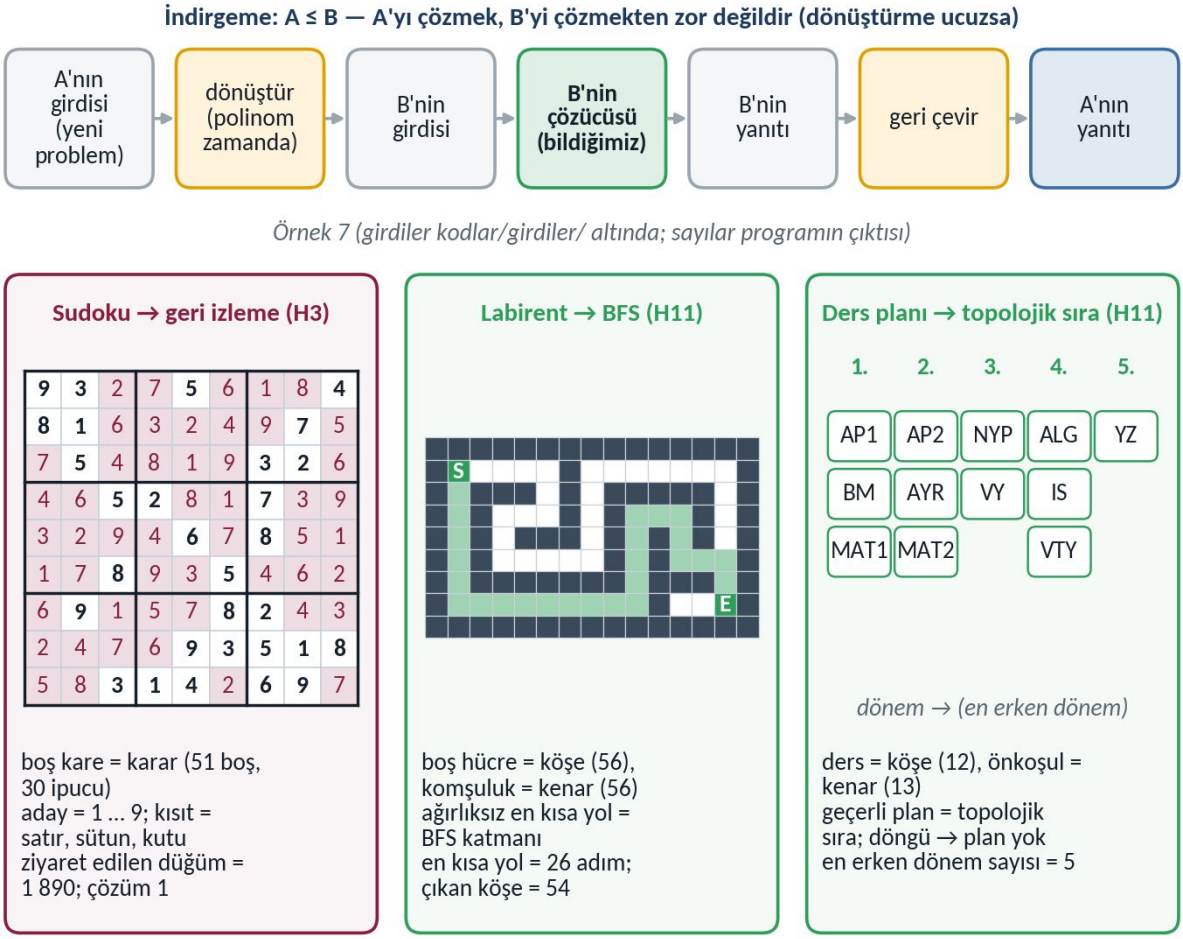
**Şekil 14.2**
*Strateji karar ağacı: sorular yukarıdan aşağı sorulur, ilk "evet" stratejiyi ve ilgili haftaları verir (web'deki strateji seçici sihirbazı aynı kuralları kullanır ve 26 örnek problemle sınanmıştır)*




## 3.3 Problem indirgeme ve modelleme: "Bunu daha önce gördüm mü?"

**İndirgeme**, A problemini çözmek için girdisini B'nin girdisine çevirmek, B'nin bilinen çözücüsünü çalıştırmak ve sonucu geri çevirmektir (Şekil 14.3); çeviri ucuzsa A, B'den zor değildir. Örnek 7 üç modelleme yapar: **sudoku → geri izleme** (boş kare = karar, aday 1..9, kısıt = satır/sütun/kutu; 30 ipucu, 51 boş kare, 1 890 düğümde tek çözüm), **labirent → BFS** (boş hücre = köşe, komşu = kenar; 9 × 15 ızgara, 56 köşe; en kısa yol 26 adım) ve **ders planı → topolojik sıralama** (ders = köşe, önkoşul = kenar; 12 ders 5 dönemde; kurgusal katalog). Final projesinin modülleri de indirgemedir: **sınav programı → graf boyama** (sınav = köşe, ortak öğrenci = kenar, renk = gün; Alıştırma 14.15), **ders seçimi → 0/1 sırt çantası**, **yinelenen numara → karma kümesi**. Kuramda indirgemenin ikinci yönü önemlidir: B kolaysa A da kolaydır, **A zorsa B de zordur**; NP-tamlık bu yönle yayılır.

**Şekil 14.3**
*İndirgeme şeması: problemi modelle (köşe, kenar, karar, kısıt), bilinen çözücüyü çalıştır, sonucu geri çevir; Örnek 7'nin üç indirgemesi gerçek girdileriyle*



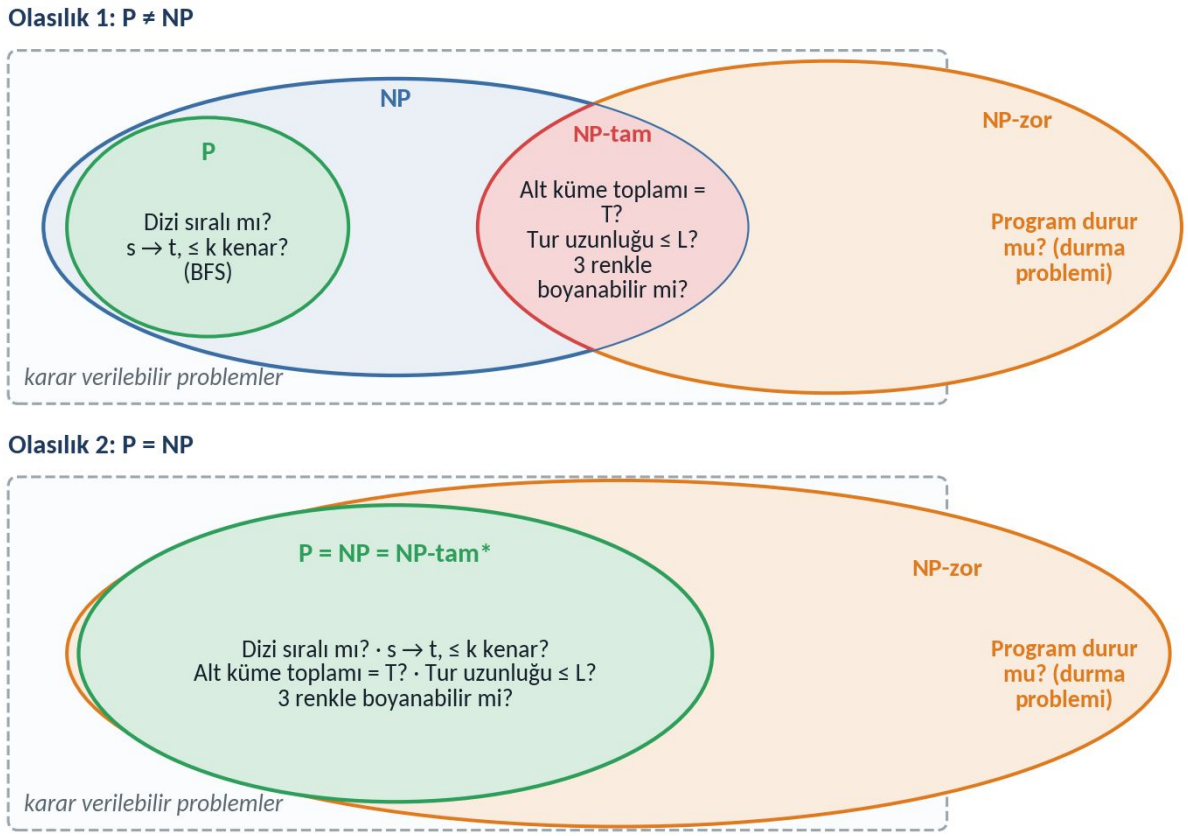

## 3.4 P, NP ve NP-tamlık (sezgisel düzeyde)

Kuram **karar problemleriyle** (decision problem) çalışır: "en kısa tur nedir?" yerine "uzunluğu en çok L olan tur var mı?". Karar sürümünü hızlı çözen bir yöntem L üzerinde ikili aramayla en iyi değeri de bulur (H4).

- **P:** polinom zamanda (bazı k için $O(n^k)$) **çözülebilen** karar problemleri: sıralı mı, s'den t'ye yol var mı, uzunluğu ≤ L yol var mı (Dijkstra)?
- **NP:** "evet" yanıtı bir **sertifikayla** polinom zamanda **doğrulanabilen** karar problemleri. Gezgin satıcıda sertifika bir şehir sırasıdır; doğrulayıcı her şehrin bir kez geçtiğini ve uzunluğun ≤ L olduğunu O(n) adımda denetler (Alıştırma 14.11). N, "nondeterministic"tir: doğru sertifikayı "tahmin edebilen" makine onu polinom zamanda doğrular.
- **P ⊆ NP:** çözebilen doğrulayabilir de. "NP = polinom olmayan" bu yüzden **yanlıştır**: "dizi sıralı mı?" gibi P'deki her karar problemi NP'dedir (Hata Avcısı 1).
- **NP-tam:** NP'de olan ve NP'deki **her** problemin polinom zamanda indirgendiği problemler; birinin polinom algoritması bulunursa hepsi çözülür. İlki SAT'tır (Cook, 1971); Karp (1972) 0–1 tamsayı programlama, Hamilton devresi, kromatik sayı (graf boyama) ve alt küme toplamı dahil 21 problemi ekledi. Hamilton devresi gezgin satıcının karar sürümüne indirgenir, bu yüzden o da NP-tamdır; sırt çantasının karar sürümü ve 3 renkle boyama da NP-tamdır.

- **NP-zor:** en az NP-tam kadar zor, ama NP'de olması gerekmeyen problemler: en kısa turun **kendisi** (en iyileme) ya da durma problemi.

**Şekil 14.4**
*P ⊆ NP ve NP-tam: iki olası dünya (P = NP sorusu AÇIKTIR); NP-zor bölge NP'nin dışına taşar; durma problemi karar verilebilir problemlerin dışındadır*




**Hangisi doğru, bilinmiyor: P = NP sorusu açıktır (Clay Milenyum Problemi, 2000). Kesin olan: P ⊆ NP.**
** "her zaman evet" ve "her zaman hayır" gibi iki önemsiz problem dışında. Problemler Hata Avcısı 1'in düzeltilmiş çıktısından; durma problemi NP-zordur ama karar verilemez (NP'de değildir).*

**Doğrulama ile bulma ölçülebilir biçimde farklıdır** (Örnek 6): a = {13, 7, 21, 4, 18, 9}, hedef 38 için sertifika {0, 1, 4} 4 adımda kabul edilir; bulmak için geri izleme 45 düğüm (en kötü 127), DP 273 hücre gezer. n = 30 "evet" örneğinde doğrulama 11 adım, geri izleme 56 305 düğüm, DP 192 882 hücredir; n = 18 "hayır" örneğinde gösterilecek sertifika yoktur ve geri izleme 327 949 düğüm gezer. **Sözde polinom zaman:** alt küme DP'si Θ(n·T) zamanlıdır; T'nin **değerinde** polinomdur, ama T'yi yazmak $b \approx \log_2 T$ bit sürdüğü için girdinin **uzunluğunda** üsteldir. Hata Avcısı 2 bunu 0/1 sırt çantasında ölçer ($n = 20$, $W = 2^b - 1$): b = 24'te tablo 306 017 383 hücre işler, b = 40'ta ayrılamaz. Bu yüzden bu DP P = NP'yi kanıtlamaz; ama değer küçükse (AKTS sınırı 45) problem pratikte kolaydır (H13).

**P = NP mi?** Kimse bilmiyor (Şekil 14.4); çoğu araştırmacı P ≠ NP olduğunu düşünür, ama bu bir kanıt değildir. P ≠ NP ise NP-tam problemlerin hiçbirinin polinom algoritması yoktur; P = NP ise üç sınıf çakışır. Mühendis için sonuç aynıdır: problemin NP-tam olduğunu anlamak "daha iyi bir kesin algoritma aramayı bırak, Bölüm 3.6'nın yöntemlerine geç" demektir.

## 3.5 Hesaplanabilirliğin sınırı: durma problemi

Bazı problemler yavaş değil, **çözümsüzdür**. Durma problemi (halting problem): bir P programı ve x girdisi verildiğinde P(x) bir gün durur mu? Turing, 1936'nın Kasım–Aralık aylarında parçalar hâlinde yayımlanan (cilt yılı 1937) makalesinde bu tür sorulara her girdide doğru yanıt veren genel bir yöntem olmadığını gösterdi (Turing, 1937). Fikir bir çelişkidir: her programda doğru yanıt veren D(P, x) "durur mu?" fonksiyonu yazılabilseydi, T(P) = "D(P, P) evet derse sonsuz döngüye gir, hayır derse dur" programını kurardık. T(T) durur mu? D evet derse T sonsuza dek çalışır, hayır derse durur: D her iki durumda da yanılır. Bu yüzden derleyiciler ve statik çözümleyiciler "bu döngü her girdide biter mi?" sorusunu **genel olarak** yanıtlayamaz; uyarıları ya yanlış alarm verir ya da bazı hataları kaçırır. Boşluğu testler ve kâhinler doldurur.

## 3.6 NP-zor problemlerle başa çıkmak

1. **Küçük n: kaba kuvvet + budama.** Örnek 2 n = 10'da 362 880 tur dener; dal ve sınır (branch and bound) kısmi uzunluk en iyiyi aşınca dalı keser.
2. **Sözde polinom ya da akıllı üstel DP.** Alt küme ve sırt çantası değer küçükse kolaydır (H13); Held–Karp n = 16'da 1 720 335 geçiş yapar, kaba kuvvetin $6{,}538 \times 10^{11}$ turu yerine (Örnek 5).
3. **Açgözlü sezgiseller:** en yakın komşu (Örnek 3), en ucuz ekleme (Alıştırma 14.12), Welsh–Powell (H12) — hızlı, garantisiz.
4. **Yerel arama:** çözümü küçük değişikliklerle iyileştir; hiçbir hamle kısaltmıyorsa **yerel en iyi** (Örnek 4). 2-opt'u Croes (1958) tanıttı; Lin ve Kernighan (1973) genelleştirdi.
5. **Yaklaşık algoritmalar:** en iyiye oran için **kanıtlı** üst sınır verir. Yaklaşım oranı ρ = bulunan / en iyi ≥ 1; üçgen eşitsizliğinde en küçük kapsayan ağaçtan kurulan tur ρ ≤ 2 sağlar (Cormen vd., 2022). Sezgisellerin oranı ise ölçülür: en yakın komşunun 100 örnekteki en kötü oranı 1,4869 (Bölüm 4.2).

## 3.7 Performans mühendisliği: önce ölç

Knuth'un (1974) uyarısı tam hâliyle okunmalıdır: "We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%." (s. 268) — küçük verimlilikleri zamanın %97'sinde unutun, ama kritik %3'ü kaçırmayın. Sorun o %3'ü **tahminle** bulmaya çalışmaktır (Hata Avcısı 3): kritik yeri **profil** gösterir (gprof ve perf, Python'da cProfile; Kaputun Altında 1, Örnek 8). İyileştirme sırası Bölüm 9'da, gerekçesi sayılarla Bölüm 6'dadır.

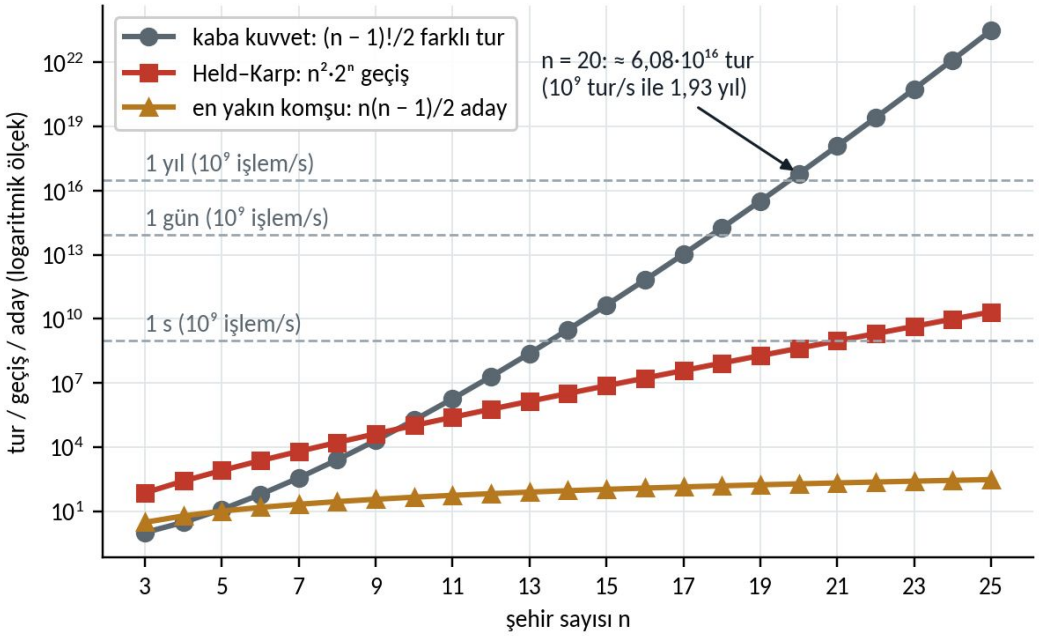
## 3.8 Kaputun Altında

> KAPUTUN ALTINDA · **1 · Profilleyici nasıl çalışır? Örnekleme, araçlama ve alev grafiği**
>
> **Örnekleme** (sampling) profilleyicisi programı sabit aralıklarla durdurup işlemcinin hangi fonksiyonda olduğuna ve çağrı yığınına bakar; örnek payı zaman payının kestirimidir. Linux `perf` böyle çalışır (bu makinede yok; mantığını K02 benzetir): gerçek payı %90,1 olan `yinelenen_ciftler`'i 10 birimde bir örnekleyen profilleyici 82/91 örnekle, 200 birimde bir örnekleyen yalnız 5 örnekle (4/5) bulur: hata ≈ 1/√(örnek sayısı), ama en sıcak fonksiyon yine doğru. **Araçlama** (instrumentation): `gcc -pg` fonksiyon girişlerine sayaç ekler; `gprof` çağrı sayılarını kesin, süreleri örneklemeyle (0,01 s adım) verir. Python'un `cProfile`'ı her çağrıyı kaydeder: sayılar kesin, ölçüm yükü yüksek. Sayılan yığınlar "katlanmış yığın" satırlarına yazılır; **alev grafiği** (flame graph) bunlardan çizilir: genişlik = örnek sayısı, yükseklik = çağrı derinliği (Şekil 14.9, Bölüm 5.4).

> KAPUTUN ALTINDA · **2 · Algoritmik iyileştirme ile mikro iyileştirmenin ölçeği**
>
> Dönem deneyinde (Bölüm 6) derleyici (-O0 → -O2) ortanca 2,73 kat, dil (Python → C) ortanca 51,7 kat hızlandırdı; algoritma değişikliği ise aramayı $10^6$ kayıtta 49 042 kat (doğrusal → karma), ders seçimini n = 35'te 280 994 kat (geri izleme → DP). İlk ikisi **sabit çarpandır**, üçüncüsü n ile **büyür** ($\Theta(n^2) \to \Theta(n \log n)$ geçişinde n/log n gibi): önce algoritma.

> KAPUTUN ALTINDA · **3 · Gezgin satıcıda durum uzayının büyüklüğü**
>
> Şehir 0 sabitlenince (n − 1)! sıra vardır; tur ile tersi aynı uzunlukta olduğu için **farklı** tur sayısı (n − 1)!/2'dir (K01, `uint64_t` ile n ≤ 21'e dek kesin: 20! 64 bite sığar, 21! sığmaz). Saniyede $10^9$ turla kaba kuvvet n = 10'da 181 µs, n = 13'te 240 ms, n = 16'da 10,9 dk, n = 20'de 6,082 × $10^{16}$ ≈ 6 × $10^{16}$ turla 1,93 yıl sürer; Held–Karp'ın $n^2 \cdot 2^n$ geçişi n = 20'de 4,194 × $10^8$'dir (Şekil 14.5). Held–Karp'ın sınırını bellek çizer (Bölüm 4.4).

**Şekil 14.5**
*Durum uzayı büyümesi (logaritmik eksen, K01): (n − 1)!/2 farklı tur, Held–Karp'ın $n^2 \cdot 2^n$ geçişi ve en yakın komşunun n(n − 1)/2 aday karşılaştırması (2-opt'un bir tam taraması da bu mertebededir); yatay çizgiler $10^9$ işlem/s'de 1 saniye, 1 gün ve 1 yıl*




# 4. Algoritmalar: Gezgin Satıcı İçin Dört Yöntem

Dört algoritma aynı problemi dört stratejiyle çözer: **kaba kuvvet** (kesin, $\Theta(n!)$), **en yakın komşu** (açgözlü sezgisel, $\Theta(n^2)$), **2-opt** (yerel arama) ve **Held–Karp** (DP, $\Theta(n^2 \cdot 2^n)$). **Problem:** n şehir ve simetrik uzaklık d(i, j) verildiğinde, şehir 0'dan başlayıp her şehre bir kez uğrayan ve 0'a dönen en kısa turu (Hamilton devresini) bul. Veri: Örnek 2'nin el örneği (0(0, 0), 1(6, 1), 2(1, 4), 3(5, 5), 4(3, 2)) ve ortak LCG'nin şehirleri (0..99). Uzaklık Öklid'dir; eşitlik kararları tamsayı **kare** uzaklıkla ya da EPS = $10^{-9}$ payıyla verilir, böylece üç dil aynı turu seçer.

## 4.1 Kaba kuvvet: bütün turlar (Örnek 2)

**Fikir.** Haritaya bütün güzergâhları çizip cetvelle ölçmek: sıkıcı ama yanılmaz. Şehir 0 sabitlenince kalan n − 1 şehrin her **permütasyonu** bir turdur; H3'ün üreticisi (`kullanildi[]` ile seç – dene – geri al)

bunları sözlük sırasıyla üretir, kısmi uzunluk her adımda bir kenarla taşınır. **Elle izleme:** beş şehirde 4! = 24 tur vardır ve her tur tersiyle aynı uzunluktadır: 12 farklı tur (Tablo 14.1).

**Tablo 14.1**
*Örnek 2: beş şehirde 24 tur, 12 çift (sözlük sırası numaraları); en kısa 19,14*

| # | Tur · tersi | Uzunluk | # | Tur · tersi | Uzunluk |
|---|---|---|---|---|---|
| 1·24 | 0-1-2-3-4-0 · 0-4-3-2-1-0 | 23,25 | 7·23 | 0-2-1-3-4-0 · 0-4-3-1-2-0 | 21,29 |
| 2·18 | 0-1-2-4-3-0 · 0-3-4-2-1-0 | 25,42 | 8·17 | 0-2-1-4-3-0 · 0-3-4-1-2-0 | 23,79 |
| 3·22 | 0-1-3-2-4-0 · 0-4-2-3-1-0 | 20,76 | 9·20 | 0-2-3-1-4-0 · 0-4-1-3-2-0 | 19,14 |
| 4·12 | 0-1-3-4-2-0 · 0-2-4-3-1-0 | 20,76 | 11·14 | 0-2-4-1-3-0 · 0-3-1-4-2-0 | 21,31 |
| 5·16 | 0-1-4-2-3-0 · 0-3-2-4-1-0 | 23,27 | 13·21 | 0-3-1-2-4-0 · 0-4-2-1-3-0 | 23,46 |
| 6·10 | 0-1-4-3-2-0 · 0-2-3-4-1-0 | 21,10 | 15·19 | 0-3-2-1-4-0 · 0-4-1-2-3-0 | 23,79 |

En kısa uzunluk 19,14: 0-2-3-1-4-0 ve tersi. Kaba kuvvet eşitlikte **ilk** bulunanı korur (`toplam < en_iyi − EPS`): 0-2-3-1-4-0; 65 çağrı.

SÖZDE KOD · Gezgin satıcı, kaba kuvvet (başlangıç: en_iyi ← ∞; DENE(1, 0))

```
algoritma DENE(k, kısmi)      ▷ yol[0..k−1] dolu (yol[0] = 0); kısmi = uzunluğu
    eğer k = n ise
        toplam ← kısmi + d[yol[n−1]][0]          ▷ dönüş kenarı
        eğer toplam < en_iyi − EPS ise en_iyi ← toplam;  en_iyi_tur ← yol
        döndür
    s ← 1'den n−1'e kadar her s için
        eğer kullanıldı[s] değil ise
            kullanıldı[s] ← doğru;  yol[k] ← s                ▷ seç
            DENE(k + 1, kısmi + d[yol[k−1]][s])             ▷ dene
            kullanıldı[s] ← yanlış                           ▷ geri al
```

**Doğruluk.** Değişmez: k. düzeyde yol[0..k−1] birbirinden farklı şehirlerden oluşan kısmi bir turdur ve kismi bu yolun uzunluğudur; en_iyi o ana dek tamamlanan turların en kısasıdır. Üretici her permütasyonu tam bir kez üretir (H3); en_iyi bütün turların en küçüğüdür. **Analiz.** (n − 1)! yaprak, ≈ e·(n − 1)! çağrı: n = 10'da 362 880 tur, 986 410 çağrı; uzunluk artımlı taşındığı için tur başına iş O(1) (kimlik kartındaki Θ(n!) tur başına Θ(n) toplamı varsayar); ek bellek O(n).

**Tablo 14.2**
*Gezgin satıcı (kaba kuvvet) kimlik kartı*

| ALGORİTMA KİMLİK KARTI **Gezgin satıcı: kaba kuvvet (bütün turlar)** *(traveling salesman problem: brute force (exhaustive search))* | |
|---|---|
| **Strateji** | Kaba kuvvet / geri izlemeyle üretme (H3): şehir 0 sabitlenir, kalan n − 1 şehrin bütün permütasyonları kullanildi[] dizisiyle üretilir, her turun uzunluğu hesaplanır ve en kısası tutulur |
| **Girdi önkoşulu** | Tam graf; simetrik uzaklık d(i, j) = d(j, i) (burada tamsayı koordinatlar, Öklid uzaklığı); uygulanabilirlik n ≤ 10–11 |
| **Zaman (en iyi · ortalama · en kötü)** | Θ(n!) · Θ(n!) · Θ(n!) |
| **Ek bellek** | O(n) |

| | |
|---|---|
| **Özellikler** | 5 şehirli el örneğinde 24 = 4! tur; en kısa 0-2-3-1-4-0 = 19,14 ve tersi 0-4-1-3-2-0 aynı uzunlukta (farklı tur sayısı 12 = 4!/2). LCG örneğinde (tohum 42) n = 10: 362 880 tur, 986 410 çağrı, en iyi 324,21. Durum uzayı (n − 1)!/2: n = 20'de ≈ 6,08 × $10^{16}$ tur; saniyede $10^{9}$ turla ≈ 1,93 yıl |
| **Değişmez / doğruluk** | k. düzeyde yol[0..k−1] birbirinden farklı şehirlerden oluşan kısmi bir turdur ve kismi bu yolun uzunluğudur; en_iyi o ana dek tamamlanan turların en kısasıdır |
| **Kütüphane karşılığı** | C: yok (elle) · C++: `std::next_permutation` (sözlük sırasında sonraki permütasyon; örnekte elle) · Python: `itertools.permutations` (örnekte elle; yalnızca kâhin) |
| **Ne zaman kullanılır** | n küçükken (≈ 10) kesin cevap gerektiğinde; sezgisellerin kalitesini ölçen kâhin olarak (Örnek 3–4) |
| **Ne zaman kullanılmaz** | n ≳ 12: (n − 1)! patlar; daha büyük n'de Held–Karp (n ≤ 16–18) ya da sezgisel + yerel arama |

**C** `c/H14_02_gs_kaba_kuvvet.c · satır 54–84`

```c
static void dene(int k, double kismi)
{
    cagri_sayisi++;
    if (k == n) {                                   /* yaprak: tam tur */
        if (yalniz_yari && yol[1] > yol[n - 1]) {
            return;                                 /* tersi zaten sayıldı */
        }
        double toplam = kismi + d[yol[n - 1]][0];   /* dönüş kenarı */
        tur_sayisi++;
        if (yaz_mi) {
            printf("    %3lld  ", tur_sayisi);
            tur_yaz(yol);
            printf("  %7.2f\n", toplam);
        }
        if (toplam < en_iyi - EPS) {
            en_iyi = toplam;
            for (int i = 0; i < n; i++) {
                en_iyi_tur[i] = yol[i];
            }
        }
        return;
    }
    for (int s = 1; s < n; s++) {
        if (!kullanildi[s]) {
            kullanildi[s] = 1;                      /* seç */
            yol[k] = s;
            dene(k + 1, kismi + d[yol[k - 1]][s]);  /* dene */
            kullanildi[s] = 0;                      /* geri al */
        }
    }
}
```

`yalniz_yari` bayrağı simetriyi sayar: yalnız yol[1] < yol[n − 1] olan turlar (n − 1)!/2'dir. **Üç dilde** (seç – dene – geri al; hardal satırlar C'den anlamca ayrılan satırlardır, açıklamaları ve fonksiyonların tamamı Ek A'da):

C++ **H14_02_gs_kaba_kuvvet.cpp · satır 75–82**

```cpp
    for (int s = 1; s < n; s++) {
        if (!kullanildi[s]) {
            kullanildi[s] = 1;                      // seç
            yol[k] = s;
            dene(k + 1, kismi + d[yol[k - 1]][s]);  // dene
            kullanildi[s] = 0;                      // geri al
        }
    }
```

Python **H14_02_gs_kaba_kuvvet.py · satır 64–69**

```python
    for s in range(1, n):
        if not kullanildi[s]:
            kullanildi[s] = 1                       # seç
            yol[k] = s
            dene(k + 1, kismi + d[yol[k - 1]][s])   # dene
            kullanildi[s] = 0                       # geri al
```

**Test.** Kaba kuvvet bu haftanın **kâhinidir**; kendisi elle sayılan tur sayılarıyla (n = 4: 3, n = 5: 12 farklı tur) ve n ≤ 10'da Held–Karp'la sınanır. **Deney** (LCG, tohum 42): n = 4: 6; n = 5: 24; n = 6: 120; n = 7: 720; n = 8: 5 040; n = 9: 40 320; n = 10: 362 880 tur. **Varyantlar:** dal ve sınır (kısmi ≥ en_iyi ise kes); `std::next_permutation`, `itertools.permutations` (Alıştırma 14.16).

## 4.2 En yakın komşu sezgiseli (Örnek 3)

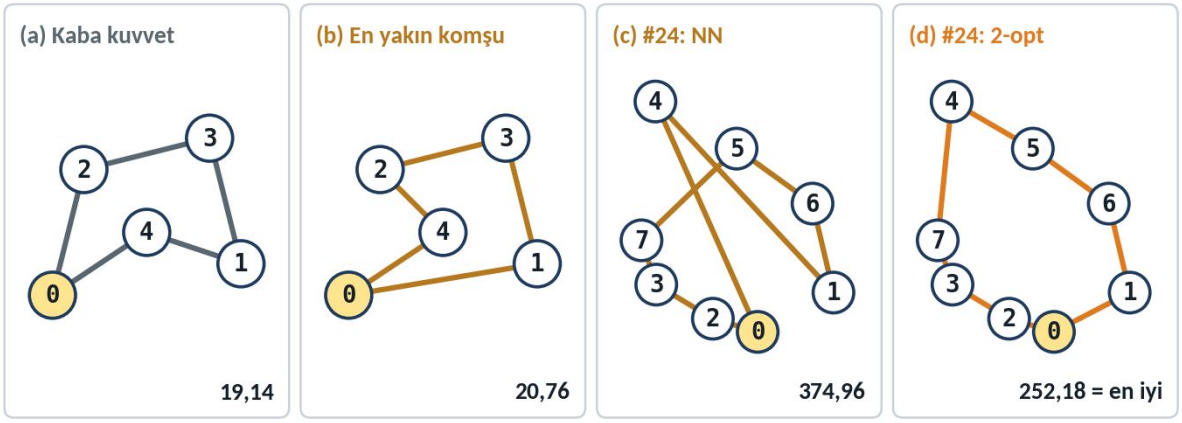
**Fikir.** Yorgun turist her akşam yalnız şunu sorar: "görmediğim şehirlerden en yakını hangisi?" Plan yapmaz, geri dönmez: bir **açgözlü** kural (H12), hızlı ama **garantisiz**. Eşitlik kuralı: kare uzaklık, eşitlikte küçük indis.

**Tablo 14.3**
*Örnek 3: en yakın komşu, şehir 0'dan (beş şehir); tur 0-4-2-3-1-0 = 20,76*

| Adım | Son şehir | Adaylar (şehir: $d^2$) | Seçilen |
|---|---|---|---|
| 1 | 0 | 1: 37  2: 17  3: 50  4: 13 | 4 |
| 2 | 4 | 1: 10  2: 8  3: 13 | 2 |
| 3 | 2 | 1: 34  3: 17 | 3 |
| 4 | 3 | 1: 17 | 1 |

**En yakın komşu turu en iyi mi?** Hayır: en iyi tur 19,14 uzunluğundadır (Tablo 14.1); oran 1,085. İlk adımda 0'a en yakın şehir 4'tür ($d^2$ = 13); en iyi tur ise 0'dan 2'ye gider ($d^2$ = 17) ve 4'ü sona saklar: yerel olarak doğru seçim küresel olarak yanlıştır (Şekil 14.6).

**Şekil 14.6**
*Gezgin satıcı turları: beş şehirde (a) kaba kuvvetin en iyi turu (19,14; 24 tur) ve (b) en yakın komşu (20,76; oran 1,085); Örnek 3'ün en kötü örneğinde (#24) (c) en yakın komşu (374,96) ve (d) 4 hamlelik 2-opt sonrası (252,18 = en iyi); birbirine çok yakın 3 ve 7 numaralı şehirler çizimde biraz ayrık gösterilmiştir*




SÖZDE KOD · En yakın komşu sezgiseli

```
algoritma EN-YAKIN-KOMŞU(d, baş)
    tur[0] ← baş;  ziyaret[baş] ← doğru
    k ← 1'den n−1'e kadar her k için
        son ← tur[k−1];  seç ← −1
        s ← 0'dan n−1'e kadar her s için
            eğer ziyaret[s] değil ve (seç = −1 ya da d²(son, s) < d²(son, seç)) ise
seç ← s
        tur[k] ← seç;  ziyaret[seç] ← doğru
    döndür tur
```

**Doğruluk** (geçerlilik, en iyilik değil). Değişmez: k. adımdan sonra tur[0..k] farklı şehirlerdir ve ziyaret[] tam olarak onları işaretler; n − 1 adımdan sonra tur bir permütasyondur. En iyilik kanıtlanamaz, çünkü yanlıştır (karşı örnek yukarıda). **Analiz.** k. adımda n − k aday: toplam $n(n-1)/2 = \Theta(n^2)$ (sayaç: n = 10: 45; n = 20: 190; n = 40: 780; n = 80: 3 160; n = 160: 12 720); bellek O(n).

**Tablo 14.4**
*En yakın komşu sezgiseli kimlik kartı*

| ALGORİTMA KİMLİK KARTI | **En yakın komşu sezgiseli (gezgin satıcı)** *(nearest neighbour heuristic)* |
|---|---|
| **Strateji** | Açgözlü (H12): her adımda henüz gidilmemiş en yakın şehre git; bütün şehirler bitince başlangıca dön |
| **Girdi önkoşulu** | Tam graf; karşılaştırma tamsayı $d^2$ ile; eşit uzaklıkta küçük indisli şehir seçilir (belirlenimci) |
| **Zaman (en iyi · ortalama · en kötü)** | $\Theta(n^2)$ · $\Theta(n^2)$ · $\Theta(n^2)$ |
| **Ek bellek** | O(n) |
| **Özellikler** | 5 şehirli el örneğinde 0-4-2-3-1-0 = 20,76, en iyi 19,14 (oran 1,0850); aday incelemesi 10 = n(n − 1)/2. 100 rastgele 8 şehirli örnekte (tohum 2026) oran en kötü 1,4869, ortalama 1,0964; yalnızca 16/100'de en iyi. Her başlangıç şehri denenip en kısası alınınca ortalama 1,0208, 58/100'de en iyi |
| **Değişmez / doğruluk** | k adım sonra tur[0..k] gidilmiş k + 1 farklı şehirdir ve her tur[i + 1], tur[i]'ye o an gidilmemişler arasında en yakın şehirdir |

| | |
|---|---|
| **Kütüphane karşılığı** | C: yok (elle) · C++: yok (elle) · Python: yok (elle) |
| **Ne zaman kullanılır** | Hızlı bir başlangıç turu gerektiğinde (2-opt gibi yerel aramanın girdisi); n büyük ve kesin çözüm olanaksızken |
| **Ne zaman kullanılmaz** | En iyi tur gerekiyorsa (garanti yok; kötü örnekte %48,7 uzun); sonucu "en iyi" diye raporlamak yanlıştır (Hata Avcısı 4) |

C kodunun çekirdeği, k. adımdaki aday taraması (`en_yakin_komsu`'nun tamamı satır 63–93; `ayrinti` bayrağı izleme tablosunu yazdırır):

**C** `c/H14_03_en_yakin_komsu.c · satır 74–91`

```c
        for (int s = 0; s < n; s++) {
            if (!ziyaret[s]) {
                long long dd = d2(son, s);
                aday_sayisi++;
                if (ayrinti) {
                    printf(" %d:%lld", s, dd);
                }
                if (sec == -1 || dd < en_az) {   /* eşitlikte ilk (küçük) */
                    sec = s;
                    en_az = dd;
                }
            }
        }
        if (ayrinti) {
            printf("  -> %d\n", sec);
        }
        tur[k] = sec;
        ziyaret[sec] = 1;
```

**Üç dilde** (en yakını seç; eşitlikte küçük indis):

**C++** `H14_03_en_yakin_komsu.cpp · satır 81–84`

```cpp
                if (sec == -1 || dd < en_az) {   // eşitlikte ilk (küçük)
                    sec = s;
                    en_az = dd;
                }
```

**Python** `H14_03_en_yakin_komsu.py · satır 64–66`

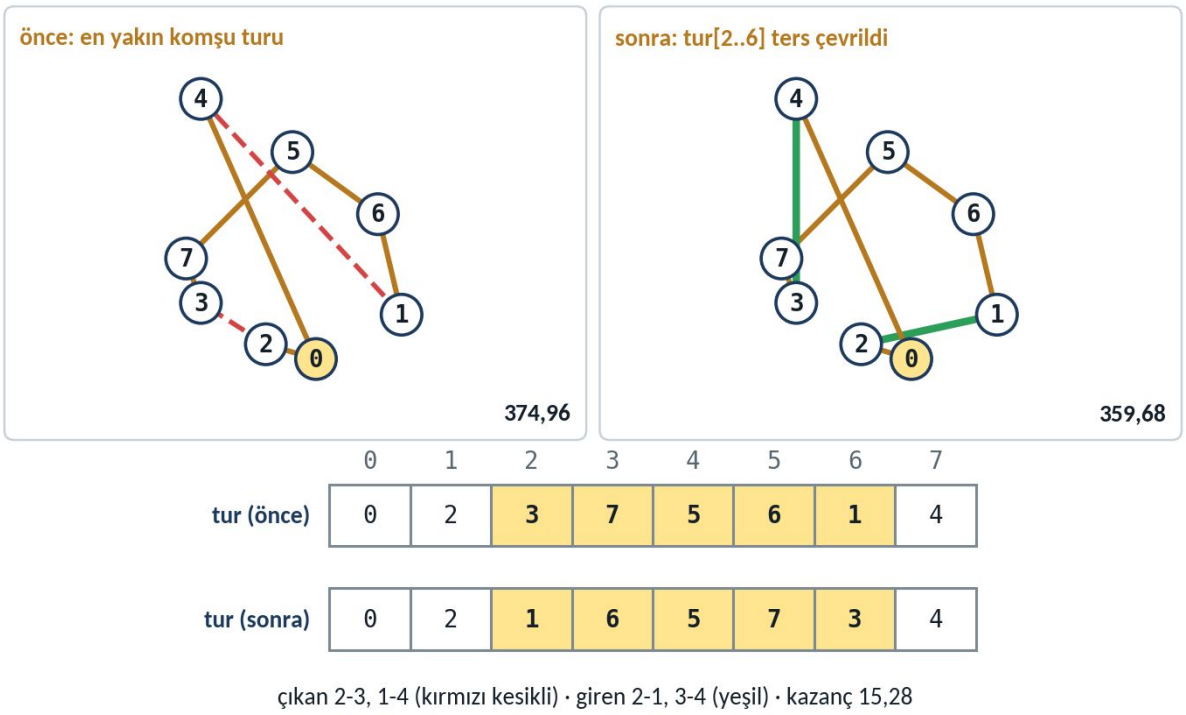
```python
                if sec == -1 or dd < en_az:      # eşitlikte ilk (küçük)
                    sec = s
                    en_az = dd
```

**Test ve deney** (kâhinle oran). Ortak LCG (tohum 2026) 100 örnek üretir (n = 8); NN turu kaba kuvvetle karşılaştırılır: şehir 0'dan oran en kötü 1,4869, ortalama 1,0964, ortanca 1,0774; en iyiyi 16/100 örnekte buldu (dağılım: = 1: 16 · (1,00, 1,05]: 21 · (1,05, 1,10]: 23 · (1,10, 1,20]: 27 · (1,20, 1,30]: 8 · > 1,30: 5). Her şehirden başlatılıp en kısası alınırsa (n kat iş): en kötü 1,1866, ortalama 1,0208, 58/100. **Varyantlar:** en ucuz ve en uzak ekleme (Alıştırma 14.12), açgözlü kenar eşleme.

## 4.3 2-opt yerel arama (Örnek 4)

**Fikir.** Turda iki kenar kesişiyorsa "X"i çözmek turu kısaltır. 2-opt hamlesi (a, b) ve (c, e) kenarlarını çıkarıp (a, c) ve (b, e)'yi ekler: tur dizisinde b'den c'ye kadar olan parçayı **ters çevirmekle** aynıdır (Şekil 14.7). Hamlenin **kazancı** çıkan ve giren kenarların farkıdır: [d(a, b) + d(c, e)] − [d(a, c) + d(b, e)]; pozitifse tur kısalır (kod aynı farkı ters işaretle `kazanc` değişkeninde tutar ve hamleyi `kazanc < -EPS` iken uygular). **İlk iyileştirme** kuralında (i, j) sabit sırayla (i = 1..n − 2, j = i + 1..n − 1) taranır, kısaltan ilk hamle uygulanır ve tarama baştan başlar; hiçbir hamle kısaltmıyorsa tur bir **2-opt yerel en iyisidir**.

**Şekil 14.7**
*2-opt hamlesi (Örnek 4, #24, 1. adım): çıkan kenarlar 2-3, 1-4, giren kenarlar 2-1, 3-4; tur[2..6] ters çevrilir, kazanç 15,28*




**Tablo 14.5**
*Örnek 4: #24'te ilk iyileştirmeli 2-opt (4 hamle, 61 denenen; tam tarama 21 hamle)*

| Adım | (i, j) | Çıkan | Giren | Kazanç | Uzunluk | Tur |
|---|---|---|---|---|---|---|
| 1 | (2, 6) | 2-3, 1-4 | 2-1, 3-4 | 15,28 | 359,68 | 0-2-1-6-5-7-3-4-0 |
| 2 | (2, 7) | 2-1, 4-0 | 2-4, 1-0 | 24,81 | 334,87 | 0-2-4-3-7-5-6-1-0 |
| 3 | (2, 3) | 2-4, 3-7 | 2-3, 4-7 | 9,61 | 325,25 | 0-2-3-4-7-5-6-1-0 |
| 4 | (3, 4) | 3-4, 7-5 | 3-7, 4-5 | 73,08 | 252,18 | 0-2-3-7-4-5-6-1-0 |

**Elle izleme.** NN turu 0-2-3-7-5-6-1-4-0 (374,96); (i, j) = (2, 6)'da çıkan kenarlar 2-3, 1-4, girenler 2-1, 3-4, kazanç 15,28: hamle uygulanır, uzunluk 359,68 olur. Dört hamlede tur 252,18'e iner ve kaba kuvvetin en iyisiyle aynıdır: NN'nin en iyiye göre %48,7 fazlası silinmiştir.

SÖZDE KOD · 2-opt yerel arama (ilk iyileştirme)

```
algoritma İKİ-OPT(tur, d)          ▷ ilk iyileştirme
    değişti ← doğru
    değişti iken
```

```
4         değişti ← yanlış
5         i ← 1'den n−2'ye, j ← i+1'den n−1'e kadar her (i, j) için (değişti
değilken)
6             a, b, c, e ← tur[i−1], tur[i], tur[j], tur[(j+1) mod n]
7             eğer (d[a][c] + d[b][e]) − (d[a][b] + d[c][e]) < −EPS ise
8                 TERS-ÇEVİR(tur, i, j);  değişti ← doğru
9     döndür tur                    ▷ 2-opt yerel en iyisi
```

**Doğruluk ve sonlanma.** Değişmez: her uygulanan hamleden sonra tur geçerli bir permütasyondur ve uzunluk **en az EPS** azalır; sonlu sayıda tur olduğu için algoritma durur ve durduğunda hiçbir 2-opt hamlesi kısaltmaz. Tur **yerel** en iyidir, **genel** en iyi olması gerekmez: #9'da 2-opt 329,99'da durur (yeniden tarama 21 hamlenin hiçbirini kısaltıcı bulmaz), en iyi tur 317,36 (oran 1,0398). **Analiz.** Bir tam tarama $(n-1)(n-2)/2 = \Theta(n^2)$ hamle değerlendirir (her biri O(1)); uygulanan hamlenin ters çevirmesi O(n). Kaç hamle uygulanacağı girdiye bağlıdır: n = 25: 5 iyileştirme, 1 208 denenen; n = 50: 17 iyileştirme, 6 378 denenen; n = 100: 34 iyileştirme, 93 163 denenen.

**Tablo 14.6**
*2-opt yerel arama kimlik kartı*

| ALGORİTMA KİMLİK KARTI | **2-opt yerel arama** *(2-opt local search)* |
|---|---|
| **Strateji** | Yerel arama: turdaki (a, b) ve (c, e) kenarları çıkarılıp (a, c) ve (b, e) eklenir, yani tur[i..j] parçası ters çevrilir; tur kısalıyorsa hamle uygulanır (ilk iyileştirme, sabit (i, j) sırası); hiçbir hamle kısaltmayınca durulur |
| **Girdi önkoşulu** | Geçerli bir başlangıç turu (burada en yakın komşu); simetrik uzaklıklar; kazanç (çıkan − giren) > $10^{-9}$ ise hamle (kayan nokta eşitliğinde döngüye girmemek için EPS) |
| **Zaman (en iyi · ortalama · en kötü)** | — · Yineleme başına $O(n^2)$ · — |
| **Ek bellek** | O(n) |
| **Özellikler** | En kötü NN örneğinde (#24) 4 iyileştirmeyle 374,96 → 252,18 = en iyi. 100 örnekte NN + 2-opt en kötü 1,0488, ortalama 1,0028, 87/100'de en iyi (NN tek başına 16/100). Yerel en iyi ≠ genel en iyi: örnek #9'da 2-opt 329,99'da durur, en iyi 317,36 (oran 1,0398). n = 100'de 991,98 → 841,35 (34 iyileştirme, 93 163 denenen hamle) |
| **Değişmez / doğruluk** | Uygulanan her hamleden sonra tur yine geçerli bir permütasyondur (şehir 0 başta) ve uzunluğu kesin olarak azalmıştır; sonlu sayıda tur olduğundan arama sonlanır |
| **Kütüphane karşılığı** | C: yok (elle) · C++: `std::reverse` parçayı ters çevirebilir (örnekte elle) · Python: dilim ataması `tur[i:j + 1] = tur[i:j + 1][::-1]` (örnekte elle) |
| **Ne zaman kullanılır** | Sezgisel bir turu iyileştirmek için; büyük n'de makul sürede iyi (ama kanıtsız) tur |
| **Ne zaman kullanılmaz** | Kanıtlanmış en iyi gerekiyorsa (yerel en iyide takılabilir); çok büyük n'de tam $O(n^2)$ tarama yerine komşu listeleri ya da Lin–Kernighan türü yöntemler |

C kodunun çekirdeği (`iki_opt`'un tamamı satır 93–121; izleme satırları `ayrinti` ile yazdırılır):

**C** `c/H14_04_iki_opt.c · satır 96–108`

```
 96      while (degisti) {
 97          degisti = 0;
 98          for (int i = 1; i < n - 1 && !degisti; i++) {
 99              for (int j = i + 1; j < n && !degisti; j++) {
100                  int a = tur[i - 1], b = tur[i], c = tur[j];
```

```
                int e = tur[(j + 1) % n];
                double kazanc = (d[a][c] + d[b][e])
                              - (d[a][b] + d[c][e]);
                denenen++;
                if (kazanc < -EPS) {
                    ters_cevir(tur, i, j);
                    iyilestirme++;
                    degisti = 1;
```

**Üç dilde** (kazanç ve hamle; C/C++'ta kazanç satırı iki satıra bölünmüştür):

C++ **H14_04_iki_opt.cpp · satır 100–106**

```
                int a = tur[i - 1], b = tur[i], c = tur[j];
                int e = tur[(j + 1) % n];
                double kazanc = (d[a][c] + d[b][e])
                              - (d[a][b] + d[c][e]);
                denenen++;
                if (kazanc < -EPS) {
                    ters_cevir(tur, i, j);
```

Python **H14_04_iki_opt.py · satır 84–89**

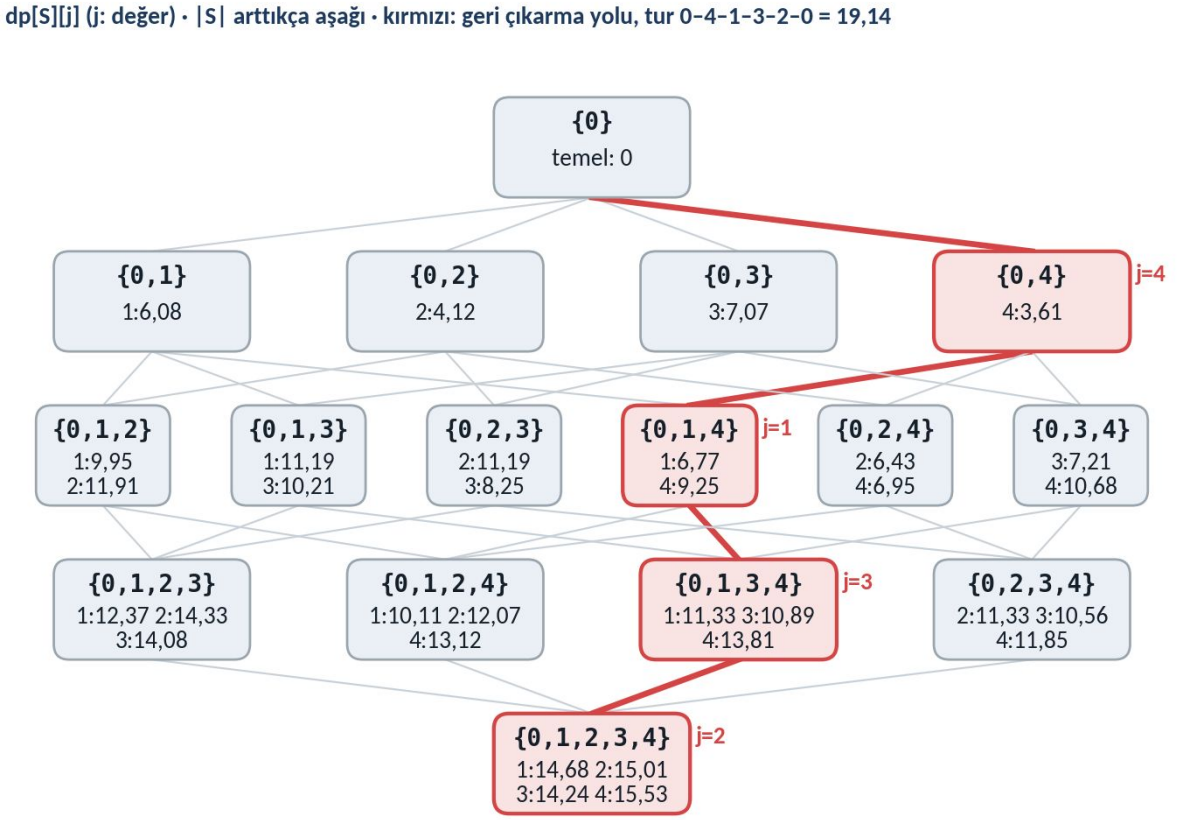
```
                a, b, c = tur[i - 1], tur[i], tur[j]
                e = tur[(j + 1) % n]
                kazanc = (d[a][c] + d[b][e]) - (d[a][b] + d[c][e])
                denenen += 1
                if kazanc < -EPS:
                    ters_cevir(tur, i, j)
```

**Test.** İki kâhin: bitmiş turda bütün (i, j) çiftlerini yeniden tarayan **denetim** (kısaltan hamle kalmamalı; Alıştırma 14.9) ve n ≤ 10'da kaba kuvvet. **Deney:** 100 örnekte NN + 2-opt en kötü 1,0488, ortalama 1,0028; 87/100 örnekte en iyi (ortalama 2,02 hamle). Büyük n'de (tohum 7): n = 25: 540,00 → 516,42; n = 50: 801,33 → 643,35; n = 100: 991,98 → 841,35. **Varyantlar:** en iyi iyileştirme (Alıştırma 14.13), Or-opt, 3-opt, Lin–Kernighan (1973), çok başlangıçlı yerel arama (Atölye 2).

## 4.4 Held–Karp: bit maskesi DP'si (Örnek 5, merak)

**Fikir.** 0-1-2-3-… ile 0-2-1-3-… turlarının ikisinde de {0, 1, 2} gezilmiş ve 3'teyiz: kalan yolun en iyisi buraya **nasıl** geldiğimize bağlı değildir. Held ve Karp (1962) durumu iki bilgiyle tanımladı: **gezilen küme** S (0'ı içerir) ve **son şehir** j. dp[S][j] = 0'dan başlayıp S'nin her şehrinden bir kez geçerek j'de biten en kısa yol. Küme n bitlik bir tamsayıdır (**bit maskesi**): j ∈ S ⇔ S & (1 << j), S \ {j} = S ^ (1 << j). Geçiş: dp[{0}][0] = 0; dp[S][j] = min { dp[S \ {j}][i] + d(i, j) : i ∈ S \ {j} }; yanıt = min_j dp[tümü][j] + d(j, 0).

**Şekil 14.8**
*Held–Karp alt problem uzayı (Örnek 5, beş şehir): düğümler dp[S][j] durumları (|S|'ye göre katmanlar) ve değerleri, oklar geçişler; geri çıkarma yolu kırmızı (yanıt 19,14, tur 0-4-1-3-2-0)*




*durum = 32, geçiş = 52 (Örnek 5, n = 5)*

**Elle izleme.** dp[{0,4}][4] = d(0, 4) = 3,61; dp[{0,1,4}][1] = dp[{0,4}][4] + d(4, 1) = 6,77 (önceki kümede yalnız 4 var). Tablo küme numarası sırasıyla dolar: S \ {j} sayıca S'den küçük olduğu için gereken değerler önceden hesaplanmıştır. Geri çıkarma: son şehir 2, ebeveyn zinciri 2 ← 3 ← 1 ← 4 ← 0; tur 0-4-1-3-2-0 = 19,14 (kaba kuvvetin turunun tersi). Durum 32 = (n − 1)·$2^{n-2}$, geçiş 52.

SÖZDE KOD · Held-Karp (bit maskesi DP)

```
algoritma HELD-KARP(d)
    dp[·][·] ← ∞;  dp[{0}][0] ← 0
    S ← 1'den 2ⁿ−1'e kadar 0'ı içeren her S için (artan sırada)
        S'deki her j ≠ 0 ve S \ {j}'deki her i için
            eğer dp[S\{j}][i] + d[i][j] < dp[S][j] ise
                dp[S][j] ← dp[S\{j}][i] + d[i][j];  ebeveyn[S][j] ← i
    yanıt ← min_j dp[tümü][j] + d[j][0];  turu ebeveyn zinciriyle geri çıkar
```

**Doğruluk.** |S| üzerinde tümevarım: en kısa yolun son kenarı (i, j) ise j'den önceki kısım S \ {j}'yi gezip i'de biten **en kısa** yol olmalıdır, yoksa onu değiştirip daha kısa bir yol elde ederdik (en iyi alt yapı); formül bütün i'leri dener. **Analiz.** (n − 1)·$2^{n-2}$ durum, durum başına O(n) geçiş: Θ($n^2$·$2^n$) zaman; n·$2^n$ hücre (8 bayt dp + 1 bayt ebeveyn). n = 16'da 245 760 durum, 1 720 335 geçiş ($n^2$·$2^n$ = 16 777 216). Sınırı bellek çizer: n = 10: 0,1 MiB; n = 12: 0,4 MiB; n = 14: 2 MiB; n = 16: 9 MiB; n = 18: 40,5 MiB; n = 20: 180 MiB; n = 25: 7 200 MiB; n = 30: 276 480 MiB.

**Tablo 14.7**
*Held–Karp kimlik kartı*

| ALGORİTMA KİMLİK KARTI **Held–Karp: bit maskesi DP ile gezgin satıcı** *(Held–Karp algorithm (bitmask dynamic programming for TSP))* | |
|---|---|
| **Strateji** | Dinamik programlama (H13): dp[S][j] = 0'dan başlayıp S'deki şehirleri birer kez gezerek j'de biten en kısa yol; dp[S][j] = min dp[S \ {j}][k] + d(k, j) (k ∈ S \ {j}); cevap min_j dp[tümü][j] + d(j, 0) |
| **Girdi önkoşulu** | Tam graf, simetrik uzaklıklar; küme bit maskesiyle (1 << i) gösterilir; tablo n·2ⁿ hücre olduğundan n ≤ 16–18 |
| **Zaman (en iyi · ortalama · en kötü)** | Θ(n²·2ⁿ) · Θ(n²·2ⁿ) · Θ(n²·2ⁿ) |
| **Ek bellek** | O(n·2ⁿ) |
| **Özellikler** | 5 şehirli el örneğinde 32 = (n − 1)·2ⁿ/4 durum, 52 geçiş, cevap 19,14 (kaba kuvvetle aynı). n ≤ 10'da LCG örneklerinin hepsinde kaba kuvvetle aynı uzunluk; n = 16'da 245 760 durum, 1 720 335 geçiş, en iyi 340,75 (kaba kuvvet 15!/2 ≈ 6,5 × $10^{11}$ tur isterdi). Tablo n = 16'da 9,0 MiB, n = 20'de 180,0 MiB, n = 30'da 276 480 MiB (≈ 270 GiB) |
| **Değişmez / doğruluk** | Kümeler eleman sayısı artan sırayla işlenirken, daha küçük her T kümesi ve j ∈ T için dp[T][j], 0'dan başlayıp T'yi gezen ve j'de biten en kısa yolun uzunluğudur |
| **Kütüphane karşılığı** | C: yok (elle) · C++: yok (elle) · Python: yok (elle) |
| **Ne zaman kullanılır** | Kesin en iyi tur gerekiyor ve n ≈ 12–18 ise (kaba kuvvetin bittiği, belleğin yettiği aralık); alt küme üzerinde DP kalıbının örneği |
| **Ne zaman kullanılmaz** | n ≳ 20: zaman ve bellek üstel; büyük örneklerde sezgisel + yerel arama ya da dal-sınır / kesme düzlemi tabanlı kesin çözücüler (tanıtım) |

`c/H14_05_held_karp.c · satır 84–107`

```c
    for (int S = 1; S < (1 << n); S += 2) {      /* 0'ı içeren kümeler */
        for (int j = 1; j < n; j++) {
            if (!(S & (1 << j))) {
                continue;
            }
            int onceki = S ^ (1 << j);
            double en = SONSUZ;
            int arg = -1;
            for (int i = 0; i < n; i++) {
                if (!(onceki & (1 << i)) || dp[onceki * n + i] >= SONSUZ) {
                    continue;
                }
                double aday = dp[onceki * n + i] + d[i][j];
                gecis++;
                if (aday < en) {
                    en = aday;
                    arg = i;
                }
            }
            dp[S * n + j] = en;
            ebeveyn[S * n + j] = (signed char)arg;
            durum++;
        }
    }
```

Kesit `held_karp`'ın DP döngüsüdür; tablo `malloc` ile ayrılır (sahiplik modülde, `hk_birak` bırakır), yanıt seçimi ve geri çıkarma (`S ^= 1 << son`) satır 108–127'dedir. **Üç dilde** (geçiş: i döngüsünün gövdesi):

C++ **H14_05_held_karp.cpp · satır 82–86**

```
82                  double aday = dp[onceki * n + i] + d[i][j];
83                  gecis++;
84                  if (aday < en) {
85                      en = aday;
86                      arg = i;
```

Python **H14_05_held_karp.py · satır 71–75**

```
71                  aday = dp[onceki * n + i] + d[i][j]
72                  gecis += 1
73                  if aday < en:
74                      en = aday
75                      arg = i
```

**Test.** n ≤ 10'da Örnek 2'yle aynı uzunluk (n = 4: 174,56, n = 5: 179,28, n = 6: 279,09, n = 7: 296,81, n = 8: 309,93, n = 9: 318,91, n = 10: 324,21). **Deney:** geçiş / $n^2 \cdot 2^n$ oranı n = 16'da 0,103 (sabit çarpan; büyüme $2^n$). **Varyantlar:** yalnız $|S| = k - 1$ ve k katmanlarını tutmak belleği azaltır ama geri çıkarmayı zorlaştırır (H13'ün iki satırlık LCS'si gibi); aynı kalıp "küme + son durum" biçimli birçok sıralama problemine uygulanır.

### 4.5 Dört yöntemin karşılaştırması

**Tablo 14.8**
*Gezgin satıcı için dört yöntem (oranlar Örnek 3–4'ün çıktısından)*

| Yöntem | Strateji | En iyiyi garanti? | Zaman | Uygulanabilir n | Ölçülen (n = 8, 100 örnek) |
|---|---|---|---|---|---|
| Kaba kuvvet | kaba kuvvet / üretme | evet | $\Theta(n!)$ | ≈ 10–11 | kâhin (oran 1) |
| Held–Karp | DP (bit maskesi) | evet | $\Theta(n^2 \cdot 2^n)$ | ≈ 16–18 (bellek) | kaba kuvvetle aynı (n ≤ 10) |
| En yakın komşu | açgözlü sezgisel | hayır | $\Theta(n^2)$ | çok büyük | en kötü 1,4869, ort. 1,0964 |
| NN + 2-opt | yerel arama | hayır (yerel en iyi) | tarama başına $\Theta(n^2)$ | binlerce | en kötü 1,0488, ort. 1,0028 |

## 5. Çözümlü Örnekler ve Uygulamalar

Merdivenin kalan altı basamağı stratejinin, karmaşıklık kuramının ve ölçmenin **uygulamalı** yüzüdür. Her örneğin bütün kodu `kodlar/{c,cpp,python}/` altındadır; burada çekirdek fonksiyonlar ve çıktı kesitleri verilir. C++ ve Python karşılıkları Ek A'dadır.

## 5.1 Problem–strateji eşleme: on iskelet (Örnek 1)

Örnek 1 on kısa problemi tek programda çözer; her problem bir fonksiyondur ve ekrana strateji, karmaşıklık, gerekçe ve küçük girdideki sonuç yazılır (Tablo 14.9). P8 aynı problemi iki stratejiyle çözdüğü için öğreticidir: {1, 3, 4} paralarıyla 6 birim DP'de iki, açgözlüde üç paradır.

**Tablo 14.9**
*Örnek 1: on problem, on iskelet (ciktilar/H14_01_strateji_esleme.txt)*

| # | Problem | Strateji · karmaşıklık · hafta | Sonuç (Örnek 1'in çıktısı) |
|---|---|---|---|
| P1 | En yakın nokta çifti (8 nokta) | kaba kuvvet · O(n²) · H2 | 1 (43, 8) ile 3 (53, 22); uzaklık 17,20; 28 çift |
| P2 | 6 x 6 tahtada 6 vezir: kaç çözüm? | geri izleme · O(n!) budamalı · H3 | 4 çözüm; 153 düğüm (budamasız ağaç 6^6 = 46656 yaprak) |
| P3 | Sıralı 15 notta 70 ve üstü kaç not? | ikili arama · O(log n) · H4 | ilk >= 70 konumu 8; 7 not; 4 yoklama |
| P4 | Ters çevrim sayısı (8 sayı) | böl ve fethet · O(n log n) · H6 | [8 4 2 1 7 3 6 5] -> 15 ters çevrim (en çok 8·7/2 = 28) |
| P5 | Numaralarda yinelenen var mı? | karma · ortalama O(n) · H10 | ilk yinelenen 2201017 (konum 6); 9 yoklama |
| P6 | Ağda 0'dan 6'ya en az aktarma | graf dolaşma (BFS) · O(V + E) · H11 | 4 aktarma; yol 0 1 3 5 6 |
| P7 | En çok çakışmayan sınav (8 aralık) | açgözlü · O(n log n) · H12 | 3 sınav: s0[1, 4) s3[5, 7) s7[8, 11) |
| P8 | {1, 3, 4} paralarla 6: en az para | DP · O(n·T) · H13 | DP 2 para (3 + 3); açgözlü 3 para |
| P9 | 8 şehirde kısa tur (gezgin satıcı) | sezgisel (en yakın komşu) · O(n²) · H14 | NN turu 312,40; en iyi (kaba kuvvet, 5040 tur) 309,93; oran 1,0080 |
| P10 | Ağırlıklı ağda 0'dan 5'e en kısa yol | açgözlü (Dijkstra) · O(V²) · H12 | uzaklık 11; yol 0 2 5 (doğrudan 0-5 kenarı 14) |

Her iskelet kısa bir fonksiyondur; örneğin P8'in DP tablosu ve açgözlü karşılaştırması `p8_bozuk_para`'dadır (`c/H14_01_strateji_esleme.c`, satır 262–289; C++ ve Python sürümü Ek A.1'de).

## 5.2 Doğrulama ile bulma (Örnek 6)

Doğrulayıcı sertifikadaki her indisi bir kez inceler: dizi sınırı, tekrar ve toplam. Döngü sertifika boyunca döner (k + 1 adım); `goruldu[]` dizisinin hazırlanmasıyla birlikte iş O(n + k) = O(n)'dir (k ≤ n). Bulma için iki yöntem aynı dosyadadır: H3'ün budamasız geri izlemesi ve H13'ün tablosu (`geri_izle`, `dp_bul`).

**C** `c/H14_06_dogrulama_bulma.c · satır 38–58`

```c
    for (int t = 0; t < k; t++) {
        (*adim)++;
        if (sec[t] < 0 || sec[t] >= n) {
            *neden = "indis dizi dışında";
            return 0;
        }
        if (goruldu[sec[t]]) {
            *neden = "aynı indis iki kez";
            return 0;
        }
        goruldu[sec[t]] = 1;
        toplam += a[sec[t]];
    }
    (*adim)++;
    if (toplam != hedef) {
        *neden = "toplam hedefe eşit değil";
```

```
        return 0;
    }
    *neden = "toplam = hedef";
    return 1;
}
```

ÇIKTI (BÖLÜM 1)

```
$ ./H14_06_dogrulama_bulma
1) a = {13, 7, 21, 4, 18, 9} (indis 0..5), hedef = 38
   Doğrulama (sertifika = seçilen indisler):
   {0, 1, 4}  -> KABUL (toplam = hedef); adım = 4
   {0, 1, 3}  -> RED   (toplam hedefe eşit değil); adım = 4
   {0, 0, 4}  -> RED   (aynı indis iki kez); adım = 2
   {2, 9}     -> RED   (indis dizi dışında); adım = 2
   Bulma, geri izleme (H3; önce almadan): çözüm {1, 3, 4, 5}; düğüm = 45 (2^(n+1) - 1
= 127)
   Bulma, DP (H13): çözüm {0, 2, 3}; hücre = 273 = (n+1)(T+1)
   DP'nin çözümü doğrulayıcıdan: KABUL
```

## 5.3 İndirgeme örnekleri: sudoku, labirent, ders planı (Örnek 7)

Üç indirgemenin ortak kalıbı: (1) girdiyi dosyadan oku ve **modele** çevir; (2) dönemin bilinen çözücüsünü değiştirmeden çalıştır; (3) sonucu problemin diliyle yaz. Labirentte model, ızgaranın kendisidir: hücre (r, c) köşe numarası r·C + c'dir, dört komşu boş hücre kenardır ve kenar listesi hiç kurulmaz — BFS komşuları dolaşırken hesaplar. Kuyruğun ilk elemanı başlangıç, uzaklık dizisi “ziyaret” işaretidir (H11).

c/H14_07_indirgeme.c · satır 220–240

```
    while (bas_i < son_i) {
        int u = kuyruk[bas_i++];
        (*ziyaret_sayisi)++;
        if (u == hedef) {
            break;
        }
        for (int k = 0; k < 4; k++) {           /* artan köşe numarası */
            int r = u / C + dr[k], c = u % C + dc[k];
            if (r < 0 || r >= R || c < 0 || c >= C || izgara[r][c] == '#') {
                continue;
            }
            int v = r * C + c;
            if (uzaklik[v] == -1) {
                uzaklik[v] = uzaklik[u] + 1;
                ebeveyn[v] = u;
                kuyruk[son_i++] = v;
            }
        }
    }
    return uzaklik[hedef];
}
```

ÇIKTI (BÖLÜM 2)

```
$ ./H14_07_indirgeme ../girdiler/H14_07_sudoku.txt
../girdiler/H14_07_labirent.txt ../girdiler/H14_07_ders_plani.txt
   9 x 15 ızgara: köşe = 56, kenar = 56; S = (1, 1) köşe 16, E = (7, 13) köşe 118
   en kısa yol = 26 adım; kuyruktan çıkan köşe = 54
   ###############
   #S....#.......#
   #*###.#.#####.#
   #*#...#.#***#.#
   #*#.###.#*#*#.#
   #*#.....#*####*#
   #*#######*###*#
   #*********#..E#
```

**Girdi biçimleri.** Ders planı dosyası H11–H12'nin `V E [yonlu|yonsuz]` graf biçimini değil, alana özgü bir biçimi kullanır ve bunu başlık yorumunda belgeler: her satır bir `ÖNKOŞUL DERS` çiftidir (ÖNKOŞUL, DERS'ten önce alınır); `#` ile başlayan satırlar yorumdur. Program ders kodlarını alfabetik sırayla köşe numarasına çevirir; Kahn algoritması giriş derecesi 0 olan en küçük numaralı dersi seçer. Sudoku dosyası 9 satır × 9 karakterdir (`.` boş kare); labirent dosyası ilk satırda satır ve sütun sayısını, ardından ızgarayı verir. 12 dersin dönemlere yerleşimi: 1. dönem AP1, BM, MAT1; 2. dönem AP2, AYR, MAT2; 3. dönem NYP, VY; 4. dönem ALG, IS, VTY; 5. dönem YZ (kurgusal katalog). Hatalı girdi dosyalarında üç dil aynı iletiyi ve çıkış kodunu (1) verir.

## 5.4 Profil çıkarma: darboğazı bulmak (Örnek 8)

Örnek 8 küçük bir kayıt işleme programıdır ve **kasıtlı** bir darboğaz içerir: yinelenen numaraları bütün çiftleri karşılaştırarak bulan $\Theta(n^2)$ fonksiyon. Programın stdout'u belirleyicidir; adım süreleri stderr'e yazılır. Profil iki araçla çıkarılır; C'nin komutları kodun altındaki kutudadır (oradaki `-fno-*` seçenekleri GCC'nin fonksiyonları birleştirip gprof'u yanıltmasını önler):

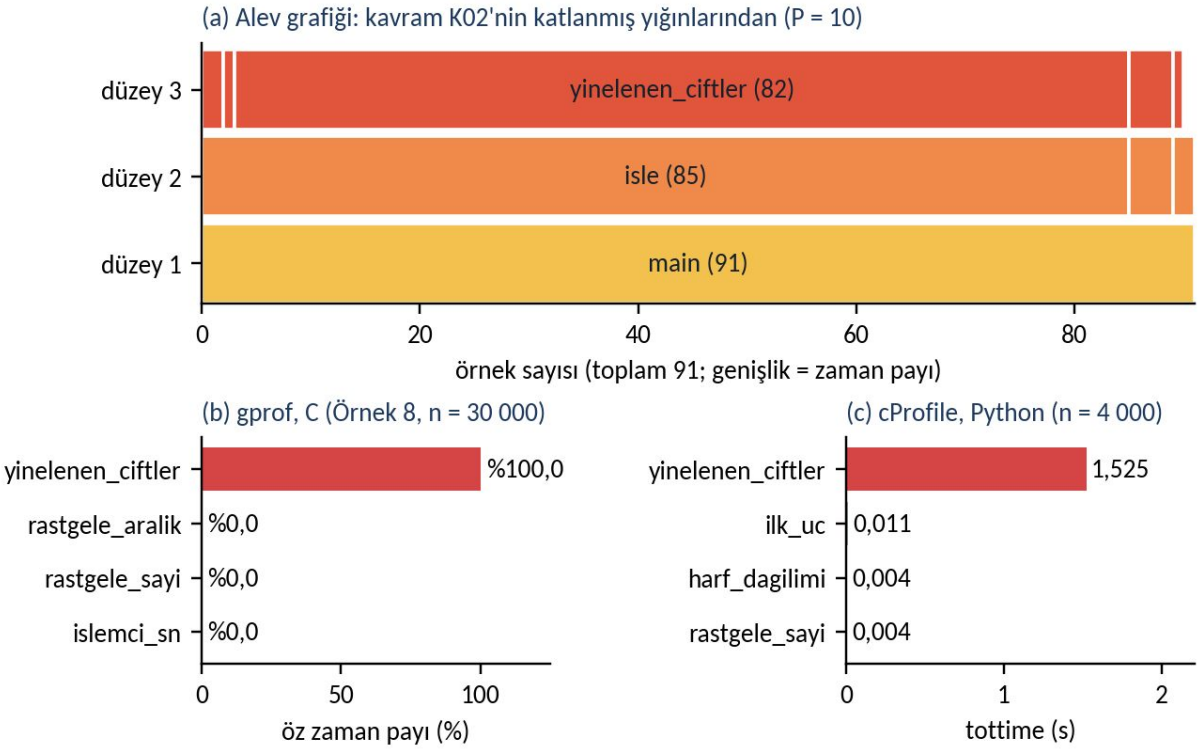
C `c/H14_08_profil_cikarma.c · satır 83–93`

```c
static long long yinelenen_ciftler(long long *kars)
{
    long long cift = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            (*kars)++;
            cift += numara[i] == numara[j];
        }
    }
    return cift;
}
```

PROFİL KOMUTLARI (C, GPROF)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -O2 -g -pg \
      -fno-inline -fno-ipa-cp -fno-ipa-sra \
      H14_08_profil_cikarma.c ortak_rastgele.c zamanlayici.c \
      -o H14_08_profil_cikarma
$ ./H14_08_profil_cikarma 30000 --surum yavas
$ gprof -b ./H14_08_profil_cikarma gmon.out
```

**Şekil 14.9**
*Alev grafiği (K02'nin katlanmış yığınlarından, P = 10) ve Örnek 8'in gerçek profil çıktıları: gprof düz profili (C, n = 30 000) ve cProfile tablosu (Python, n = 4000), bu çalıştırmada; üçünde de darboğaz yinelenen_ciftler*




İki araç da aynı yeri gösterir: C'de `yinelenen_ciftler` zamanın %100,0'ını (öz süre 0,29 s, 1 çağrı), Python'da 1,525 s'nin neredeyse tamamını alır (-pg'siz ikilinin adım süreleriyle %99,8). Darboğaz **algoritma değiştirilerek** giderilir: `yinelenen_siralama` numaraları kopyalayıp sıralar (`qsort`, H6) ve yalnız komşuları karşılaştırır, Θ(n log n). Programın stdout'u iki sürümün aynı sonucu verdiğini yazar:

ÇIKTI (KESİT)

```
$ ./H14_08_profil_cikarma   (n = 4000, bölüm 3)
  3) yinelenen   : 27 eşit çift (tüm çiftler, 7998000 karşılaştırma)
  3) yinelenen   : 27 eşit çift (sırala + komşular, 42836 karşılaştırma)
     iki sürüm aynı mı: evet
```

Karşılaştırma sayısı 7 998 000'den 42 836'ya iner (186,7 kat); n büyüdükçe fark n/log n gibi büyür. Aynı yöntem Mini-OBS 2 v2.1'in `denetle` aracında da darboğazı bulur: 40 000 kayıtta `yinelenen_bul` zamanın %93,33'ünü aldı (kavram çıktısı K05; bu çalıştırmada).

## 5.5 Dönem kıyaslama programı (Örnek 9)

Örnek 9 dönemin sekiz algoritmik çekirdeğini sabit boyutlarda çalıştırır: ikili arama (H4), birleştirmeli ve öbek sıralaması (H6–H7), karma tablosu (H10), BFS (H11), Dijkstra (H12), LCS ve 0/1 sırt çantası (H13). stdout'a **yalnız denetim toplamları** yazılır (üç dilde aynı); süreler stderr'e ya da `--csv` ile CSV'ye gider ve Haftanın Deneyi'nin (b) bölümünü besler. Üç dilin aynı denetim toplamını üretmesi için `karma` fonksiyonu 32 bitlik işaretsiz aritmetik kullanır (H10'un Fibonacci karması; Python'da `& 0xFFFFFFFF`).

ÇIKTI

```
$ ./H14_09_donem_kiyas
Dönem kıyaslaması (ölçek 1): denetim toplamları
```

```
çekirdek        hafta         n              kontrol  boyut
ikili_arama     H4        20000           66071326  n sorgu, sıralı n dizide
birlestirmeli   H6        20000      5109755549047  n sayı
obek_siralama   H7        20000      5109755549047  n sayı
karma_tablosu   H10       20000              59778  n ekleme + n arama
bfs             H11       20000             145232  V = n, E = 5n
dijkstra        H12       20000            4104860  V = n, E = 5n, w 1..100
lcs             H13         600                391  n × n tablo
sirt_cantasi    H13         100               4280  n nesne, W = 1000
```

## 5.6 Final projesi entegrasyonu: Mini-OBS çekirdeği (Örnek 10)

Örnek 10 final projesinin mimarisini **küçük bir modelle** gösterir; tam sistem Mini-OBS 2 v3.0'dır (Bölüm 10) ve bu örnekle karıştırılmamalıdır. Program komutları standart girdiden okur ve her komutu dönemin bir modülüne yönlendirir: depo (dinamik dizi, ×2); dizin (karma, doğrusal yoklama, α ≤ 1/2); sırala (birleştirmeli, kararlı); yinelenen (karma kümesi); önkoşul planı (Kahn); öneri (Levenshtein, iki satır). Komut çözümleyici (`komut_isle`) yalnız yönlendirir; her modül kendi verisini tutar. Bu ayrım v3.0'ın "komut katmanı ↔ algoritma modülleri" sınırının küçük hâlidir.

**c/H14_10_miniobs_cekirdek.c · satır 314–322**

```c
    } else if (t == 2 && strcmp(k[0], "ara") == 0 &&
               sayi_coz(k[1], 0, 999999999, &no)) {
        int i = dizin_bul(no, &y);
        if (i >= 0) {
            printf("  bulundu: %d %s %d (%d yoklama)\n", kayit[i].no,
                   kayit[i].ad, kayit[i].notu, y);
        } else {
            printf("  bulunamadı (%d yoklama)\n", y);
        }
```

**ÇIKTI (KESİT)**

```
$ ./H14_10_miniobs_cekirdek < ../girdiler/H14_10_miniobs_cekirdek.txt
Mini-OBS çekirdeği (H14 Örnek 10; tam sürüm: Mini-OBS 2 v3.0)
> ekle 220101001 AYSE 78
  kayıt 1 eklendi (depo kapasitesi 2, dizin 1/8)
> ekle 220101002 MEHMET 91
  kayıt 2 eklendi (depo kapasitesi 2, dizin 2/8)
…
> onkosul MAT1 MAT2
  MAT1 -> MAT2 (ders 2)
> onkosul AP1 AP2
  AP1 -> AP2 (ders 4)
…
Özet: 28 komut, 2 hata, 10 kayıt, 7 ders
```

Özet satırı (28 komut, 2 hata, 10 kayıt, 7 ders) komut dosyasının hatalı satırlarını da sayar: tanınmayan bir komut ya da geçersiz bir argüman oturumu bitirmez, iletisi yazılır ve sayılır. v3.0'da aynı ilke "tek tip hata bildirimi" adıyla bütün modüllere yayılır.

# 6. Haftanın Deneyi: Dönem Kıyaslama Raporu

## 6.1 Soru ve ölçüm düzeni

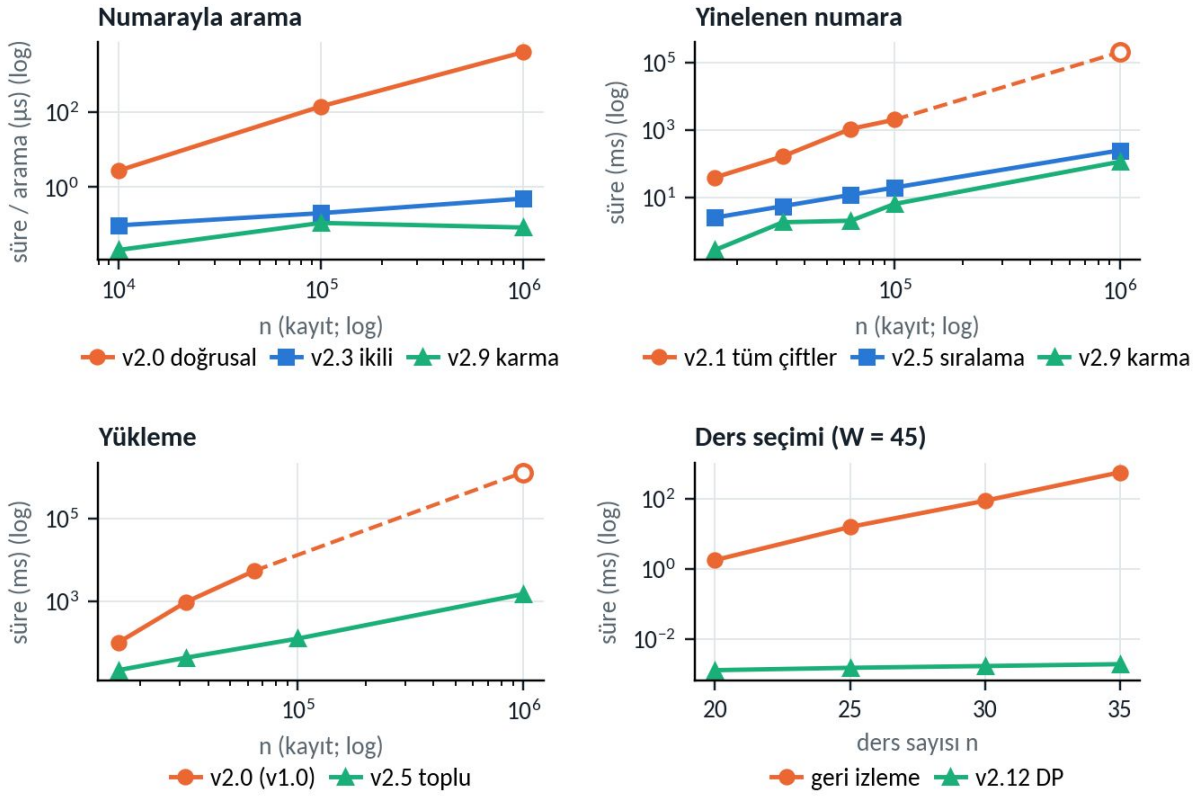
**Soru.** Dönem boyunca Mini-OBS 2'nin işlemlerini birer algoritmayla hızlandırdık. Bu kazanç, derleyiciyi açmanın (-O0 → -O2) ya da dili değiştirmenin (Python → C) kazancıyla nasıl karşılaştırılır? **Yöntem** (Sedgewick ve Wayne, 2011): (a) Mini-OBS 2'nin **gerçek** sürümleri (v2.0, v2.1, v2.3, v2.5, v2.9, v2.12) kendi Makefile'larıyla derlenip aynı ölçücüyle ölçüldü; algoritmik sürümler $10^6$ kayıtta, karesel ilk sürümler $10^4$–$10^5$'te ölçülüp $10^6$ için **kestirildi**; (b) dönemin 16 çekirdeği (Örnek 9 ve dizi modelli yinelenen/arama çekirdekleri) C -O0, C -O2, C++ -O2 ve Python'da ölçüldü; (c) her haftanın deneyinden bir bulgu derlendi.

**Ortam** (`kodlar/deney/OLCUM_ORTAMI.txt`): Intel(R) Xeon(R) Processor @ 2.80GHz, 2 (sanal makine); önbellek L1d 64 KiB (2 instances), L2 2 MiB (2 instances), L3 33 MiB (1 instance); gcc 13.3.0, Python 3.11.15. Düzen: 1 ısınma + 5 tekrar, ortanca; işlemci zamanı esas (1 ms altında duvar saati); yerel log–log eğimi serinin ortancasından 0,4'ten çok sapan aralığın uçları 2 kez daha ölçüldü. Toplam süre 257,4 s (sürücünün duvar saati); makinede başka süreçler de çalışıyordu: süreler bu makineye ve bu ana özgüdür, sayaçlar ve denetim toplamları değildir.

## 6.2 (a) İlk ↔ son sürüm: Mini-OBS 2'nin dört işlemi

Şekil 14.10 dört işlemin 10 sürümünü verir (`h14_ozet_surumler.csv`, C -O2; arama sorgu başına); bütün katlar, ölçümler ve kestirimler Ek B'dedir (Tablo 14.20). Kestirimler işaretlidir: kaba = $T(n)\cdot(10^6/n)^2$, eğimli = ölçülen log–log eğimiyle.

**Şekil 14.10**
*Dönem kıyaslaması (a): dört işlemin ilk, ara ve son sürümleri (log–log); boş işaret ve kesikli çizgi: $10^6$ kestirimi*



Ölçülen en büyük kazançlar: arama $10^6$'da 49 042 kat (doğrusal → karma), ders seçimi n = 35'te 280 994 kat (geri izleme → DP). Yinelenen tespitinde tüm çiftler → karma kümesi $10^5$'te ölçülen 311 kat, $10^6$'da kaba kestirimle 1 744, eğimli kestirimle 2 922 kattır. Karesel ilk sürümlerin $10^6$ değerleri **kestirimdir**: v2.1'in tüm çiftleri n = 64 000'de önbellek sınırını (L2 → L3) aştığı için ölçülen eğim 2'nin üstündedir; gerçek değer kaba ve eğimli kestirim arasında beklenir. Proje belgelerindeki (v2.x) aynı n'deki ölçümlerle oran (bu ölçüm / proje) 0,45–1,43, ortanca 0,84'tür (32 nokta): proje duvar saatiyle, bu deney işlemci zamanıyla ölçtü; büyüme biçimleri ve katların büyüklük sırası aynıdır.

## 6.3 (b) Algoritmik ↔ derleyici ↔ dil kazancı

**Tablo 14.10**
*Derleyici ve dil kazançları (h14_ozet.json → kazanc_ozet; kat)*

| Kazanç türü | Ölçüm | En az | Ortanca | En çok |
|---|---|---|---|---|
| Derleyici: C -O0 / C -O2 (16 çekirdek) | 16 | 1,54 | 2,73 | 6,71 |
| Derleyici: -O0 / -O2 (Mini-OBS ölçücüleri) | 11 | 1,62 | 2,16 | 7,06 |
| Dil: Python / C -O2 (16 çekirdek) | 16 | 18,73 | 51,72 | 207,11 |
| Dil: C++ -O2 / C -O2 (16 çekirdek) | 16 | 0,79 | 1,02 | 1,95 |

Aynı bilgisayarda aynı algoritma: derleyici ortanca 2,73 kat, Python/C ortanca 51,7 kat kazandırır ve bu çarpanlar n ile değişmez. Algoritma değişikliği ise dizi modelli çekirdeklerde bile yinelenen yin_ciftler → yin_siralama 14,1 kat (n = 4 000); yinelenen yin_ciftler → yin_karma 265 kat (n = 4 000); yinelenen yin_siralama → yin_karma 19,9 kat (n = 100 000) kazandırır ve n ile büyür (Tablo 14.20). C++ ile C arasındaki fark çoğunlukla gürültü düzeyindedir (ortanca 1,02); tek aykırı nokta (yin_karma, n = 100 000: 1,95) 1 ms altındaki ortancaların duvar saatiyle ölçülmesinden kaynaklanır, dil farkı olarak yorumlanmamalı.

## 6.4 (c) Dönemin sentezi: her haftadan bir bulgu

Dönemin on üç deneyinden birer bulgu, o haftanın CSV'sinden programla alınarak Ek B'de toplanmıştır (Tablo 14.21). Ortak ders üç cümledir. **Büyüme sınıfı değişince fark n ile büyür:** H3'te budama 8-vezir ağacını 9 321 kat küçülttü, H11'de seyrek grafta komşuluk listesi matrise göre 605 kat hızlıydı, H13'te memo Fibonacci(35)'in çağrı sayısını 29 860 703'ten 69'a indirdi. **Küçük n'de sabitler belirleyicidir:** H4'te doğrusal arama n ≈ 43'e dek ikili aramadan hızlıydı. **Kuram ve donanım birlikte okunur:** H10'da doğrusal yoklama Knuth'un formülünden en çok %4,10 saptı; H8'de aynı Θ(n) dolaşmada dağınık liste bitişik listeden 29,6 kat yavaştı (önbellek).

## 6.5 Yorum ve sınırlılıklar

- Algoritmik iyileştirmenin kazancı **n ile büyür**; derleyici ve dil kazancı **sabit çarpandır**. İkisi aynı tabloda karşılaştırılırken n mutlaka yazılmalıdır: $10^3$ kayıtta "10 kat" olan bir algoritma kazancı $10^6$'da binlerce kattır.
- Karesel sürümler $10^6$'da ölçülemez; kestirim yöntemi (kaba $n^2$ ↔ ölçülen eğim) sonucu birkaç kat değiştirebilir, bu yüzden ikisi birlikte verilir ve tabloda **kestirim** diye işaretlenir.
- Python/C oranı aynı algoritmada bile çekirdeğe göre değişir: sıkı döngülerde yorumlayıcı bedeli en büyüktür, işin C'de yazılmış bir araca (heapq, sorted) gittiği yerde fark küçülür (Kavram K03; Bölüm 8).

- Sınırlılıklar: 2 çekirdekli sanal makine, ölçüm sırasında başka süreçler; clock() 1 µs adımlıdır ve µs altı işlemler duvar saatiyle ölçüldü; ders seçiminde budama etkisi n ile değiştiği için süre düzgün üstel değildir.

# 7. Hata Avcısı

Beş hata bu haftanın beş yanılgısıdır. Her birinin kasıtlı hatalı dosyası (başlığında "BU DOSYA KASITLI HATA İÇERİR"), düzeltilmiş eşi ve gerçek çıktıları `kodlar/hata_avcisi/` altındadır; aşağıdaki çıktılar ortam ve derleme satırları atılarak sadeleştirilmiştir (süreler bu makinede, bu çalıştırmada).

**Tablo 14.11**
*Hata Avcısı: hata | neden | çözüm (belirtiler aşağıda, gerçek çıktılarla)*

| # | Hata | Neden | Çözüm |
|---|---|---|---|
| 1 | "NP = polinom olmayan" yanılgısı | NP, "evet" sertifikası polinom zamanda doğrulanabilen problemlerdir; P ⊆ NP | sınıfı doğrulayıcıyla belirle; NP-tamda P sütununa "?" |
| 2 | Sözde polinom DP'yi polinom sanmak | tablo n·(W + 1) hücre; W = $2^b$ − 1: bit sayısına göre üstel | maliyeti n·(W + 1) ile hesapla; bütçeyi aşınca başka yöntem |
| 3 | Ölçmeden optimizasyon | darboğaz başka yerde (Θ($n^2$) yinelenen); açılan döngü önemsiz | önce profil; en sıcak fonksiyonun algoritmasını değiştir |
| 4 | Sezgiseli "en iyi" diye raporlamak | NN garantisizdir; bu örnekte en iyi tur çok daha kısa | "sezgisel" etiketi; 2-opt; küçük n'de kâhinle oran |
| 5 | Kâhinsiz "çalışıyor" | elle testler hatalı sınırı (son çift) tetiklemiyor | sınır durumları + tohumlu kâhinli rastgele test + küçültme |

### Hata 1 · "NP, polinom OLMAYAN demektir".

C `hata_avcisi/H14_HA1_np_yanilgisi.c · satır 37–40`

```c
        int p_mi = p->polinom_cozum_biliniyor;
        int np_mi = !p->polinom_cozum_biliniyor;     /* HATA: yanılgı */
        printf("  P: %s  NP: %s  %s\n", p_mi ? "evet " : "hayır",
               np_mi ? "evet " : "hayır", p->ad);
```

BELİRTİ

```
$ ./H14_HA1_np_yanilgisi
Karmaşıklık sınıfı raporu
  P: evet   NP: hayır  Dizi sıralı mı?
  P: evet   NP: hayır  s'den t'ye en çok k kenarlı yol var mı? (BFS)
  P: hayır  NP: evet   Toplamı T olan alt küme var mı?
  P: hayır  NP: evet   Uzunluğu en çok L olan tur var mı? (gezgin satıcı)
  P: hayır  NP: evet   Graf 3 renkle boyanabilir mi?
  P: hayır  NP: evet   Program bu girdide durur mu? (durma problemi)
P'de 2, NP'de 4 problem; ikisinde birden 0 problem
```

Rapor P'deki iki problemi NP'nin dışına, karar verilemeyen durma problemini NP'nin içine koyar. Düzeltilmiş sürüm sınıfı **doğrulayıcıyla** tanımlar (`np_mi = p->polinom_dogrulayici_var`) ve NP-tamda P sütununa "?" yazar: "P'de 2, NP'de 5 problem; ikisinde birden 2; P'de olup NP'de olmayan 0 (P ⊆ NP)".

### Hata 2 · Sözde polinom DP'yi polinom sanmak.

C hata_avcisi/H14_HA2_sozde_polinom.c · satır 38–42

```c
        long long tahmin = (long long)N * b;     /* HATA: girdi uzunluğu */
        if (tahmin > BUTCE) {
            printf("  %2d  %10lld   bütçe aşıldı\n", b, tahmin);
            continue;
        }
```

BELİRTİ

```
$ ./H14_HA2_sozde_polinom
0/1 sırt çantası, n = 20 nesne; kapasite W = 2^b - 1
   b  tahmin n·b   karar          gerçek hücre       değer
   8         160   DP uygun               4372         731
  12         240   DP uygun              71628         881
  16         320   DP uygun            1140618         629
  20         400   DP uygun           17639419         562
  24         480   DP uygun          306017383         966
  40         800   DP uygun       HATA: 1099511627776 hücrelik tablo ayrılamadı
```

Tahmin girdi uzunluğuyla (n·b) yapıldığı için her b'de “DP uygun” görünür. Düzeltilmiş sürüm maliyeti n·(W + 1) hücre olarak hesaplar ve bütçe ($5 \cdot 10^7$ hücre) aşılınca geri izlemeye geçer:

DÜZELTİLMİŞ (KESİT)

```
$ ./H14_HA2_sozde_polinom_duzeltilmis
   b         n·(W+1)  2^(n+1)-1  seçim         iş (hücre/düğüm)   değer  denetim
  20        20971520    2097151  DP                    17639419     562  geri izleme
aynı
  24       335544320    2097151  geri izleme            1822855     966  tablo bütçeyi
aşıyor
  40  21990232555520    2097151  geri izleme             565557     562  tablo bütçeyi
aşıyor
```

### Hata 3 · Ölçmeden optimizasyon.

Python hata_avcisi/H14_HA3_olcmeden_optimizasyon.py · satır 69–71

```python
    ort1, cift1 = isle(ortalama)
    t1 = islemci_sn()
    ort2, cift2 = isle(ortalama_hizli)              # HATA: ölçmeden
```

BELİRTİ

```
$ python3 H14_HA3_olcmeden_optimizasyon.py
n = 3000: ortalama 49.40, yinelenen çift 483
'hızlı' sürüm: ortalama 49.40, yinelenen çift 483
[stderr, bu çalıştırmada] önce 121.4 ms, sonra 123.4 ms; hızlanma 0.98 kat (işlemci,
bu çalıştırmada)
```

“Ortalama yavaştır” tahminiyle döngü dörder açılmış; fark gürültü düzeyindedir. Düzeltilmiş sürüm önce profil çıkarır (`cProfile` + `pstats`), en sıcak fonksiyonun (`yinelenen_ciftler`) algoritmasını değiştirir (sırala + komşular) ve sonucun aynı olduğunu yazar.

Düzeltilmiş çıktı: “sıralamalı sürüm: ortalama 49.40, yinelenen çift 483; aynı”; “en çok zaman (tottime): yinelenen_ciftler (bu çalıştırmada)”; “önce 472.8 ms (profil açık), sonra 4.0 ms; hızlanma 117.6 kat (işlemci, bu çalıştırmada)”.

**Hata 4 · Sezgisel çözümü en iyi diye raporlamak.**

C++ `hata_avcisi/H14_HA4_sezgisel_en_iyi.cpp · satır 62–66`

```cpp
    std::printf("ROTA RAPORU (%d şehir)\nEN KISA TUR:", n);   // HATA
    for (int s : tur) {
        std::printf(" %d", s);
    }
    std::printf(" %d\nUzunluk: %.2f (en iyi çözüm)\n", tur[0], u);
```

BELİRTİ

```
$ ./H14_HA4_sezgisel_en_iyi
ROTA RAPORU (8 şehir)
EN KISA TUR: 0 2 3 7 5 6 1 4 0
Uzunluk: 374.96 (en iyi çözüm)
```

Düzeltilmiş sürüm sonucu “sezgisel” diye etiketler, 2-opt uygular ve küçük n'de kâhinle oranı yazar: “2-opt ile iyileştirilmiş tur : 252.18 — yerel en iyi”; “Oranlar: sezgisel/en iyi = 1.4869, 2-opt/en iyi = 1.0000”.

**Hata 5 · Kâhin testi olmadan “çalışıyor” demek.**

C `hata_avcisi/H14_HA5_kahinsiz_test.c · satır 34–39`

```c
    qsort(b, (size_t)n, sizeof *b, kucuk_once);
    for (int i = 1; i < n - 1; i++) {            /* HATA: i < n olmalı */
        if (b[i - 1] == b[i]) {
            return 1;
        }
    }
```

BELİRTİ: ELLE TESTLER GEÇER, KÂHİN İLK DENEMEDE KARŞI ÖRNEK BULUR

```
$ ./H14_HA5_kahinsiz_test --kahin
3/3 elle test geçti: "çalışıyor"
  deneme 1 (tohum 1, n = 3): [15 15 9] -> hızlı 0, kâhin 1
  küçültülmüş karşı örnek: [15 15] -> hızlı 0, kâhin 1
```

Elle yazılmış testlerde yinelenen çift hiç dizinin sonunda değildir. Tohumlu rastgele testler kâhine (v2.1'in tüm çiftleri) karşı ilk denemede karşı örnek bulur; **küçültme** onu [x x]'e indirir. Final projesinin test paketi bu yüzden birim + kâhin + stres testlerinden oluşur (yönerge § 4.5).

Düzeltilmiş sürümde (`i < n`) aynı paket geçer: “sınır durumları (n = 0, 1, 2; üçlü; en büyükte yinelenen): 5/5 geçti”; “kâhinli rastgele test: 1000/1000 uyuştu (611 denemede yinelenen vardı)”.

# 8. Dil Penceresi: C · C++ · Python

**Dil seçimi bir mühendislik kararıdır.** Dönemin ölçümleri üç dilin yerini sayılarla gösterir: aynı algoritmada C++ C ile aynı hızdadır (ortanca 1,02), Python 19–207 kat yavaştır (ortanca 51,7); buna karşılık Python kodu daha kısadır (dönemin 184 örneğinde satır oranı C = 1 iken C++ 0,93, Python 0,72;

K04) ve geliştirme hızlıdır. C denetim ve öngörülebilir başarım, C++ soyutlama ile başarımı birlikte verir. Doğru soru "hangi dil daha hızlı?" değil, "bu iş için süre bütçesi, bakım ve güvenlik ödünleşimi nedir?" sorusudur.

**Tablo 14.12**
*Dil karşılıkları: haftanın yapıları, araçları ve ölçülen çarpanlar (bu makinede)*

| Öğe | C | C++ | Python |
|---|---|---|---|
| Ölçülen süre (aynı algoritma) | `1 (taban)` | `≈ 1,02 × C` | `≈ 52 × C (ortanca)` |
| Permütasyon üretme | `elle (H3, kullanildi[])` | `std::next_permutation` | `itertools.permutations` |
| Öncelik kuyruğu | `elle ikili öbek (H7)` | `std::priority_queue` | `heapq` |
| Bit maskesi (Held–Karp) | `S & (1 << j); int'te 1 << 31 tanımsız: n ≤ 30` | `aynı; std::bitset<N>` | `aynı; tamsayı sınırsız` |
| (n − 1)! taşması | `uint64_t: 20! sığar` | `aynı` | `taşmaz (büyük tamsayı)` |
| Eşitlikte seçim | `döngüde < ya da <=` | `aynı; std::min_element ilkini` | `min/max ilk en küçüğü` |
| Profil aracı | `gcc -pg + gprof; perf` | `aynı (g++ -pg); perf` | `cProfile + pstats; timeit` |
| Hızlandırma yolu | `algoritma; -O2` | `algoritma; -O2; STL` | `yerleşikler (sorted 12,2×), C uzantıları, NumPy, PyPy` |

> **DİL PENCERESİ · Python'da başarıma kaçış yolları (yalnız tanıtım)**
>
> Python'da hızlanmanın ilk yolu işi **C'de yazılmış yerleşiklere** devretmektir: K03'te aynı iş el yazımı döngü ↔ yerleşik araçla ölçüldü (bu makinede): toplam 5,2 kat (`sum`), sıralama 12,2 kat (`sorted`), sıklık sayımı 3,9 kat (`dict`); üçünde de sonuç aynı. Sonraki yollar dersin kapsamı dışındadır ve yalnız adlarıyla anılır: C uzantı modülleri (CPython C API), dizi işlemlerini C'de yapan NumPy ve tam zamanında derleyici kullanan PyPy. Hiçbiri algoritma sınıfını değiştirmez: $\Theta(n^2)$ bir döngü yerleşiklerle de $\Theta(n^2)$'dir; n büyüdükçe Python'un $\Theta(n \log n)$ sürümü C'nin $\Theta(n^2)$ sürümünü geçer (Örnek 8'i n = 30 000 ile iki sürümde çalıştırıp adım sürelerini karşılaştırın).

> **DİL PENCERESİ · Taşınabilir ve belirleyici kod: dönemin bilançosu**
>
> Dönemin 184 örneğinin 177'sinde (%96,2) üç dilin çıktısı bayt bayt aynıdır; kalan 7 örnekte (Δ) dil farkının kendisi konudur (tamsayı bölmesi, taşma, özyineleme sınırı …; K04). Bunu sağlayan kurallar dönem boyunca aynıydı: ortak LCG (rand() yok), 32 bit işaretsiz karma, tam sıralama tanımlayan karşılaştırıcılar ve eşitlik kuralları (bu hafta: kare uzaklık ve EPS), stdout'ta adres ve süre yok. Bu kültür final projesinin "çıktı sözleşmesidir" (yönerge § 4.4).

## 9. Mühendis Gözüyle: Önce Doğru, Sonra Ölç

Bir programı hızlandırmanın sırası vardır ve dönemin deneyleri bu sırayı kanıtlar (Örnek 8, Bölüm 6):

1. **Doğru:** önce basit, doğru ve sınanabilir bir sürüm; onun kâhini (kaba kuvvet) ve testleri. Hızlı ama yanlış bir program hiçbir işe yaramaz.
2. **Ölç:** gerçekçi girdiyle, tekrarlı ve ortancayla; ölçüm ortamını yaz.
3. **Darboğazı bul:** profil çıkar (gprof, perf, cProfile); tahmin etme (Hata Avcısı 3).
4. **Algoritmayı değiştir:** en sıcak fonksiyonun büyüme sınıfını düşür ($\Theta(n^2) \to \Theta(n \log n) \to \Theta(n)$); sonucu eskisiyle karşılaştır.
5. **Mikro iyileştirme:** ancak en sonda ve ölçerek: derleyici bayrakları, veri yerleşimi, döngü düzeni.

**Dil seçimi bir mühendislik kararıdır.** Bir dağıtım şirketinin rota servisini düşünün: rota sezgiseli saniyede binlerce kez çağrılıyorsa C ya da C++ (Python ortanca 52 kat yavaştı); rotaları haftada bir planlayan bir analiz betiğiyse Python'un geliştirme hızı ve kütüphaneleri ağır basar. Çoğu gerçek sistem ikisini birleştirir: sıcak çekirdek C/C++'ta, gerisi betik dilinde. Karar ölçümle ve bakım maliyetiyle birlikte verilir; “C her zaman daha iyidir” de, “donanım ucuz, Python yeter” de ölçülmemiş genellemelerdir.

# 10. Final Projesi: Mini-OBS 2 v3.0 ve Seçenek Havuzu

Final projesi dönemin sentezidir: her hafta bir algoritmayla büyüttüğünüz programı (ya da seçenek havuzundan seçtiğiniz bir projeyi) **tek bir uygulamada** bütünleştirir, **otomatik bir test paketiyle** doğrular, ilk ve son gerçekleştirimleri aynı düzenekte **ölçüp** rapor yazar ve H14'te **canlı demoyla** sunarsınız. Bu bölüm final projesi yönergesinin, rapor şablonunun ve v3.0 belgesinin (üçü de Mini-OBS 2 proje klasöründe: `FINAL_PROJE_YONERGESI.md`, `FINAL_RAPOR_SABLONU.md`, `v3.0/`) öğrencinin işini yapması için gerekenlerini bir araya getirir; ayrıntı ve güncel duyurular için belgelerin kendisi esastır. Çalışma bireyseldir; kesin tarihler ve teslim ortamı kurumun akademik takvimine göre duyurulur.

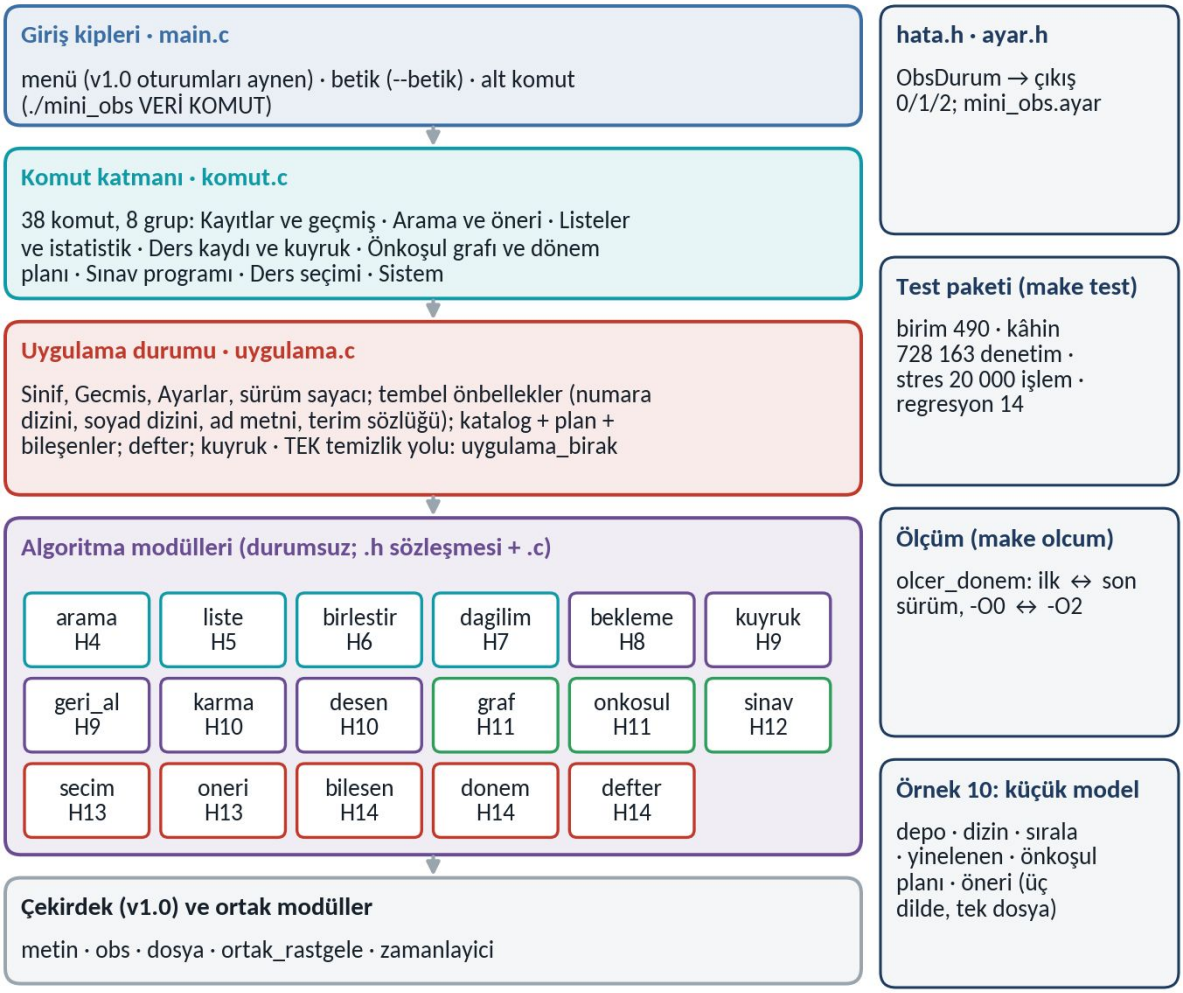
## 10.1 Yönerge özeti ve takvim

**Takvim** (yönerge § 3; zamanlar öneridir): H11: yönerge ve şablon verilir; seçenek havuzu tanıtılır · H11 sonu: konu seçimi: varsayılan proje ya da havuzdan bir seçenek (havuz için bir sayfalık öneri (yönerge § 2.2); öğretim üyesi onayı) · H12 sonu: ara kontrol (atölyede ≈ 10 dakika) (çalışan iskelet: `make`, `make test` (en az iki modül kâhinle), mimari diyagram taslağı, ölçüm planı (hangi çiftler, hangi n)) · H13: bütünleştirme ve stres testi; kod dondurma önerisi: H13 sonu (`make test`, `make asan`, `make valgrind` temiz) · H13 sonu – H14 başı: `make olcum` (tek oturumda), rapor, sunum (CSV, rapor, slaytlar) · H14: teslim (ilk dersin başı) ve sunumlar (H14 atölyeleri; Yol Haritası 5.14) (yönerge § 6'daki dosyalar).

## 10.2 Varsayılan proje: Mini-OBS 2 v3.0

**Asgari kapsam** (yönerge § 2.1; giriş kipleri, bütünleştirme, test paketi ve kalite satırları aşağıdaki gereksinimlerle aynıdır). **Modüller:** arama (ikili, karma; soyad öneki; ad içinde Rabin–Karp) · sıralama (çok ölçütlü, Türkçe, büyük veri) · karma · bekleme listesi ve kayıt kuyruğu · geri al/yinele · önkoşul grafı, ders planı ve **AKTS sınırlı dönem planı** · sınav programı (**gün sınırı, salon kapasitesi, öğrenci başına günde en çok bir sınav**) · ders seçimi optimizasyonu · "Bunu mu demek istediniz?" önerisi · istatistik ve not dağılımı. **Yapı:** modül sınırları (`.h` sözleşmesi + `.c`); tek tip hata bildirimi (dönüş kodları → çıkış kodu 0/1/2); yapılandırma dosyası; bellek sahipliği tablosu **tek bir belgede**. **Ölçüm raporu: dönem kıyaslaması**: her işlemin ilk (v2.0–v2.2 kaba kuvvet ya da işlemin ilk naif sürümü) ve son (algoritmik) gerçekleştirimi; algoritmik sürümler $10^6$ kayıtta, karesel sürümler $10^4$–$10^5$'te ölçülüp $10^6$ için işaretli kestirimle.

**Şekil 14.11**
*Mini-OBS 2 v3.0 mimarisi (v3.0 belgesi § 4): giriş kipleri → komut katmanı → uygulama durumu → algoritma modülleri (dönemin haftalarının renkleriyle) → çekirdek; sağda Örnek 10'un küçük modeli*




v3.0'ın komutları sekiz grupta toplanır (v3.0 belgesi Tablo 7): **Kayıtlar ve geçmiş** (v1.0, v2.8), **Arama ve öneri** (v2.3, v2.9, v2.12), **Listeler ve istatistik** (v2.1, v2.4–v2.6), **Ders kaydı ve kuyruk** (v2.7–v2.8, v3.0), **Önkoşul grafı ve dönem planı** (v2.10, v3.0), **Sınav programı** (v2.11, v3.0), **Ders seçimi** (v2.12), **Sistem** (v3.0). Her işlem bir durum kodu döndürür ve kod tek bir fonksiyonla çıkış koduna (0 başarı, 1 dosya/iş kuralı/yokluk, 2 hatalı kullanım) çevrilir. Ne verilir, ne yazılır (yönerge § 2.1):

**Verilir:** v2.0–v2.12 referans sürümleri ve belgeleri; v3.0'ın yeni başlık dosyalarının sözleşmeleri (v3.0 belgesi § 4–5'te: `hata.h`, `ayar.h`, `bilesen.h`, `donem.h`, `defter.h`, `uygulama.h`, `komut.h`); `mini_obs.ayar`, `ornek_veri/` ve `testler/` girdileri (betikler, iki döngülü katalog, yabancı numaralı kayıt dosyası, v1.0 oturumlarının altın çıktıları); ortak modüller; bu yönerge ve rapor şablonu. **Siz yazarsınız:** v3.0 modüllerinin `.c` dosyaları, `main.c`, komut katmanı, testler (birim, kâhin, stres, regresyon betiği), ölçüm aracı ve `make olcum`, ölçüm raporu ve onu CSV'den üreten betik, README, sunum. **Değiştirilmez:** v1.0 veri modeli, dosya biçimleri ve not kuralları; ortak modüller (`ortak_rastgele`, `zamanlayici`).

**Gereksinimler** (v3.0 belgesi § 11; her birinin ayrıntısı Ek B'de, Tablo 14.22): (1) üç giriş kipi; (2) komutlar; (3) tek tip hata bildirimi; (4) yapılandırma; (5) bütünleştirme kuralları; (6) test paketi (`make test`); (7) kalite; (8) dönem kıyaslaması (`make olcum`, ≤ 6 dakika). Bütünleştirme kuralları modüllerin birbirine dayandığı yerleri tanımlar: ders kayıtları yalnız kayıtlı öğrencilerle yapılır, önkoşul döngülerinin hepsi

bulunur, dönem planı AKTS sınırıyla kurulur, sınav programı gün ve salon sınırlarına uyar, saklanan dizinler kayıt dizisi değişince geçersiz sayılır ve geri al/yinele çok derslidir.

**İpucu** (v3.0 belgesi § 11'deki 9 ipucunun ilki; başlangıç noktası ve öbürleri belgede): Bütünleştirmeye **durum yapısını** tasarlayarak başlayın: hangi yapı kimin, ne zaman geçersiz olur, kim bırakır? v3.0 belgesi Tablo 6'yı kod yazmadan önce doldurun.

## 10.3 Seçenek havuzu ve ortak asgari gereksinimler

Öne çıkmak isteyen öğrenciler, **ortak asgari gereksinimleri** (yönerge § 2.3) karşılamak koşuluyla Mini-OBS 2 yerine aşağıdaki altı projeden birini seçebilir (Yol Haritası 10.3). Havuzdan seçenler konu seçiminde (yönerge § 3) bir sayfalık **proje önerisi** verir: kapsam, veri, algoritmalar, ortak asgari gereksinimlerin karşılığı (yönerge § 2.3 tablosu doldurulmuş olarak) ve dönem kıyaslamasının "ilk ↔ son" çiftleri. Öneri öğretim üyesinin onayıyla kesinleşir. Havuz projelerinde de bütün teknik kurallar (yönerge § 4), teslim edilecekler (yönerge § 6) ve rubrik (yönerge § 8) aynıdır; "dönem kıyaslaması" yerine **taban çizgisi ↔ algoritmik gerçekleştirim** kıyaslaması yapılır.

**Tablo 14.13**
*Seçenek havuzu (yönerge § 2.2; yol haritası 10.3): her projenin asgari gereksinimleri, kabul ölçütleri, önerilen verisi ve zorluk notu yönergededir*

| Proje | Kapsam |
|---|---|
| Kampüs navigasyonu | Bina–yol grafında (yönsüz, ağırlıklı) en az durakla yol (BFS), en kısa yol (Dijkstra) ve bütün binaları en düşük toplam uzunlukla bağlayan ağ (en küçük kapsayan ağaç: Kruskal + birleşim–bul ya da Prim); harita dosyadan okunur; bina adıyla arama |
| Mini arama motoru | Metin derleminde (satır ya da paragraf = belge) ters dizin (terim → belge listesi; karma tablosu), VE/VEYA sorguları, sıralı sonuçlar (terim sıklığına dayalı puan, ilk-k öbekle) ve düzenleme uzaklığıyla öneri ("Bunu mu demek istediniz?") |
| Sıralama laboratuvarı | Algoritmaları aynı düzenekte karşılaştıran terminal aracı: girdi türleri, sayaçlar, kararlılık denetimi, CSV çıktısı ve rapor; ara projenin genişletilmiş ve **uyarlanabilir** hâli |
| Bulmaca çözücü | Sudoku, labirent ve kakuro (en az ikisi) için geri izleme, BFS ve budama stratejilerinin karşılaştırması |
| Huffman tabanlı dosya sıkıştırıcı | Bayt sıklıkları, öbekle Huffman ağacı, kanonik kodlar, bit düzeyinde yazıcı/okuyucu ve sürümlü ikili dosya biçimi; sıkıştırma ve açma |
| Ders/sınav çizelgeleyici | Çakışma grafında sınav günlerinin atanması (graf boyama sezgiselleri), salon ataması (aralık bölümleme) ve kısıtlar; sezgisellerin kaba kuvvetle (küçük n) karşılaştırılması |

**Tablo 14.14**
*Ortak asgari gereksinimler (yönerge § 2.3): hangi proje seçilirse seçilsin hepsi bulunur*

| Gereksinim | Varsayılan projedeki karşılığı (örnek) | Kanıt |
|---|---|---|
| en az bir **arama** algoritması | ikili arama, karma tablosu, Rabin–Karp | kimlik kartı + kâhin |
| en az bir **sıralama** algoritması | birleştirmeli, sayma, öbek (ilk-k) | kimlik kartı + kâhin + kararlılık denetimi |
| en az bir **dinamik veri yapısı** | dinamik dizi, bağlı liste (bekleme), dairesel kuyruk, iki yığın (geri al), karma tablosu | bellek sahipliği tablosu + ASan/Valgrind |
| en az bir **graf ya da DP** algoritması | Kahn, güçlü bağlı bileşenler, Welsh–Powell, 0/1 sırt çantası, Levenshtein | kimlik kartı + kâhin |
| **kâhin testleri** | her algoritma sınanan koddan farklı bir yoldan hesaplayan kâhine karşı, tohumlu rastgele girdilerle | `make test` |

| Gereksinim | Varsayılan projedeki karşılığı (örnek) | Kanıt |
|---|---|---|
| **ölçüm raporu** | ilk ↔ son gerçekleştirim, tablo, grafik, kuramla karşılaştırma | rapor şablonu |
| **C ile modüler gerçekleştirim** | C17, `.h` sözleşmesi + `.c`, Makefile | `make temiz && make` |

## 10.4 Teknik kurallar ve teslim edilecekler

**Dil ve derleme** (yönerge § 4.1): Yalnızca **C17** (komut aşağıda); **sıfır uyarı**. `-O0` … `-Os` düzeylerinde de uyarısız derlemeniz önerilir. Yalnızca standart kütüphane. Algoritmanın kendisi kütüphaneye devredilmez: `qsort`/`bsearch` yalnızca kâhin olarak ya da açıkça karşılaştırma için kullanılır. `rand()` kullanılmaz; rastgelelik ortak LCG'den (`ortak_rastgele`) gelir ve tohumla raporlanır.

DERLEME (YÖNERGE § 4.1)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -O2 -g …
```

**Makefile hedefleri** (yönerge § 4.3; ASan ve Valgrind için WSL ya da Linux): `make` — bütün programları derler; `make test` — birim, kâhin, stres ve regresyon testlerini derler ve çalıştırır; biri başarısızsa hata verir (kendi makinenizde ≈ 2 dakikayı aşmamalı); `make olcum` — raporun bütün ölçümlerini yapar: `olcum/*.csv` ve `olcum/OLCUM_ORTAMI.txt` (≈ 6 dakikayı aşmamalı); `make asan` — `-fsanitize=address,undefined -fno-sanitize-recover=all` ile testler ve örnek çalıştırmalar; rapor çıkmamalı; `make valgrind` — sanitizer'sız `-O0 -g` ikililerle testler ve örnek çalıştırmalar Valgrind altında: her çalıştırmada "ERROR SUMMARY: 0 errors" ve "definitely lost: 0 bytes"; `make kapsam` — (önerilir) gcov ile satır kapsamı: modül başına satır, çalışan satır, yüzde; `make temiz` — derleme ürünlerini ve geçici dosyaları siler.

**Çıktı sözleşmesi** (yönerge § 4.4; otomatik değerlendirme buna dayanır): **stdout deterministiktir:** aynı girdi ve argümanlarla bayt bayt aynı. **Süreler yalnızca stderr'e** ya da CSV'ye yazılır; adres (`%p`) yazdırılmaz. **Çıkış kodları:** 0 başarı; 1 dosya/bellek hatası, aranan kaydın yokluğu, iş kuralı reddi (ör. döngülü önkoşul grafı, karşılanamayan gün sınırı), başarısız test; 2 hatalı kullanım (bilinmeyen komut, eksik/geçersiz argüman). **Betik kipinde** her komut `>` ile başlayan bir satırla yankılanır, ardından komutun çıktısı gelir; betik hatalı bir komutta durmaz; çıkış kodu oturumdaki en ağır hatadır (2 > 1 > 0). **Program, komut ve seçenek adları** (varsayılan proje) v3.0 belgesinin kullanım bölümündeki gibidir; havuz projelerinde README'de tanımlanır. **Çıktı biçimi serbesttir;** ancak denetlenen değerler — öğrenci numaraları, ders kodları, sayaçlar, ortanca ve ortalamalar, gün ve salon sayıları — çıktıda **ondalık noktayla ve binlik ayırıcısız** yazılır (ör. `77.5`, `4932`). Rapor ise Türkçe sayı biçimini kullanır (77,5; 4 932).

**Teslim edilecekler** (yönerge § 6) yedi kalemdir: kod, Makefile, test paketi, ölçüm verisi ve raporu, README.md, sunum ve beyan. Her kalemin içeriği (klasörler, dosya adları) Ek B'dedir (Tablo 14.23).

## 10.5 Kabul ölçütleri ve test senaryoları

**Değerlendirmeye alınma koşulları** (yönerge § 7.1; hepsi sağlanmalı): (1) `make temiz && make` derleme hatası vermez (hata değerlendirmeyi durdurur; uyarılar kod kalitesinden puan kaybettirir); (2) `make test` çalışır; v1.0'ın 99 doğrulaması geçer (varsayılan proje); (3) Rapor (PDF), raporun sayılarını üreten CSV'ler ve sunum dosyası teslimdedir; rapordaki sayılar CSV'lerle tutarlıdır; (4) Havuz projelerinde: konu seçiminde onaylanmış bir öneri vardır; (5) Teslimde derleme ürünü ve büyük veri dosyası yoktur.

**Kabul ölçütleri ve test senaryoları** (v3.0 belgesi § 11) Ek B'dedir: 11 denetlenebilir ölçüt (Tablo 14.24; README'nizde ✓/✗ listesi olarak) ve 12 sınır senaryosu (Tablo 14.25). Ölçütler üç kümedir: **kalite** (sıfır

uyarı; `make test`, `make asan` ve `make valgrind` temiz), **işlevsel doğruluk** (v1.0 oturumlarının çıktısı, arama ve öneri, kontenjan ile geri al, saklanan dizinin geçerliliği, önkoşul döngüleri, dönem planı, sınav programı, çıkış kodları, kâhinin kasıtlı hataları) ve **ölçüm** (kat eşikleri). Senaryolar boş veri dosyası, hatalı argüman, sınırı 1 olan geri al ve kuyruk, iki döngülü katalog, yabancı numaralı kayıt dosyası ve $10^6$ kayıt gibi sınır durumlarını sınar.

Değerlendirmede yönerge § 7.2'deki 22 komutluk işlevsel kabul listesi teslim klasörünüzde otomatik çalıştırılır; değerler stdout'ta verilen sırayla görünmeli, çıkış kodları birebir tutmalıdır (liste bu ölçütleri ve senaryoları komut düzeyinde ayrıntılandırır). **Ölçümle ilgili 8 eşik** (yönerge § 7.3; ör. $10^6$ kayıtta yinelenen tespiti: tüm çiftler (kestirim) / karma kümesi ≥ 500 kat, `make olcum` toplam süresi ≤ 6 dakika) referans makinenin değerleriyle birlikte Ek B'dedir (Tablo 14.26). Bir eşik kendi makinenizde sağlanmıyorsa bu otomatik bir kayıp değildir: nedeni raporda ölçümle tartışılır; açıklanmayan sapma puan kaybettirir.

## 10.6 Rubrik (yol haritası Tablo 10.3)

**Tablo 14.15**
*Final projesi rubriği (Yol Haritası Tablo 10.3; yönerge § 8)*

| Ölçüt | Ağırlık | Ne değerlendirilir? (Yol Haritası Tablo 10.3) |
|---|---|---|
| İşlevsel doğruluk | %25 | Tüm menü işlevleri beklenen çıktıyı üretir; sınır durumları doğru işlenir. |
| Algoritma seçimi ve analiz | %25 | Her işlev için seçilen algoritma ve veri yapısı gerekçelendirilir; kimlik kartları ve karmaşıklık analizi doğrudur. |
| Test ve doğrulama | %15 | Birim, kâhinle karşılaştırmalı ve stres testleri; testlerin otomatik çalıştırılması. |
| Ölçüm raporu | %15 | İlk ve son sürümlerin karşılaştırmalı ölçümü; grafikler; kuramla tutarlılık yorumu. |
| Kod kalitesi ve bellek güvenliği | %10 | Kod standardına uyum (Yol Haritası 6.5), uyarısız derleme, sızıntısız çalışma (ASan/Valgrind). |
| Sunum | %10 | Canlı demo, tasarım kararlarının ve ölçüm bulgularının anlatımı. |

Her ölçüt dört düzeyde değerlendirilir (Yetkin (%90–100), Gelişmiş (%70–89), Gelişmekte (%40–69), Yetersiz (%0–39)); düzeylerin tanımları ve her ölçütün puanının alt kalemlere dağılımı Ek B'dedir (Tablo 14.27). Örneğin işlevsel doğrulukta “bütünleştirme yarım (modüller ayrı programlarda)”, testte “kâhin sınanan fonksiyonu çağırıyor”, analizde “sezgisel çözüm ‘en iyi’ diye sunulmuş” Gelişmekte düzeyidir.

## 10.7 Ölçüm raporu, rapor şablonu ve sunum

**Ölçüm raporu** (yönerge § 5): **Dönem kıyaslaması:** varsayılan projede en az sekiz işlem için ilk ve son gerçekleştirim (ör. yükleme v1.0 ↔ v2.5; numarayla arama v1.0 ↔ v2.9; yinelenen tespiti v2.1 ↔ v2.9; başarı listesi v2.4 ↔ v2.6; kayıt kuyruğu v2.7 ↔ v2.8; ders planı matris ↔ liste; ders seçimi v2.2 ↔ v2.12; öneri sözlüğü her sorguda ↔ saklanan). İki gerçekleştirim **aynı programda, aynı derlemeyle** ölçülür. **Ölçek:** algoritmik gerçekleştirimler $10^6$ kayıtta; karesel gerçekleştirimler $10^4$–$10^5$'te ölçülüp $10^6$ için $T(N) = T(n)\cdot(N/n)^2$ **kestirimiyle** (işaretli); üstel yöntemlerde düğüm sayısı ya da büyüme oranıyla kestirim. **Düzen:** her nokta için 1 ısınma + en az 3 tekrar, ortanca; süre bütçesini aşan noktada tek ölçüm (raporda belirtilir); `-O2`; sayaçlar süreden ayrı çalıştırmada (Sedgewick & Wayne, 2011). Ayrıca en az üç işlemde `-O0` ↔ `-O2` karşılaştırması (derleyici kazancı ↔ algoritma kazancı). **Kuramla karşılaştırma:** oran testleri, log–log eğimler, sayaçların kapalı biçimleri; sapmalar (önbellek, sabitler) ölçümle tartışılır (Cormen vd., 2022). **Tartışma:** sezgisel ↔ alt sınır ↔ kesin (NP-zor alt problemler), sınırlılıklar, makine bilgisi; ek olarak test kapsamı. **Biçim:** en çok 15 sayfa (ekler hariç), PDF + Markdown kaynağı; her sayı teslim edilen CSV'lerden betikle üretilir.

**Rapor şablonunun yapısı** (`FINAL_RAPOR_SABLONU.md`; en çok 15 sayfa, ekler hariç): Özet · 1. Amaç ve araştırma soruları · 2. Sistem tasarımı · 3. Algoritma seçimleri · 4. Doğrulama · 5. Ölçüm yöntemi · 6. Sonuçlar · 7. Kuramla karşılaştırma ve tartışma · 8. Sınırlılıklar ve geçerlilik tehditleri · 9. Sonuç ve genişletme seçenekleri · Ek A. Yeniden üretme · Ek B. Ham veri dosyaları · Ek C. Test kapsamı · Ek D. Beyan · Kaynaklar. Araştırma soruları S1–S6 şablonda hazırdır (ilk ↔ son farkı, kuramla uyum, algoritma ↔ derleyici kazancı, darboğaz, sezgisel ↔ alt sınır, ölçeklenebilirlik sınırı). Sayılar Türkçe biçimde (77,5; 4 932), her sayı CSV'den betikle; kestirimler etiketli; şekillerde yazı ≥ 9 pt, eksen adı, birim ve ölçek türü yazılı. Başvuru örneği v3.0 ölçüm raporudur: biçim içindir, sayılarınız kendi makinenizden gelmelidir.

**Sunum** (yönerge § 9): öğrenci başına 15 dakika: **10 dakika sunum** (canlı demo dahil; demo en az 3 dakika) + **5 dakika soru**. Süre aşımında sunum durdurulur. Sunum dosyası 10–12 slaytlık bir PDF'tir; slaytta kod dökümü yoktur ve her ölçüm sayısı raporunuzdaki CSV'den gelir. Canlı demo **temiz bir kopyadan** yapılır: derleme, `sunum/demo.txt` betiği (en az bir hata durumu ve bir geri alma), büyük veride bir komut ve bir ölçüm noktası. Önerilen slayt akışı, demonun ayrıntısı ve soru türleri Ek B'dedir.

Teslimden önce yönerge § 12'deki denetim listesini temiz bir kopyada uygulayın (`make temiz && make`, `make test`, `make asan`, `make valgrind`, tek oturumda `make olcum`, README'de kabul ölçütlerinin ✓/✗ listesi, `make temiz`).

# 11. Alıştırmalar ve Atölyeler

Türler: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez, D) hata bulma, E) Parsons (Parsons ve Haden, 2006), F) kod yazma, G) diller arası okuma. Kodlu alıştırmaların dosyaları `kodlar/alistirmalar/` altındadır: önce **çalıştırmadan** tahmin edin, sonra çalıştırıp karşılaştırın. Kod yazarken önce kaba kuvvet bir kâhin yazın; C/C++'ı AddressSanitizer ile çalıştırın. Çözümler öğretim üyesindedir.

## 11.1 İzleme ve çıktı tahmini (A)

**Alıştırma 14.1** ★☆☆ *Çıktı tahmini*

Altı şehirde en yakın komşu (Örnek 3'ün kuralı: kare uzaklık, eşitlikte küçük indis) 0 ve 3 numaralı şehirlerden başlatılıyor. Kare uzaklıklarla iki turu ve uzunluklarını çalıştırmadan yazın. Hangi başlangıç daha kısa tur verir? Turlardan biri en iyi mi; nasıl denetlersiniz?

**alistirmalar/H14_A01_nn_izleme.c · satır 15–16**

```
15  static const int x[N] = {0, 8, 2, 7, 4, 1};
16  static const int y[N] = {0, 1, 5, 6, 3, 9};
```

**Alıştırma 14.2** ★★☆ *İzleme*

Dört şehrin uzaklık matrisi (satır i, sütun j): 0: [0, 4, 7, 3]; 1: [4, 0, 2, 6]; 2: [7, 2, 0, 5]; 3: [3, 6, 5, 0]. Held–Karp tablosunu (bütün $S \ni 0$ ve $j \in S \setminus \{0\}$ için dp[S][j] ve ebeveyn) küme numarası sırasıyla doldurun; yanıtı ve turu geri çıkarın. Kaç durum hesapladınız? Sonucu $(4-1)!/2 = 3$ farklı turun uzunluklarıyla karşılaştırın.

**Alıştırma 14.3** ★★☆ *İzleme*

Şehirler 0(0, 0), 1(6, 0), 2(6, 4), 3(0, 4), 4(3, 6), 5(3, -2); başlangıç turu 0-2-1-5-3-4-0. İlk iyileştirmeli 2-opt'u (Örnek 4'ün tarama sırası) elle uygulayın: her uygulanan hamlenin (i, j)'sini, çıkan ve giren kenarlarını, kazancını ve yeni uzunluğu yazın. Kaç hamle denediniz? Son tur en iyi mi?

## 11.2 Karmaşıklık (B)

**Alıştırma 14.4** ★★☆ *Karmaşıklık*

(a) Saniyede $10^9$ tur değerlendiren bir makinede kaba kuvvet ($(n-1)!/2$ tur) ve Held–Karp ($n^2 \cdot 2^n$ geçiş) n = 13, 15, 17 ve 20 şehirde ne kadar sürer? (b) Held–Karp hücre başına 9 bayt (8 dp + 1 ebeveyn) kullanırsa n = 20, 25 ve 30 için kaç bayt gerekir? (c) 2-opt'un bir tam taraması n = 100 ve 1000'de kaç hamle değerlendirir? Hamle değerlendirmek neden O(1), uygulamak neden O(n)'dir?

**Alıştırma 14.5** ★★☆ *Karmaşıklık*

`alistirmalar/H14_A05_polinom_mu.c` dört fonksiyonun iş sayacını girdideki sayının bit uzunluğu b'ye göre yazdırır (b = 8, 16, 24, …): hızlı üs alma, onluk basamak toplamı, deneme bölmesiyle asallık testi (`asal_mi`, satır 49–62) ve alt küme toplamı DP'si. Önce kestirin: hangileri **girdi uzunluğunda** polinom, hangileri sözde polinomdur? Sonra çalıştırıp sınayın; asallık testinin sayacı neden b her 8 arttığında yaklaşık 16 katına çıkar?

## 11.3 Değişmez yazma (C)

**Alıştırma 14.6** ★★☆ *Değişmez yaz*

En yakın komşu sezgiselinin dış döngüsü (k = 1 … n − 1) için her yinelemenin sonunda tur[0..k] ve ziyaret[] hakkında doğru olan değişmezi yazın. Başlatma, koruma ve sonlanmadan turun geçerli bir permütasyon olduğunu çıkarın. Neden “tur en kısadır” biçiminde bir değişmez yazılamaz? Kısa bir karşı örnekle gösterin.

**Alıştırma 14.7** ★★★ *Değişmez yaz*

(a) Held–Karp için $|S| = k$ olan bütün durumlar hesaplandıktan sonra doğru olan değişmezi dp[S][j]'nin anlamıyla yazın; k üzerinde tümevarımla kanıtlayın (en iyi alt yapı adımını açıkça belirtin). Küme numarası sırası neden yeterlidir? (b) İlk iyileştirmeli 2-opt'un her durumda sonlandığını kanıtlayın. Sonlandığında hangi özellik doğrudur, hangisi doğru olmayabilir?

## 11.4 Hata bulma (D)

**Alıştırma 14.8** ★★☆ *Hata bul*

Program en yakın komşu turunu kaba kuvvet kâhiniyle karşılaştırıyor; beş örneğin dördünde oran 1'den küçük: sezgisel en iyiden kısa! Hatalı satırı bulun, nedenini açıklayın ve düzeltin. Düzeltmeden sonra oranlar hangi aralıkta olmalı? Kâhin aynı hatalı yardımcıyı kullansaydı hata yakalanır mıydı?

**alistirmalar/H14_A08_hata_bul_nn.c · satır 29–36**

```
29  static double tur_uzunlugu(const int *tur)
30  {
31      double t = 0.0;
32      for (int k = 1; k < N; k++) {
33          t += d[tur[k - 1]][tur[k]];
34      }
35      return t;
36  }
```

**Alıştırma 14.9** ★★☆ *Hata bul*

İlk iyileştirmeli 2-opt'tan sonra çalışan denetim, 100 örneğin yarıdan fazlasında hâlâ kısaltan bir hamle buluyor ve hepsinde ilk hamle (2, 7). Hatalı satırı bulun; hangi kenar çiftlerinin hiç denenmediğini açıklayın, düzeltin. Düzeltilmiş sürümde #24'ün sonucu ne olmalı (Örnek 4)?

**alistirmalar/H14_A09_hata_bul_iki_opt.c · satır 74–79**

```
74          for (int i = 1; i < N - 1 && !degisti; i++) {
75              for (int j = i + 1; j < N - 1 && !degisti; j++) {
76                  int a = tur[i - 1], b = tur[i], c = tur[j];
77                  int e = tur[(j + 1) % N];
78                  double kazanc = (d[a][c] + d[b][e])
79                                  - (d[a][b] + d[c][e]);
```

## 11.5 Parsons (E)

**Alıştırma 14.10** ★★☆ *Parsons*

Held–Karp'ın geri çıkarma fonksiyonunun karıştırılmış satırları; iki satır fazladır (dosya bu hâliyle derlenmez). Doğru sırayı ve girintiyi yazın; Örnek 2'nin beş şehrinde son = 2 ile tur[] = 0 4 1 3 2 olmalı. Fazla iki satır neden yanlıştır? Birini kullanan bir sürümün ilk yinelemede hangi tablo hücresini okuyacağını söyleyin.

**alistirmalar/H14_A10_parsons_held_karp.c · satır 9–21**

```
        S |= 1 << son;
    tur[0] = son;
        tur[k] = son;
{
    for (int k = n - 1; k >= 1; k--) {
        son = i;
static void geri_cikar(int son, int *tur)
        int i = ebeveyn[(S ^ (1 << son)) * n + son];
    }
    int S = (1 << n) - 1;
        S ^= 1 << son;
}
        int i = ebeveyn[S * n + son];
```

## 11.6 Kod yazma (F)

**Alıştırma 14.11** ★☆☆ *Kod yaz*

**Tur doğrulayıcı (sertifika).** Gezgin satıcının karar sürümü (“uzunluğu en çok L olan tur var mı?”) için doğrulayıcı yazın: girdi şehirler, L ve bir şehir sırası; çıktı KABUL/RED, neden ve incelenen şehir sayısı. Kurallar: tam n şehir, indisler 0..n − 1'de, tekrar yok, kapalı tur ≤ L (EPS ile); başlangıç serbest. Örnek 2'nin şehirleri ve L = 20 ile sınayın: {0, 2, 3, 1, 4}, {0, 4, 2, 3, 1}, {0, 2, 2, 1, 4}, {0, 2, 3, 1}, {0, 2, 3, 1, 7}, {3, 1, 4, 0, 2}. Doğrulayıcınız O(n) mi? C, C++ ve Python.

**Alıştırma 14.12** ★★☆ *Kod yaz*

**En ucuz ekleme sezgiseli.** Tur şehir 0 ve ona en yakın şehirle başlar; her adımda turda olmayan c ve turdaki (a, b) kenarı için d(a, c) + d(c, b) − d(a, b) en küçük olanı seçip c'yi a ile b'nin arasına ekleyin (eşitlikte küçük c, sonra küçük konum). Örnek 2'de adımları yazdırın; Örnek 3'ün 100 örneğinde (tohum 2026, n = 8) kaba kuvvete oranın en kötü, ortalama ve ortancasını en yakın komşununkiyle karşılaştırın. Karmaşıklığı nedir?

**Alıştırma 14.13** ★★☆ *Kod yaz*

**En iyi iyileştirmeli 2-opt.** Her taramada bütün (i, j) çiftlerini değerlendirip en çok kısaltan hamleyi uygulayın (eşitlikte ilk bulunan). Örnek 4'ün ilk iyileştirmesiyle karşılaştırın: #24'te adımlar; 100 örnekte en kötü/ortalama oran, "= en iyi" sayısı, ortalama hamle ve denenen hamle; n = 25, 50, 100'de (tohum 7) son uzunluk. Hangisi daha az hamle, hangisi daha az değerlendirme yapar? Biri hep daha kısa tur verir mi?

**Alıştırma 14.14** ★★★ *Kod yaz*

**Profil ödevi.** `alistirmalar/H14_A14_yavas_rapor.c` (ve Python eşi) ders kayıtlarından doğru ama yavaş bir rapor üretir. (1) m = 6000 ve m = 40 000 için süreyi ölçün; (2) gprof ve cProfile ile profil çıkarıp zamanın nerede geçtiğini tablolayın; (3) yalnız darboğaz(lar)ın **algoritmasını** değiştirin (hangi veri yapısı ya da sıralama?) ve stdout'un bayt bayt aynı kaldığını `cmp` ile gösterin; (4) hızlanmayı iki m için verin ve büyüme sınıflarıyla açıklayın. Ders listesindeki doğrusal arama neden önemsizdir?

**Alıştırma 14.15** ★★★ *Kod yaz*

**Modelleme: sınav programı → graf boyama.** 12 sınav ve her biri 3 farklı sınava giren 12 öğrenci ortak LCG ile üretilsin (tohum 4). Çakışma grafını kurun (aynı öğrencinin iki sınavı arasında kenar); açgözlü boyama (derecesi büyük köşe önce, eşitlikte küçük indis; komşularda olmayan en küçük renk) ve geri izlemeyle en az renk sayısını (k = 1, 2, ...) bulun; oturumları yazdırın. Tohum 1..50'de açgözlü kaç örnekte en az renge eşittir? Açgözlü sonuç neden "en iyi" diye sunulamaz?

## 11.7 Diller arası okuma (G)

**Alıştırma 14.16** ★★☆ *Diller arası okuma*

Python programı Örnek 2'nin 24 turunun en kısasını ve en uzununu `min`/`max` ile seçiyor; C eşi (`alistirmalar/H14_A16_esitlik_kirma.c`, 40–47. satırlar) aynı turları aynı sırayla bir döngüde `<=` ve `>=` ile seçiyor. İki programın iki satırını çalıştırmadan yazın. Hangi satır farklı çıkar, neden? En uzun tur ile tersi de matematiksel olarak eşit uzunluktadır; iki program yine de neden aynı turu yazar? Örnek 2 bu sorunu nasıl önler?

**alistirmalar/H14_A16_esitlik_kirma.py · satır 32–35**

```
32  def main():
33      turlar = [(0,) + p for p in itertools.permutations(range(1, 5))]
34      yaz("en kısa", min(turlar, key=uzunluk))
35      yaz("en uzun", max(turlar, key=uzunluk))
```

**Alıştırma 14.17** ★★☆ *Diller arası okuma*

C programı farklı tur sayısını (n − 1)!/2 32 bitlik işaretsiz tamsayıyla, Python eşi (`alistirmalar/H14_A17_tur_sayisi.py`) Python tamsayısıyla hesaplıyor. C çıktısı ilk hangi n'de yanlışlaşır, o satırda hangi sayı yazar? İşaretsiz taşma C'de tanımlı mıdır? İşaretli `int` kullanılsaydı ne değişirdi? `uint64_t` ile sınır nereye kayar (K01)?

**alistirmalar/H14_A17_tur_sayisi.c · satır 17–23**

```
17      uint32_t f = 1;
18      for (int n = 2; n <= 16; n++) {
19          f *= (uint32_t)(n - 1);           /* f = (n − 1)! */
20          if (n >= 3) {
21              printf("n = %2d: (n-1)!/2 = %u\n", n, (unsigned)(f / 2));
22          }
23      }
```

## 11.8 Atölye 1 · Problem–strateji eşleme turnuvası (takımlar, ≈ 30 dk)

ALGORİTMA ATÖLYESİ · **12 problem kartı, dört takım**

Sınıf dört takıma ayrılır. Her turda bir kart okunur; takımlar 2 dakika içinde kâğıda **strateji**, **algoritma**, **karmaşıklık** ve **bir cümlelik gerekçe** yazar. Puanlama: doğru strateji 2, doğru karmaşıklık 1, gerekçe 1; başka bir takımın seçiminin neden yanlış olduğunu bir **karşı örnekle** gösteren takım 1 bonus alır. Tartışmada strateji karar ağacı (Şekil 14.2) hakemdir; birden çok doğru yanıt olabilir (ör. hem karma hem ikili arama).

- **K1.** Kütüphane kataloğunda $10^6$ kitap ISBN'e göre sıralı; günde $10^5$ "bu kitap var mı?" sorgusu.
- **K2.** Sosyal ağda iki kişi arasındaki en kısa arkadaşlık zinciri.
- **K3.** Bağımlılıkları olan 30 proje görevi için geçerli bir yapılış sırası.
- **K4.** Kargo aracının 40 teslimat noktalı günlük rotası; bir dakikada iyi bir rota.
- **K5.** Belleğe sığmayan $10^8$ satırlık kayıt dosyasını parçalar hâlinde sıralayıp birleştirmek.
- **K6.** Kasada {1, 5, 10, 25} kuruşla para üstü: en az madeni para.
- **K7.** Yazılan bir sözcüğe sözlükte en yakın sözcükler (yazım düzeltme).
- **K8.** Yazıcıya gönderilen işleri geliş sırasıyla basmak.
- **K9.** 5 × 5 tahtada atın her kareye bir kez uğradığı bir tur.
- **K10.** Web sunucusunda son bir saatteki farklı ziyaretçi (IP) sayısı.
- **K11.** Santralden her ilçeye en az kayıplı iletim yolu (kayıplar negatif değil).
- **K12.** Robot kolunun 14 lehim noktasını dolaşma sırası; en kısa yol kanıtlanmalı.

Turnuva sonunda her takım en zor bulduğu kartı ve o kartta hangi sorunun (girdi boyutu mu, yapı mı, kesinlik mi?) kararı belirlediğini anlatır.

## 11.9 Atölye 2 · Gezgin satıcı yarışması: 50 şehirde en kısa tur (≈ 40 dk)

ALGORİTMA ATÖLYESİ · **Kendi sezgiselinizi yazın, hakem programı ölçsün**

Elli şehir ortak LCG ile üretilir (tohum 50, koordinatlar 0..999). Hakem programı `kodlar/alistirmalar/H14_A18_gs_yarismasi` (C ve Python) argümansız çalıştırılınca şehirleri ve iki taban çizgisini yazar: taban 1, en yakın komşu (şehir 0'dan): 7404.06; taban 2, + 2-opt (ilk iyileştirme) : 6115.39 (17 iyileştirme). Takımınız bir tur dosyası üretir (50 indis, her biri bir kez; örnek biçim `girdiler/H14_A18_ornek_tur.txt`) ve hakem dosyayı doğrular, uzunluğu yazar: `./H14_A18_gs_yarismasi tur.txt`. Geçersiz tur (eksik, tekrar, dizi dışı indis) iletisiyle reddedilir.

- **Kurallar:** programınız ortak LCG ile aynı şehirleri üretir, bu makinede en çok 60 saniye çalışır ve

belirleyicidir (aynı tohum → aynı tur); kod ve tur dosyası birlikte teslim edilir.

- **Fikirler:** her başlangıçtan en yakın komşu; en ucuz ekleme (Alıştırma 14.12); 2-opt'un ilk ve en iyi iyileştirmesi (Örnek 4, Alıştırma 14.13); Or-opt; çok başlangıçlı yerel arama; zaman bütçesi içinde rastgele yeniden başlatma (LCG ile).
- **Sıralama:** en kısa geçerli tur kazanır; eşitlikte daha kısa çalışma süresi. Her takım turunun taban 2'ye oranını ve kullandığı yöntemlerin karmaşıklığını sunar.
- **Tartışma:** kazanan turun en iyi olduğunu kanıtlayabilir miyiz? 50 şehirde kaba kuvvet ve Held–Karp neden olanaksızdır? Bir alt sınır (ör. en küçük kapsayan ağacın uzunluğu) bize ne söyler?

### 11.10 Atölye 3 · Final projesi sunumları: canlı demo, ölçüm raporu ve kod incelemesi

ALGORİTMA ATÖLYESİ · **Sunum oturumu (yönerge § 9; öğrenci başına 15 dk)**

Her öğrenci 10 dakika sunar (en az 3 dakikası canlı demo; düzen Bölüm 10.7'de ve Ek B'de) ve 5 dakika soru yanıtlar. Dinleyiciler üç gözlemci rolünden birini üstlenir ve kısa bir form doldurur (akran değerlendirmesi isteğe bağlıdır ve puana katılmaz):

- **Canlı demo gözlemcisi:** temiz kopyadan derleme, hata durumu ve geri alma gösterildi mi; çıkış kodları ve iletiler tek tip mi?
- **Ölçüm raporu gözlemcisi:** her sayı CSV'den mi geliyor; log–log grafik, eğim ya da oran testi var mı; kestirimler işaretli ve kuramdan sapma ölçümle açıklanmış mı?
- **Kod incelemesi gözlemcisi:** seçilen fonksiyonun değişmezi, karmaşıklığı ve bellek sahipliği açıklandı mı; kâhin sınanan koddan bağımsız mı; sezgisel sonuç “en iyi” diye sunulmuş mu?

Oturum sonunda sınıf, en iyi üç tasarım kararını ve en sık üç hatayı listeler; liste bir sonraki dönemin final yönergesine geri bildirim olur.

## 12. Kendini Değerlendirme ve Final Hazırlığı

- ☐ Bir problemi strateji karar ağacıyla bir stratejiye eşleyip seçimimi girdi boyutu ve yapıyla gerekçelendirebiliyorum (ÖÇ 14.1).
- ☐ Labirenti BFS'ye, ders planını topolojik sıralamaya, sınav programını graf boyamaya indirgeyip modeli (köşe, kenar, ağırlık) yazabiliyorum (ÖÇ 14.2).
- ☐ P, NP, NP-tam ve NP-zor kavramlarını doğrulama ile bulma farkıyla açıklayabiliyor; “NP = polinom olmayan” yanılgısını düzeltebiliyorum (ÖÇ 14.3).
- ☐ Sözde polinom zamanı bit uzunluğuyla açıklayabiliyor; durma probleminin neden karar verilemez olduğunu çelişkiyle anlatabiliyorum.
- ☐ Gezgin satıcıyı kaba kuvvet, en yakın komşu, 2-opt ve Held–Karp ile çözebiliyor; yaklaşım oranını kâhinle ölçebiliyorum.
- ☐ Bir programın profilini çıkarıp darboğazı bulabiliyor ve algoritmik iyileştirmeyi ölçümle belgeleyebiliyorum (ÖÇ 14.4).
- ☐ Derleyici, dil ve algoritma kazançlarını aynı tabloda n'yi belirterek karşılaştırabiliyorum.
- ☐ Final projemi tasarlayıp birim + kâhin + stres testleriyle doğrulayabiliyor, rapor ve canlı demoyla sunabiliyorum (ÖÇ 14.5).

## 12.1 H1–H14 konu haritası

Dönemin 74 kimlik kartı beş modüle dağılır (modül, haftalar, kart sayısı): I (H1–H3) 11; II (H4–H7) 21; III (H8–H10) 20; IV (H11–H12) 13; V (H13–H14) 9. Her haftanın konusunu, algoritmalarını en kötü durum karmaşıklıklarıyla ve ait oldukları stratejileri veren konu haritası Ek B'dedir (Tablo 14.28); çalışırken her kartı kâğıda yeniden yazmak iyi bir tekrardır.

## 12.2 Soru türleri ve soru bankası

**Tablo 14.16**
*Final sınavı soru türleri ve final hazırlık soru bankası (73 soru; H1–H14 her haftadan en az 5; yanıt harfleri dengeli)*

| Soru türü (yol haritası Tablo 10.2) | Örnek | Ölçtüğü beceri | Bankada |
|---|---|---|---|
| İzleme | “Bu dizide hızlı sıralamanın ilk bölümlemesinden sonraki durum nedir?” | izleme, kavramsal makine | 11 |
| Karmaşıklık | “Bu kodun en kötü durum karmaşıklığı nedir? Gerekçelendiriniz.” | analiz | 11 |
| Değişmez | “Verilen döngünün değişmezini yazınız / verilen ifade değişmez midir?” | doğruluk | 10 |
| Hata bulma | “Bu ikili arama hangi girdide sonsuz döngüye girer?” | test ve hata ayıklama | 10 |
| Parsons / kod tamamlama | karışık satırlardan BFS'yi oluşturma | yapı bilgisi | 10 |
| Strateji seçimi | “Bu problem için açgözlü yaklaşım doğru mudur? Karşı örnek veriniz.” | tasarım ve gerekçelendirme | 11 |
| Diller arası okuma | “Bu Python parçası hangi çıktıyı verir; C karşılığı aynı çıktıyı verir mi?” | dil farkı farkındalığı | 10 |
| Kod yazma | kısa bir algoritmanın C ile gerçekleştirimi | gerçekleştirim | alıştırmalar (F) |

Final hazırlık soru bankası web sayfasındadır: hafta ve türe göre süzülür, her yanıttan sonra açıklama ve **doğrulama kaydı** (yanıtın hangi program çıktısından ya da ölçümden geldiği) gösterilir. Bankanın her yanıtı programla doğrulanmıştır: kodlu soruların programları derlenip çalıştırıldı, kod tamamlama sorularının dört seçeneği ayrı ayrı derlenip yalnız doğru seçeneğin beklenen çıktıyı verdiği denetlendi, sayısal yanıtlar çıktı dosyalarından ve deney CSV'lerinden hesaplandı. Bankadan iki örnek soru Ek B'dedir.

## 12.3 Çalışma planı (iki hafta)

Önerilen plan ilk on bir günde modülleri sırayla tekrar eder (kimlik kartlarını yeniden yazma, elle izleme, bankanın o haftalara ait soruları); son üç günde bankanın tamamı süreyle çözülür, yanlışlar türe göre gruplanır ve deneme sınavı yapılır. Gün gün plan Ek B'dedir (Tablo 14.29).

# 13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · **Bu haftanın konuları nerede derinleşiyor?**

- **Veri Yapıları:** ağaçlar, dengeli arama ağaçları (AVL, kırmızı–siyah), B-ağaçları, ileri graf yapıları. **Algoritma Analizi ve Tasarımı:** amortize analiz, ağ akışları, rastgele algoritmalar, stratejilerin kanıt teknikleri (Kleinberg ve Tardos, 2006; Skiena, 2020).
- **Hesaplama Kuramı / Biçimsel Diller ve Özdevinirler:** Turing makineleri, karar verilebilirlik, P ve NP'nin biçimsel tanımı; Cook (1971) ve Karp (1972) indirgemeleri. **İşletim Sistemleri:** profilleyicilerin dayandığı zamanlayıcı kesmeleri, sanal bellek ve önbellek hiyerarşisi (Bölüm 6'daki L2 → L3 geçişi).
- **Yöneylem Araştırması:** tamsayı programlama, dal ve kesme; gezgin satıcının kesin çözücüleri (Applegate vd., 2006). **Yapay Zekâ:** sezgisel arama (A*), yerel arama, meta-sezgiseller. **Paralel Programlama:** dönem çekirdeklerinin çok çekirdekli sürümleri.
- **Yarışma programlama ve teknik mülakat:** ICPC türü yarışmalar "girdi boyutundan stratejiye" akıl yürütmesini dakikalar içinde ister; final bankası iyi bir başlangıçtır. **Açık kaynak katkısı:** kâhin, belirleyicilik ve ölçüm alışkanlıkları gerçek projelerde de değerlidir; küçük bir hata düzeltmesi ya da test katkısıyla başlanabilir.

## 13.1 Merak Kutusu: Held–Karp neden 16–18 şehirde durur?

Held ve Karp (1962) gezgin satıcıyı $\Theta(n^2 \cdot 2^n)$ zamanda çözdü; kaba kuvvetin $\Theta(n!)$'ine göre devasa bir kazanç. Ama tablo $n \cdot 2^n$ hücredir: Örnek 5'in hesabıyla n = 16'da 9 MiB, n = 18'de 40,5 MiB. Her ek şehir belleği iki katından fazla büyütür. Kesin çözücüler bu yüzden başka yoldan gider: turu doğrusal programlamayla alttan sınırlayıp dal ve kesmeyle aramayı budamak (Applegate vd., 2006).

MERAK KUTUSU · **Soru**

Held–Karp'ın tablosunu katmanlara ($|S| = k$) bölüp yalnız iki katmanı tutsaydık bellek ne kadar azalırdı? En büyük katman hangisidir (ipucu: $C(n-1, k)$'nin en büyük olduğu k)? Geri çıkarma için ne kaybederdik?

## 13.2 Merak Kutusu: derleyicim neden "bu döngü sonsuz" demiyor?

Durma problemi (Bölüm 3.5) yalnız bir kuram sonucu değildir: "bu program her girdide biter mi?", "bu değişken hiç sıfır olur mu?" gibi sorulara **her programda** doğru yanıt veren bir araç yazılamaz. Derleyici uyarıları ve statik çözümleyiciler bu yüzden ya bazı hataları kaçırır ya da bazı doğru programlara yanlış alarm verir; ikisinden birini seçmek zorundadır. Pratikte boşluğu testler, kâhinler ve çalışma zamanı denetimleri (AddressSanitizer, UBSan) doldurur.

MERAK KUTUSU · **Soru**

Sonsuz döngüyü **bazı** programlarda kesin olarak bulan basit bir denetim tasarlayın (ör. değişmeyen bir koşul). Durma problemi bu denetimin varlığıyla çelişir mi? Neden?

# 14. Özet, Kavram Haritası ve Terim Sözlüğü

- Strateji seçimi iki bilgiyle başlar: girdi boyutu (uygulanabilir karmaşıklık) ve problemin yapısı (sıralı mı, graf mı, örtüşen alt problem mi, yerel seçim güvenli mi, NP-zor mu?). Dönemin kimlik kartları on bir stratejiye dağılır.
- İndirgeme yeni bir problemi bilinen bir çözücüye çevirir: labirent → BFS, ders planı → topolojik sıralama, sınav programı → graf boyama, ders seçimi → 0/1 sırt çantası.

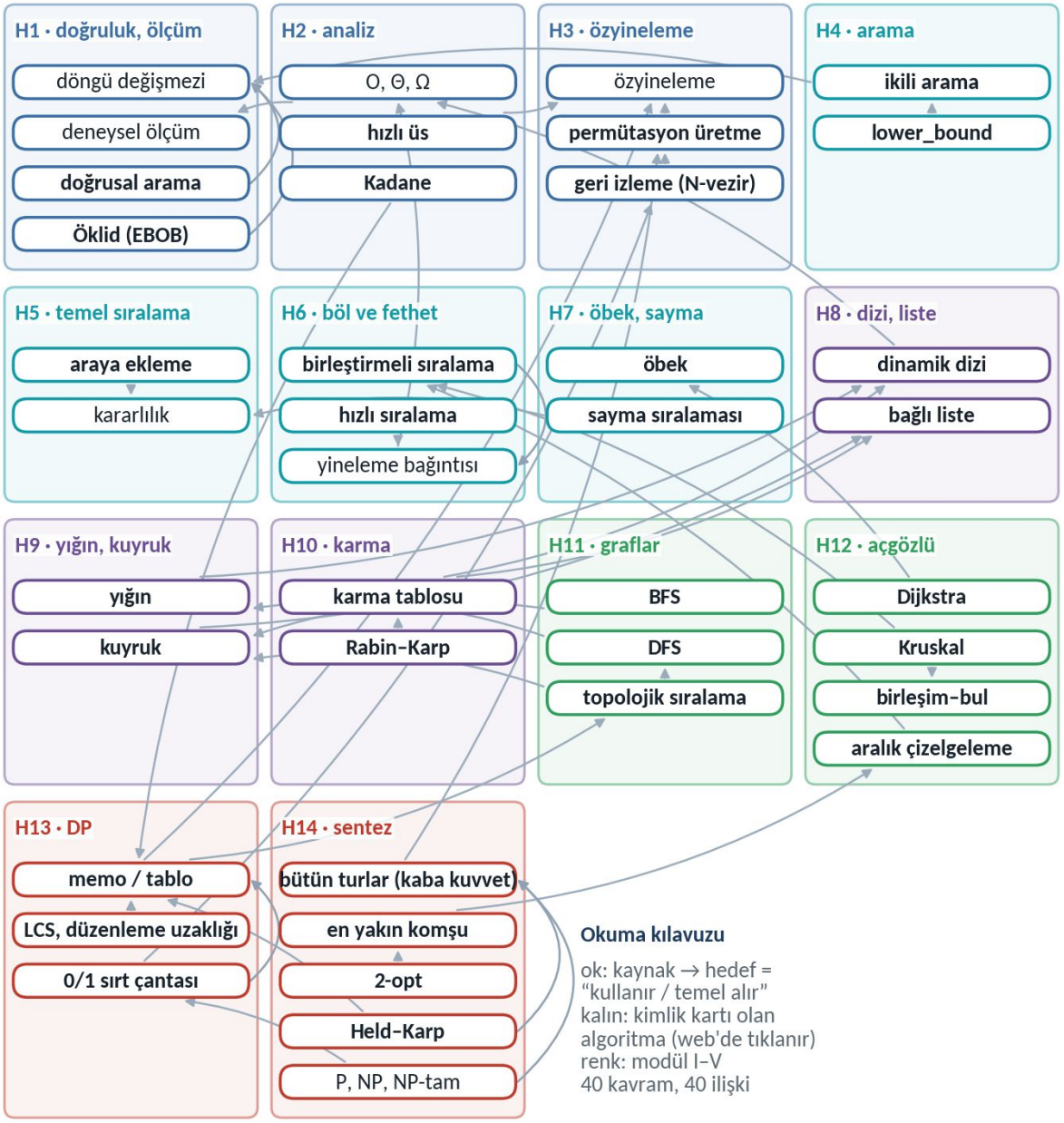
- P çözülebilen, NP doğrulanabilen karar problemleridir; P ⊆ NP. NP-tam problemlerden biri polinom zamanda çözülürse hepsi çözülür; P = NP sorusu açıktır. Sözde polinom algoritmalar sayının değerinde polinom, onu yazan bit sayısında üsteldir. Durma problemi karar verilemez.
- Gezgin satıcı: kaba kuvvet $\Theta(n!)$, Held–Karp $\Theta(n^2 \cdot 2^n)$, en yakın komşu $\Theta(n^2)$ (100 örnekte ortalama oran 1,0964), 2-opt yerel en iyiye yakınsar (NN + 2-opt ortalama 1,0028).
- Performans mühendisliği: doğru → ölç → profil → algoritma → mikro iyileştirme. Algoritmik kazanç n ile büyür; derleyici ve dil kazancı sabit çarpandır.
- Final projesi: tek uygulama, birim + kâhin + stres + regresyon testleri, ölçüm raporu ve canlı demo; rubrik %25 + %25 + %15 + %15 + %10 + %10.

**Tablo 14.17**

*Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu; dönemin öbür terimleri önceki haftaların sözlüklerinde)*

| Türkçe | İngilizce | Not |
|---|---|---|
| algoritma tasarım stratejisi; indirgeme; modelleme | algorithm design strategy; reduction; modeling | “bunu daha önce gördüm mü?” |
| karar problemi; sertifika; doğrulayıcı | decision problem; certificate; verifier | evet yanıtının kanıtı |
| P; NP; NP-tam; NP-zor | P; NP; NP-complete; NP-hard | P = NP açık soru |
| sözde polinom zaman | pseudo-polynomial time | değerde polinom, bit uzunluğunda üstel |
| durma problemi; karar verilemez | halting problem; undecidable | Turing (1936/1937) |
| gezgin satıcı problemi; Hamilton devresi | traveling salesman problem (TSP); Hamiltonian cycle | |
| sezgisel; yerel arama; yerel en iyi | heuristic; local search; local optimum | en yakın komşu, 2-opt |
| yaklaşık algoritma; yaklaşım oranı | approximation algorithm; approximation ratio | ρ = bulunan / en iyi |
| bit maskesi DP'si | bitmask dynamic programming | Held–Karp |
| profil çıkarma; darboğaz; alev grafiği | profiling; bottleneck; flame graph | gprof, perf, cProfile |
| örnekleme profilleyicisi; araçlama | sampling profiler; instrumentation | -pg |
| erken optimizasyon | premature optimization | Knuth (1974) |

**Şekil 14.12**
*Dönem kavram haritası: on dört haftanın 40 algoritma ve yapısı ile aralarındaki "kullanır / indirgenir / kâhinidir" ilişkileri (web'de etkileşimli sürümü)*




# 15. Kaynaklar

Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). *The traveling salesman problem: A computational study*. Princeton University Press.

Clay Mathematics Institute. (t.y.). *The Millennium Prize Problems*. Erişim tarihi 22 Ekim 2026, https://www.claymath.org/millennium-problems

Cook, S. A. (1971). The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing* (pp. 151–158). ACM. https://doi.org/10.1145/800157.805047

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, *6*(6), 791–812. https://doi.org/10.1287/opre.6.6.791

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, *10*(1), 196–210. https://doi.org/10.1137/0110015

Karp, R. M. (1972). Reducibility among combinatorial problems. In R. E. Miller & J. W. Thatcher (Eds.), *Complexity of computer computations* (pp. 85–103). Plenum Press. https://doi.org/10.1007/978-1-4684-2001-2_9

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson/Addison-Wesley.

Knuth, D. E. (1974). Structured programming with go to statements. *ACM Computing Surveys*, *6*(4), 261–301. https://doi.org/10.1145/356635.356640

Lin, S., & Kernighan, B. W. (1973). An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, *21*(2), 498–516. https://doi.org/10.1287/opre.21.2.498

Parsons, D., & Haden, P. (2006). Parson's programming puzzles: A fun and effective learning tool for first programming courses. In *Proceedings of the 8th Australasian Conference on Computing Education* (Vol. 52, pp. 157–163). Australian Computer Society.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Skiena, S. S. (2020). *The algorithm design manual* (3rd ed.). Springer.

Turing, A. M. (1937). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, *s2-42*(1), 230–265. https://doi.org/10.1112/plms/s2-42.1.230

University of Waterloo. (t.y.-a). *Milestones in the solution of TSP instances*. Traveling Salesman Problem. Erişim tarihi 22 Ekim 2026, https://www.math.uwaterloo.ca/tsp/history/milestone.html

University of Waterloo. (t.y.-b). *Optimal 85,900-point tour*. Traveling Salesman Problem. Erişim tarihi 22 Ekim 2026, https://www.math.uwaterloo.ca/tsp/pla85900/

University of Waterloo. (t.y.-c). *Optimal tour of Sweden*. Traveling Salesman Problem. Erişim tarihi 22 Ekim 2026, https://www.math.uwaterloo.ca/tsp/sweden/

# Ek A. Dil Karşılıkları

Bu ekte on örneğin C++ ve Python sürümlerinden çekirdek fonksiyonlar dosyadaki satır numaralarıyla verilir; öbür fonksiyonlar (yazdırma, girdi okuma, kâhinler) tam dosyalardadır: `kodlar/cpp/` ve `kodlar/python/`. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); gösterilen fonksiyonların dışındaki farklar not kutusunda açıklanır. Fonksiyon ve değişken adları üç dilde aynıdır; on örneğin hepsinde üç dilin çıktısı bayt bayt aynıdır ("=").

**Tablo 14.18**
*Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H14_03_en_yakin_komsu)*

| Dil | Derleme / çalıştırma | Not |
|---|---|---|
| C | `gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c [ortak_rastgele.c zamanlayici.c] -o X -lm · ./X` | ortak modüller yalnız #include eden örneklerde |
| C++ | `g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X` | ortak başlıklar (.hpp) yalnızca #include |
| Python | `python3 X.py (Windows: py X.py)` | Python 3.10 ve üstü; yalnızca standart kütüphane |
| Girdi | `./H14_07_indirgeme ../girdiler/H14_07_sudoku.txt ../girdiler/H14_07_labirent.txt ../girdiler/H14_07_ders_plani.txt · ./H14_10_miniobs_cekirdek < ../girdiler/H14_10_miniobs_cekirdek.txt` | girdi dosyaları kodlar/girdiler/ altında |

## A.1 Örnek 1 · Problem–strateji eşleme (P8: DP ↔ açgözlü)

C++ **cpp/H14_01_strateji_esleme.cpp · satır 269–294**

```cpp
static void p8_bozuk_para()
{
    const int para[3] = {1, 3, 4};
    std::vector<int> enaz(7, 1000), son(7, -1);
    enaz[0] = 0;
    for (int t = 1; t <= 6; t++) {
        for (int c : para) {
            if (c <= t && enaz[t - c] + 1 < enaz[t]) {
                enaz[t] = enaz[t - c] + 1;
                son[t] = c;
            }
        }
    }
    int acgozlu = 0, kalan = 6;
    for (int c = 2; c >= 0; c--) {
        while (para[c] <= kalan) {
            kalan -= para[c];
            acgozlu++;
        }
    }
    std::printf("    sonuç    : DP %d para (", enaz[6]);
    for (int t = 6; t > 0; t -= son[t]) {
        std::printf(t == 6 ? "%d" : " + %d", son[t]);
```

```
    }
    std::printf("); açgözlü %d para\n", acgozlu);
}
```

Python **python/H14_01_strateji_esleme.py · satır 225–248**

```python
def p8_bozuk_para():
    """P8 — DP: en az para; karşılaştırma için açgözlü"""
    para = [1, 3, 4]
    enaz = [1000] * 7
    son = [-1] * 7
    enaz[0] = 0
    for t in range(1, 7):
        for c in para:
            if c <= t and enaz[t - c] + 1 < enaz[t]:
                enaz[t] = enaz[t - c] + 1
                son[t] = c
    acgozlu = 0
    kalan = 6
    for c in range(2, -1, -1):
        while para[c] <= kalan:
            kalan -= para[c]
            acgozlu += 1
    parcalar = []
    t = 6
    while t > 0:
        parcalar.append(str(son[t]))
        t -= son[t]
    print(f"    sonuç    : DP {enaz[6]} para (" + " + ".join(parcalar)
          + f"); açgözlü {acgozlu} para")
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Problem kartları C/C++'ta yapı dizisi, Python'da demet listesidir. BFS kuyruğu C'de dizi + iki indis, C++'ta `std::queue`, Python'da `collections.deque` (H9'un kütüphane karşılığı). Bulunan yol C'de geriye sayan döngüyle, C++'ta `rbegin()`, Python'da `reversed` ile ters yazdırılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 30, 213, 232; Python 19, 65, 184, 199.)

## A.2 Örnek 2 · Gezgin satıcı: kaba kuvvet

C++ **cpp/H14_02_gs_kaba_kuvvet.cpp · satır 55–83**

```cpp
static void dene(int k, double kismi)
{
    cagri_sayisi++;
    if (k == n) {                                   // yaprak: tam tur
        if (yalniz_yari && yol[1] > yol[n - 1]) {
            return;                                 // tersi zaten sayıldı
        }
        double toplam = kismi + d[yol[n - 1]][0];   // dönüş kenarı
        tur_sayisi++;
        if (yaz_mi) {
            std::printf("    %3lld  ", tur_sayisi);
```

```
            tur_yaz(yol);
            std::printf("  %7.2f\n", toplam);
        }
        if (toplam < en_iyi - EPS) {
            en_iyi = toplam;
            en_iyi_tur = yol;
        }
        return;
    }
    for (int s = 1; s < n; s++) {
        if (!kullanildi[s]) {
            kullanildi[s] = 1;                      // seç
            yol[k] = s;
            dene(k + 1, kismi + d[yol[k - 1]][s]);  // dene
            kullanildi[s] = 0;                      // geri al
        }
    }
}
```

Python **python/H14_02_gs_kaba_kuvvet.py · satır 49–69**

```
def dene(k, kismi):
    """yol[0..k-1] dolu (yol[0] = 0); kismi = yol boyunca uzunluk"""
    global cagri_sayisi, tur_sayisi, en_iyi, en_iyi_tur
    cagri_sayisi += 1
    if k == n:                                      # yaprak: tam tur
        if yalniz_yari and yol[1] > yol[n - 1]:
            return                                  # tersi zaten sayıldı
        toplam = kismi + d[yol[n - 1]][0]           # dönüş kenarı
        tur_sayisi += 1
        if yaz_mi:
            print(f"    {tur_sayisi:3d}  {tur_metni(yol)}  {toplam:7.2f}")
        if toplam < en_iyi - EPS:
            en_iyi = toplam
            en_iyi_tur = yol[:]
        return
    for s in range(1, n):
        if not kullanildi[s]:
            kullanildi[s] = 1                       # seç
            yol[k] = s
            dene(k + 1, kismi + d[yol[k - 1]][s])   # dene
            kullanildi[s] = 0                       # geri al
```

DİL PENCERESİ · **Farklılaşan satırlar**

Uzaklıklar üç dilde de IEEE 754 çift duyarlıklı `sqrt` ile hesaplanır ve tur boyunca aynı sırayla toplanır; bu yüzden `%.2f` çıktıları bayt bayt aynıdır. En iyi turun kopyası C'de döngüyle, C++'ta vektör atamasıyla (`en_iyi_tur = yol`), Python'da dilimle (`yol[:]`) alınır; Python'da yalnız `en_iyi_tur = yol` yazmak aynı listeye ikinci bir ad verirdi ve tur değiştikçe kayıtlı 'en iyi' de değişirdi. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 26, 37; Python 42, 46, 132.)

## A.3 Örnek 3 · En yakın komşu

C++ cpp/H14_03_en_yakin_komsu.cpp · satır 62–93

```cpp
static void en_yakin_komsu(int bas, std::vector<int>& tur, bool ayrinti)
{
    std::vector<int> ziyaret(n, 0);
    tur.assign(n, 0);
    tur[0] = bas;
    ziyaret[bas] = 1;
    for (int k = 1; k < n; k++) {
        int son = tur[k - 1], sec = -1;
        long long en_az = 0;
        if (ayrinti) {
            std::printf("    %d     %d    ", k, son);
        }
        for (int s = 0; s < n; s++) {
            if (!ziyaret[s]) {
                long long dd = d2(son, s);
                aday_sayisi++;
                if (ayrinti) {
                    std::printf(" %d:%lld", s, dd);
                }
                if (sec == -1 || dd < en_az) {   // eşitlikte ilk (küçük)
                    sec = s;
                    en_az = dd;
                }
            }
        }
        if (ayrinti) {
            std::printf("  -> %d\n", sec);
        }
        tur[k] = sec;
        ziyaret[sec] = 1;
    }
}
```

Python python/H14_03_en_yakin_komsu.py · satır 48–71

```python
def en_yakin_komsu(bas, ayrinti):
    """En yakın komşu: bas'tan başla; ayrinti ise her adımı yaz"""
    global aday_sayisi
    ziyaret = [0] * n
    tur = [bas]
    ziyaret[bas] = 1
    for k in range(1, n):
        son = tur[k - 1]
        sec = -1
        en_az = 0
        satir = f"    {k}     {son}    "
        for s in range(n):
            if not ziyaret[s]:
                dd = d2(son, s)
                aday_sayisi += 1
                satir += f" {s}:{dd}"
                if sec == -1 or dd < en_az:      # eşitlikte ilk (küçük)
```

```
                    sec = s
                    en_az = dd
        if ayrinti:
            print(f"{satir}  -> {sec}")
        tur.append(sec)
        ziyaret[sec] = 1
    return tur
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Ortanca için 100 oran sıralanır: C'de `qsort` + karşılaştırıcı, C++'ta `std::sort`, Python'da `sorted` (sıralama burada araçtır). Eşitlik kararları tamsayı kare uzaklıkla verilir (C/C++'ta `long long`, Python'da sınırsız `int`); `sqrt` yalnız tur uzunluğu için kullanılır. Python'da `bool` değerler (`oran < 1 + 1e-9`) toplamaya doğrudan 0/1 olarak katılır; C'de karşılaştırma zaten `int` verir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 28, 170, 198; Python 145, 158, 165.)

## A.4 Örnek 4 · 2-opt yerel arama

C++ **cpp/H14_04_iki_opt.cpp · satır 92–122**

```cpp
static int iki_opt(std::vector<int>& tur, bool ayrinti)
{
    int iyilestirme = 0;
    bool degisti = true;
    while (degisti) {
        degisti = false;
        for (int i = 1; i < n - 1 && !degisti; i++) {
            for (int j = i + 1; j < n && !degisti; j++) {
                int a = tur[i - 1], b = tur[i], c = tur[j];
                int e = tur[(j + 1) % n];
                double kazanc = (d[a][c] + d[b][e])
                              - (d[a][b] + d[c][e]);
                denenen++;
                if (kazanc < -EPS) {
                    ters_cevir(tur, i, j);
                    iyilestirme++;
                    degisti = true;
                    if (ayrinti) {
                        std::printf("    %2d  (%d, %d)  %d-%d, %d-%d  "
                                    "%d-%d, %d-%d  %7.2f  %8.2f   ",
                                    iyilestirme, i, j, a, b, c, e, a, c, b,
                                    e, -kazanc, tur_uzunlugu(tur));
                        tur_yaz(tur);
                        std::printf("\n");
                    }
                }
            }
        }
    }
    return iyilestirme;
}
```

Python **python/H14_04_iki_opt.py · satır 73–98**

```python
def iki_opt(tur, ayrinti):
    """İlk iyileştirmeli 2-opt; dönen: uygulanan hamle sayısı"""
    global denenen
    iyilestirme = 0
    degisti = True
    while degisti:
        degisti = False
        i = 1
        while i < n - 1 and not degisti:
            j = i + 1
            while j < n and not degisti:
                a, b, c = tur[i - 1], tur[i], tur[j]
                e = tur[(j + 1) % n]
                kazanc = (d[a][c] + d[b][e]) - (d[a][b] + d[c][e])
                denenen += 1
                if kazanc < -EPS:
                    ters_cevir(tur, i, j)
                    iyilestirme += 1
                    degisti = True
                    if ayrinti:
                        print(f"    {iyilestirme:2d}  ({i}, {j})  {a}-{b}, "
                              f"{c}-{e}  {a}-{c}, {b}-{e}  {-kazanc:7.2f}  "
                              f"{tur_uzunlugu(tur):8.2f}   {tur_metni(tur)}")
                j += 1
            i += 1
    return iyilestirme
```

**DİL PENCERESİ · Farklılaşan satırlar**

Ters çevirme C'de geçici değişkenle, C++'ta `std::swap`, Python'da çoklu atamayla yapılır. C/C++'taki `for (…; … && !degisti; …)` döngüleri Python'da `while` ile yazıldı: `for … break` iç döngüden çıkar ama dış döngüyü sürdürür; ilk iyileştirmede taramanın baştan başlaması için iki döngünün de durması gerekir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 85, 220; Python 68, 186.)

## A.5 Örnek 5 · Held–Karp

C++ **cpp/H14_05_held_karp.cpp · satır 62–113**

```cpp
static double held_karp(std::vector<int>& tur)
{
    const std::size_t boyut = (std::size_t{1} << n) * n;
    dp.assign(boyut, SONSUZ);
    ebeveyn.assign(boyut, -1);
    dp[1 * n + 0] = 0.0;                          // S = {0}, j = 0
    durum = 0;
    gecis = 0;
    for (int S = 1; S < (1 << n); S += 2) {       // 0'ı içeren kümeler
        for (int j = 1; j < n; j++) {
            if (!(S & (1 << j))) {
                continue;
            }
            int onceki = S ^ (1 << j);
```

```
            double en = SONSUZ;
            int arg = -1;
            for (int i = 0; i < n; i++) {
                if (!(onceki & (1 << i)) || dp[onceki * n + i] >= SONSUZ) {
                    continue;
                }
                double aday = dp[onceki * n + i] + d[i][j];
                gecis++;
                if (aday < en) {
                    en = aday;
                    arg = i;
                }
            }
            dp[S * n + j] = en;
            ebeveyn[S * n + j] = static_cast<signed char>(arg);
            durum++;
        }
    }
    int tum = (1 << n) - 1, son = -1;
    double en = SONSUZ;
    for (int j = 1; j < n; j++) {
        double aday = dp[tum * n + j] + d[j][0];
        if (aday < en) {
            en = aday;
            son = j;
        }
    }
    tur.assign(n, 0);
    int S = tum;
    for (int k = n - 1; k >= 1; k--) {           // geri çıkarma
        tur[k] = son;
        int i = ebeveyn[S * n + son];
        S ^= 1 << son;
        son = i;
    }
    tur[0] = son;                                // 0 olmalı
    return en;
}
```

Python `python/H14_05_held_karp.py · satır 52–95`

```
def held_karp():
    """Dönen: (uzunluk, tur); tablolar dp, ebeveyn (genel)"""
    global dp, ebeveyn, durum, gecis
    boyut = (1 << n) * n
    dp = [SONSUZ] * boyut
    ebeveyn = [-1] * boyut
    dp[1 * n + 0] = 0.0                          # S = {0}, j = 0
    durum = 0
    gecis = 0
    for S in range(1, 1 << n, 2):                # 0'ı içeren kümeler
        for j in range(1, n):
            if not S & (1 << j):
                continue
            onceki = S ^ (1 << j)
```

```
            en = SONSUZ
            arg = -1
            for i in range(n):
                if not onceki & (1 << i) or dp[onceki * n + i] >= SONSUZ:
                    continue
                aday = dp[onceki * n + i] + d[i][j]
                gecis += 1
                if aday < en:
                    en = aday
                    arg = i
            dp[S * n + j] = en
            ebeveyn[S * n + j] = arg
            durum += 1
    tum = (1 << n) - 1
    son = -1
    en = SONSUZ
    for j in range(1, n):
        aday = dp[tum * n + j] + d[j][0]
        if aday < en:
            en = aday
            son = j
    tur = [0] * n
    S = tum
    for k in range(n - 1, 0, -1):                # geri çıkarma
        tur[k] = son
        i = ebeveyn[S * n + son]
        S ^= 1 << son
        son = i
    tur[0] = son                                 # 0 olmalı
    return en, tur
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Tablo C'de `malloc` ile tek blok (dp için 8, ebeveyn için 1 bayt), C++'ta `std::vector<double>` ve `std::vector<signed char>`, Python'da düz listedir. Bellek hesabı C içindir: Python listesi her öğe için 8 baytlık bir başvuru tutar ve sayılar ayrı nesnelerdir. Bit işlemleri (`S & (1 << j)`, `S ^ (1 << j)`) üç dilde aynı yazılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 38, 167; Python 144, 223.)

## A.6 Örnek 6 · Doğrulama ile bulma

C++ **cpp/H14_06_dogrulama_bulma.cpp · satır 25–56**

```
static bool dogrula(long long hedef, const std::vector<int>& sec,
                    long long& adim, std::string& neden)
{
    const int n = static_cast<int>(a.size());
    std::vector<bool> goruldu(n, false);
    long long toplam = 0;
    adim = 0;
    if (static_cast<int>(sec.size()) > n) {
        neden = "öğe sayısı n'den fazla";
        return false;
    }
```

```
    for (int i : sec) {
        adim++;
        if (i < 0 || i >= n) {
            neden = "indis dizi dışında";
            return false;
        }
        if (goruldu[i]) {
            neden = "aynı indis iki kez";
            return false;
        }
        goruldu[i] = true;
        toplam += a[i];
    }
    adim++;
    if (toplam != hedef) {
        neden = "toplam hedefe eşit değil";
        return false;
    }
    neden = "toplam = hedef";
    return true;
}
```

Python **python/H14_06_dogrulama_bulma.py · satır 17–38**

```
def dogrula(hedef, sec):
    """Doğrulayıcı: sertifika sec (indisler) geçerli mi, tekrarsız mı,
    toplamı hedef mi? Her öğe için bir adım + son karşılaştırma: O(n + k).
    Dönen: (sonuç, adım, neden)"""
    n = len(a)
    goruldu = [False] * n
    toplam = 0
    adim = 0
    if len(sec) > n:
        return False, adim, "öğe sayısı n'den fazla"
    for i in sec:
        adim += 1
        if i < 0 or i >= n:
            return False, adim, "indis dizi dışında"
        if goruldu[i]:
            return False, adim, "aynı indis iki kez"
        goruldu[i] = True
        toplam += a[i]
    adim += 1
    if toplam != hedef:
        return False, adim, "toplam hedefe eşit değil"
    return True, adim, "toplam = hedef"
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Doğrulayıcının üç çıktısı (sonuç, adım sayısı, neden) C'de işaretçi parametrelerle, C++'ta referanslarla, Python'da demet döndürülerek verilir. Kısmi çözüm C'de dizi + sayaçla, C++'ta `push_back`/`pop_back`, Python'da `append`/`pop` ile (yığın gibi) tutulur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 78, 80; Python 59, 61, 89.)

## A.7 Örnek 7 · İndirgeme: labirent → BFS

C++ cpp/H14_07_indirgeme.cpp · satır 201–232

```cpp
static int bfs(int bas, int hedef, std::vector<int>& uzaklik,
               std::vector<int>& ebeveyn, int& ziyaret_sayisi)
{
    std::queue<int> kuyruk;                       // H9'un kuyruğu
    const int dr[4] = {-1, 0, 0, 1}, dc[4] = {0, -1, 1, 0};
    uzaklik.assign(R * C, -1);
    ebeveyn.assign(R * C, -1);
    uzaklik[bas] = 0;
    kuyruk.push(bas);
    ziyaret_sayisi = 0;
    while (!kuyruk.empty()) {
        int u = kuyruk.front();
        kuyruk.pop();
        ziyaret_sayisi++;
        if (u == hedef) {
            break;
        }
        for (int k = 0; k < 4; k++) {           // artan köşe numarası
            int r = u / C + dr[k], c = u % C + dc[k];
            if (r < 0 || r >= R || c < 0 || c >= C || izgara[r][c] == '#') {
                continue;
            }
            int v = r * C + c;
            if (uzaklik[v] == -1) {
                uzaklik[v] = uzaklik[u] + 1;
                ebeveyn[v] = u;
                kuyruk.push(v);
            }
        }
    }
    return uzaklik[hedef];
}
```

Python python/H14_07_indirgeme.py · satır 171–196

```python
def bfs(bas, hedef):
    """Ağırlıksız grafta BFS; köşe = r·C + c.
    Dönen: (uzaklık listesi, ebeveyn listesi, kuyruktan çıkan köşe)"""
    kuyruk = deque()                              # H9'un kuyruğu
    dr = (-1, 0, 0, 1)
    dc = (0, -1, 1, 0)
    uzaklik = [-1] * (R * C)
    ebeveyn = [-1] * (R * C)
    uzaklik[bas] = 0
    kuyruk.append(bas)
    ziyaret_sayisi = 0
    while kuyruk:
        u = kuyruk.popleft()
        ziyaret_sayisi += 1
        if u == hedef:
            break
        for k in range(4):                        # artan köşe numarası
```

```
            r, c = u // C + dr[k], u % C + dc[k]
            if r < 0 or r >= R or c < 0 or c >= C or izgara[r][c] == "#":
                continue
            v = r * C + c
            if uzaklik[v] == -1:
                uzaklik[v] = uzaklik[u] + 1
                ebeveyn[v] = u
                kuyruk.append(v)
    return uzaklik, ebeveyn, ziyaret_sayisi
```

DİL PENCERESİ · **Farklılaşan satırlar**

Dosya okuma C'de `fgets` + `strtol`/`sscanf`, C++'ta `std::getline` + `std::istringstream`, Python'da `open` + `split` ile yapılır. Hatalar C/C++'ta dönüş koduyla, Python'da istisnayla (`BicimHatasi`, `OSError`) taşınır; üç dil aynı iletiyi ve çıkış kodunu (1) üretir. Ders kodlarının alfabetik sırası `qsort`/`std::sort`/`sorted` ile bulunur (araç); ASCII kodlarda üçü aynı sırayı verir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 26, 178, 280; Python 19, 148, 237, 288.)

## A.8 Örnek 8 · Profil çıkarma: darboğaz

C++ **cpp/H14_08_profil_cikarma.cpp · satır 66–76**

```
66 static long long yinelenen_ciftler(long long& kars)
67 {
68     long long cift = 0;
69     for (int i = 0; i < n; i++) {
70         for (int j = i + 1; j < n; j++) {
71             kars++;
72             cift += numara[i] == numara[j];
73         }
74     }
75     return cift;
76 }
```

Python **python/H14_08_profil_cikarma.py · satır 57–66**

```
57 def yinelenen_ciftler():
58     """3a) Yinelenen, YAVAŞ: tüm çiftler — n(n−1)/2 karşılaştırma
59     (darboğaz). Dönen: (çift, karşılaştırma)"""
60     cift = 0
61     kars = 0
62     for i in range(n):
63         for j in range(i + 1, n):
64             kars += 1
65             cift += numara[i] == numara[j]
66     return cift, kars
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Sayaçlar C'de işaretçi parametreyle, C++'ta referansla, Python'da demet döndürülerek taşınır. Kopyalama C'de `memcpy`, C++'ta vektör ataması, Python'da dilimle yapılır. Adım süreleri dile göre çok farklıdır (stderr), ama profilin gösterdiği darboğaz üç dilde aynıdır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 26, 108; Python 93, 101.)

## A.9 Örnek 9 · Dönem kıyaslaması: karma (denetim toplamı)

C++ **cpp/H14_09_donem_kiyas.cpp · satır 166–170**

```cpp
static std::uint32_t karma(int anahtar)          // Fibonacci karması
{
    return static_cast<std::uint32_t>(static_cast<std::uint32_t>(anahtar) *
                                      2654435769u) >> (32 - karma_bit);
}
```

Python **python/H14_09_donem_kiyas.py · satır 144–146**

```python
def karma(anahtar):
    """Fibonacci karması (32 bit işaretsiz aritmetik)"""
    return ((anahtar * 2654435769) & 0xFFFFFFFF) >> (32 - karma_bit)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Dijkstra'nın öncelik kuyruğu C'de elle yazılmış ikili öbek, C++'ta `std::priority_queue` (`std::greater` ile en küçük öbek), Python'da `heapq`tır. Öğeler (uzaklık, köşe) çiftleri olduğundan üç kuyruk aynı sırayla çıkarır ve denetim toplamı aynıdır. 32 bit karma C/C++'ta `uint32_t` sarmasıyla, Python'da `& 0xFFFFFFFF` maskesiyle elde edilir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 233, 255, 261; Python 201, 221, 230.)

## A.10 Örnek 10 · Mini-OBS çekirdeği: düzenleme uzaklığı

C++ **cpp/H14_10_miniobs_cekirdek.cpp · satır 231–253**

```cpp
static int duzenleme(const std::string& a, const std::string& b)
{
    const std::size_t m = b.size();
    std::vector<int> onceki(m + 1), simdi(m + 1);
    for (std::size_t j = 0; j <= m; j++) {
        onceki[j] = static_cast<int>(j);
    }
    for (std::size_t i = 1; i <= a.size(); i++) {
        simdi[0] = static_cast<int>(i);
        for (std::size_t j = 1; j <= m; j++) {
            int en = onceki[j] + 1;                          // silme
            if (simdi[j - 1] + 1 < en) {
                en = simdi[j - 1] + 1;                       // ekleme
            }
            if (onceki[j - 1] + (a[i - 1] != b[j - 1]) < en) {
                en = onceki[j - 1] + (a[i - 1] != b[j - 1]);  // değiştir
            }
```

```
            simdi[j] = en;
        }
        onceki = simdi;
    }
    return onceki[m];
}
```

Python `python/H14_10_miniobs_cekirdek.py · satır 198–213`

```python
def duzenleme(a, b):
    m = len(b)
    onceki = list(range(m + 1))
    simdi = [0] * (m + 1)
    for i in range(1, len(a) + 1):
        simdi[0] = i
        for j in range(1, m + 1):
            en = onceki[j] + 1                              # silme
            if simdi[j - 1] + 1 < en:
                en = simdi[j - 1] + 1                       # ekleme
            fark = 1 if a[i - 1] != b[j - 1] else 0
            if onceki[j - 1] + fark < en:
                en = onceki[j - 1] + fark                   # değiştir
            simdi[j] = en
        onceki = simdi[:]
    return onceki[m]
```

DİL PENCERESİ • **Farklılaşan satırlar**

Kayıt deposu üç dilde de aynı büyütme politikasıyla (2, 4, 8, ...) elle yazıldı: C'de `realloc`, C++'ta `std::unique_ptr<Kayit[]>` ile yeni blok + taşıma, Python'da `[None] * kapasite`; bu yüzden 'depo kapasitesi' çıktısı aynıdır (`std::vector` ve Python listesinin kendi büyüme adımları gerçekleştirime bağlıdır). Yinelenen modülü C'de elle yazılmış karma kümesi, C++'ta `std::unordered_map`, Python'da `dict` kullanır; dolaşma sırası tanımsız olduğundan numaralar artan sıraya konarak yazdırılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 33, 163, 168; Python 27, 143, 231.)

# Ek B. Başvuru Tabloları

Bu ek, ana metnin başvuru niteliğindeki ayrıntı tablolarını toplar: strateji kataloğu (Bölüm 3.1), dönem kıyaslamasının bütün katları ve dönemin deneylerinden birer bulgu (Bölüm 6), final projesinin gereksinimleri, teslim kalemleri, kabul ölçütleri, test senaryoları ve ölçüm eşikleri (Bölüm 10.4–10.5), rubriğin düzey tanımları ve sunum yönergesi (Bölüm 10.6–10.7), H1–H14 konu haritası, iki haftalık çalışma planı ve bankadan örnek sorular (Bölüm 12). Ana metin her birine kısa bir özetle gönderme yapar; sayılar aynı CSV ve belgelerden programla alınır.

## B.1 Strateji kataloğu (Bölüm 3.1)

**Tablo 14.19**
*Strateji kataloğu: dokuz ana strateji (artımlı yöntem ve veri yapısıyla çözüm strateji matrisinde); örnekler her ailenin ilk kartlarıdır*

| Strateji | Ne zaman? Nasıl tanınır? | Dönemdeki örnekler (kartlardan) |
|---|---|---|
| Kaba kuvvet ve üretme | çözüm uzayı açıkça listelenebilir (bütün çiftler, alt kümeler, permütasyonlar) | En büyük eleman (H1); Doğrusal arama (H1) |
| Geri izleme ve budama | karar ağacı: her düzeyde bir seçim; kısıt ihlali dalı keser | N-vezir (H3); Alt küme toplamı (H3) |
| Azalt ve fethet (ikili arama) | her adımda aday aralığın yarısı (ya da bir parçası) güvenle elenir | Öklid algoritması (H1); İkili arama (H4) |
| Böl ve fethet | T(n) = a·T(n/b) + f(n) biçiminde yineleme bağıntısı | Hızlı üs alma (H2); Birleştirmeli sıralama (H6) |
| Açgözlü | seçimi bir kez yap, geri dönme; çoğunlukla sırala + tara ya da öncelik kuyruğu | Aralık çizelgeleme (H12); Kesirli sırt çantası (H12) |
| Dinamik programlama | durum + geçiş + temel durum; tablo ya da memo; sözde polinom olabilir | Kadane (H2); Fibonacci: memoization, tablolama, iki değişken (H13) |
| Graf modelleme ve dolaşma | köşe = durum/nesne, kenar = ilişki; soru erişilebilirlik, en az kenar, sıra | Genişlik öncelikli arama (H11); Derinlik öncelikli arama (H11) |
| Karma, ön hesaplama ve doğrudan adresleme | anahtar → konum eşlemesi; sorgudan önce bir kez hazırlanan tablo | Eratosthenes kalburu (H2); Sayma sıralaması (H7) |
| Sezgisel ve yerel arama | kurucu kural (en yakın komşu) + iyileştirme hamlesi (2-opt); sonuç alt sınır ya da kâhinle kıyaslanır | En yakın komşu sezgiseli (H14); 2-opt yerel arama (H14) |

## B.2 Dönem kıyaslamasının ayrıntı tabloları (Bölüm 6)

**Tablo 14.20**
*Algoritmik kazançlar (h14_ozet_katlar.csv; kat = yavaş süre / hızlı süre)*

| İşlem | Yavaş | Hızlı | n | Kat |
|---|---|---|---|---|
| arama | v2.0 doğrusal | v2.3 ikili | 1 000 000 | 8 229 |
| arama | v2.0 doğrusal | v2.9 karma | 1 000 000 | 49 042 |
| arama | v2.3 ikili | v2.9 karma | 1 000 000 | 5,96 |
| yinelenen | v2.1 tüm çiftler | v2.5 sıralama | 100 000 | 102 |
| yinelenen | v2.1 tüm çiftler | v2.9 karma | 100 000 | 311 |
| yinelenen | v2.5 sıralama | v2.9 karma | 100 000 | 3,06 |
| yinelenen | v2.1 tüm çiftler (kaba kestirim) | v2.5 sıralama | 1 000 000 | 805 (kestirim) |

| İşlem | Yavaş | Hızlı | n | Kat |
|---|---|---|---|---|
| yinelenen | v2.1 tüm çiftler (kaba kestirim) | v2.9 karma | 1 000 000 | 1 744 (kestirim) |
| yinelenen | v2.1 tüm çiftler (eğimli kestirim) | v2.5 sıralama | 1 000 000 | 1 348 (kestirim) |
| yinelenen | v2.1 tüm çiftler (eğimli kestirim) | v2.9 karma | 1 000 000 | 2 922 (kestirim) |
| yinelenen | v2.5 sıralama | v2.9 karma | 1 000 000 | 2,17 |
| yukleme | v2.0 (v1.0 yükleyicisi) | v2.5 toplu | 16 000 | 4,67 |
| yukleme | v2.0 (v1.0 yükleyicisi) | v2.5 toplu | 32 000 | 22,3 |
| yukleme | v2.0 (kaba kestirim) | v2.5 toplu | 1 000 000 | 886 (kestirim) |
| yukleme | v2.0 (eğimli kestirim) | v2.5 toplu | 1 000 000 | 9 881 (kestirim) |
| ders_secimi | geri izleme (v2.2 ağacı) | v2.12 DP | 20 | 1 337 |
| ders_secimi | geri izleme (v2.2 ağacı) | v2.12 DP | 35 | 280 994 |

**Tablo 14.21**

*Dönemin deneylerinden birer bulgu (h14_ozet_sentez.csv; kaynak sütunu CSV'de)*

| Hafta | Ne ölçüldü? | Kuram | Sonuç (o haftanın CSV'sinden) |
|---|---|---|---|
| H1 | doğrusal toplam, n = $10^7$: dil ve derleme | Θ(n); dil ve derleme yalnız sabit çarpanı değiştirir | Python / C -O2 = 57,5 kat; C -O0 / C -O2 = 3,58 kat |
| H2 | "gizemli" fonksiyonların log–log eğimi (C -O2) | eğim = büyüme üssü (oran testi) | gizemli_1: eğim 2,01 → $n^2$; gizemli_2: eğim 1,02 → n; gizemli_3: eğim 1,08 → n log n |
| H3 | 8-vezir: budamasız ↔ budamalı düğüm | budama ağacın büyük bölümünü keser | 19 173 961 ↔ 2 057 düğüm (9 321 kat az) |
| H4 | doğrusal ↔ ikili arama | Θ(n) ↔ Θ(log n); küçük n'de sabitler baskın | kesişim n ≈ 43; n = $10^5$'te doğrusal / ikili = 484 kat |
| H5 | araya ekleme: rastgele ↔ neredeyse sıralı girdi | en kötü/ortalama Θ($n^2$), neredeyse sıralıda Θ(n) (uyarlanabilir) | n 10 000→20 000: süre oranı 4,44 (eğim 2,15) ↔ 2,14 (eğim 1,10) |
| H6 | birleştirmeli sıralama, oran testi | Θ(n log n): T(10n)/T(n) ≈ 10·log(10n)/log n | n 1 000 000→10 000 000: oran 13,00 (kuram 11,67; karesel olsa 100) |
| H7 | n = $10^6$: seçmeli ↔ birleştirmeli; sayma (K = 100) ↔ kütüphane | Θ($n^2$) ↔ Θ(n log n); Θ(n + K) karşılaştırmasız | seçmeli 1 290 s (kestirim) ↔ birleştirmeli 120,9 ms; sayma 1,16 ms ↔ qsort 95,3 ms |
| H8 | $10^6$ düğüm dolaşma: dizi ↔ bitişik liste ↔ dağınık liste | üçü de Θ(n); fark önbellekte | 0,51 ↔ 5,23 ↔ 154,7 ns/öğe (dağınık / bitişik = 29,6 kat) |
| H9 | kuyruk: kaydırmalı dizi ↔ dairesel tampon | işlem başına Θ(n) ↔ Θ(1) | n = 100 000: 204 kat |
| H10 | doğrusal yoklama: ortalama yoklama ↔ Knuth formülü | başarısız ½(1 + 1/(1−α)²) | α = 0,95: ölçülen 192,27, kuram 200,48 (sapma %-4,10); bütün α'larda en büyük \|sapma\| %4,10 |
| H11 | seyrek grafta BFS: komşuluk matrisi ↔ listesi | Θ($V^2$) ↔ Θ(V + E) | V = 20 000: süre 605 kat, bellek 296 kat |
| H12 | birleşim–bul: saf ↔ rütbe + yol sıkıştırma | saf O(n) ağaç yüksekliği ↔ neredeyse sabit | n = 30 000: işlem başına izleme 766,5 ↔ 0,79; ağaç yüksekliği 3 073 ↔ 3 |
| H13 | Fibonacci n = 35: saf özyineleme ↔ memo | Θ($\varphi^n$) çağrı ↔ Θ(n) | çağrı 29 860 703 ↔ 69; süre katı saf/memo 199 725 |

## B.3 Final projesi: gereksinimler, teslim, kabul ölçütleri ve test senaryoları (Bölüm 10.2–10.5)

**Tablo 14.22**
*Varsayılan projenin gereksinimleri (v3.0 belgesi § 11)*

| # | Gereksinim | Ayrıntı |
|---|---|---|
| 1 | Üç giriş kipi | v1.0 menüsü (v1.0 oturumları aynen) + menüden komut kipine geçiş; betik kipi (`--betik dosya\|-`; her komut `>` ile yankılanır; hatalı komutta durmaz); alt komut kipi (`./mini_obs VERİ KOMUT …`). |
| 2 | Komutlar | v3.0 belgesi Tablo 7'deki sekiz grup; bütün modüller tek durum yapısında (`Uygulama`) buluşur, tek temizlik yolu vardır. |
| 3 | Tek tip hata bildirimi | işlemler durum kodu döndürür (v3.0 belgesi Tablo 3); kullanım hatası "HATA: … / Kullanım: …" ve çıkış 2; aranan kaydın yokluğu ve iş kuralı reddi "RET: …" ve çıkış 1; dosya, biçim ve bellek hataları "HATA: …" ve çıkış 1; betikte en ağır durum. |
| 4 | Yapılandırma | `mini_obs.ayar` (v3.0 belgesi Tablo 4); `--ayar dosya`; `ayar ANAHTAR DEĞER`; geçersiz değer atanmaz. |
| 5 | Bütünleştirme kuralları | ders kayıtları Mini-OBS kaydındaki öğrencilerle (yabancı numara → hiçbir kayıt yapılmaz); önkoşul döngülerinin hepsi (güçlü bağlı bileşenler; her birinde en küçük köşeden geçen en kısa döngü); AKTS sınırlı dönem planı (sezgisel + iki alt sınır + doğrulayıcı); sınav programında gün sınırı, salon kapasitesi, öğrenci başına günde en çok bir sınav; numara dizini, soyad dizini, ad metni ve öneri sözlüğü saklanır ve kayıt dizisi değişince geçersiz sayılır; çok dersli geri al/yinele. |
| 6 | Test paketi (`make test`) | birim (v1.0'ın 99 doğrulaması dahil), kâhin (en az üç kasıtlı hata: `kosaraju`, `ayni_donem`, `onbellek`), stres (gölge modelli rastgele işlem dizisi; $10^5$–$10^6$ kayıt; sınır durumları), regresyon. |
| 7 | Kalite | `make asan`, `make valgrind` temiz; `make kapsam` ile satır kapsamı; tek bellek sahipliği tablosu. |
| 8 | Dönem kıyaslaması (`make olcum`, ≤ 6 dakika) | en az sekiz işlemin ilk ve son sürümü aynı programda; algoritmik sürümler $10^6$'da, karesel sürümler $10^4$–$10^5$'te ölçülüp işaretli kestirimle; en az üç işlemde -O0 ↔ -O2. |

**Tablo 14.23**
*Teslim edilecekler (yönerge § 6)*

| # | Teslim edilecek | İçerik |
|---|---|---|
| 1 | Kod | `v3.0/` (ya da havuz projesinin klasörü) — kendi başına derlenir; `make temiz` yapılmış hâliyle. |
| 2 | Makefile | yönerge § 4.3'teki hedefler. |
| 3 | Test paketi | birim, kâhin (kasıtlı hata gösterimleriyle), stres ve regresyon testleri; `testler/` girdileri ve altın çıktılar. |
| 4 | Ölçüm verisi ve raporu | `olcum/*.csv`, `olcum/OLCUM_ORTAMI.txt` (raporda kullanılan çalıştırmanın kendisi); `rapor/final_raporu.pdf`, Markdown kaynağı, şekiller ve tabloları/grafikleri üreten betik. |
| 5 | README.md | derleme, kullanım (komut örnekleri), modüller, bilinen eksikler ve kabul ölçütlerinin ✓/✗ listesi (varsayılan projede v3.0 belgesinin "Kabul ölçütleri" listesi). |
| 6 | Sunum | `sunum/` altında PDF (10–12 slayt) ve canlı demo betiği (`sunum/demo.txt`: demoda çalıştıracağınız komutlar, sırasıyla). |
| 7 | Beyan | raporun Ek D'si — yararlanılan kaynaklar, referans sürümden alınan dosyalar, kullanılan araçlar ve amaçları. |

**Tablo 14.24**
*Kabul ölçütleri (v3.0 belgesi § 11; 11 ölçüt, README'de ✓/✗ listesi olarak)*

| # | Kabul ölçütü (denetlenebilir) |
|---|---|
| 1 | `make temiz && make` sıfır uyarı; `make test` geçer (birim, kâhin, stres, regresyon); `make asan` ve `make valgrind` temiz. |
| 2 | v1.0 menü oturumları T1–T6b'nin çıktısı v1.0 ikilisininkiyle aynı (bilinen üç fark dışında); T1 ve T6'nın yazdığı dosyalar bayt bayt aynı. |
| 3 | `./mini_obs ornek_veri/ogrenciler.csv ara soyad yıl` → 2301010045, 2301010048; `oneri mehmet yilmas` → MEHMET ALİ, MEHMET ALİ KAYA, CAN YILMAZ (bu sırayla). |
| 4 | `ayar kontenjan 2` + üç kayıt (`kaydol BLM101` 2301010045, …046, …048) + `birak BLM101 2301010045` → 2301010048 kayıtlı listeye aktarılır; `geri` → 2301010045 kayıtlı, 2301010048 bekleme listesinin başında. |
| 5 | `ara soyad ars` (dizin kurulur) → `ekle 2301010060;Deniz;Arslan;…` → `ara soyad ars` yeni kaydı bulur; `geri` → bulmaz; `yinele` → yine bulur (saklanan dizinin geçerliliği). |
| 6 | `testler/onkosul_iki_dongu.txt` ile `plan`: üç döngüsel bileşen (BLM201; BLM301 BLM302 BLM303; MAT102 MAT201), çıkış 1. |
| 7 | `donem 2301010046` (W = 30): 8 dönem, alt sınır 8 (en iyi); `donem - 20`: 13 dönem, alt sınır 12; doğrulama EVET. |
| 8 | `ayar kontenjan 3` + `kayit_oku ornek_veri/ders_kayitlari.txt` + `sinav ozet`: 12 sınav, 36 kenar, 5 gün (klik 5); `ayar sinav_gun 4` → YAPILAMAZ, çıkış 1; `ayar salon_kapasitesi 2` → 20 parça, 3 salon. |
| 9 | `kayit_oku testler/ders_kayit_yabanci.txt` → çıkış 1, hiçbir kayıt yapılmaz; `bilinmeyen` → çıkış 2; `goster 1` → çıkış 1. |
| 10 | `./test_kahin --hata-goster kosaraju\|ayni_donem\|onbellek` → uyuşmazlık, küçük karşı örnek, çıkış 1. |
| 11 | Dönem kıyaslaması: $10^6$ kayıtta yinelenen tespiti tüm çiftler (kestirim) / karma kümesi ≥ 500 kat; $10^5$ öğrencide toplu kayıt v2.7 / v3.0 ≥ 50 kat; öneri sorgusu her sorguda sözlük / saklanan sözlük ≥ 100 kat (kendi makinenizde; v3.0 belgesi § 10). |

**Tablo 14.25**
*Test senaryoları (v3.0 belgesi § 11)*

| # | Senaryo | Beklenen |
|---|---|---|
| 1 | boş veri dosyasıyla betik: liste, istatistik, sinav, geri | her biri tek tip iletiyle; çökme yok |
| 2 | hatalı argümanlar (`liste xyz`, `ilk basari 0`, `ayar kontenjan 0`, `yardim 9`) | "HATA: … / Kullanım: …"; betik sürer; çıkış 2 |
| 3 | geri al sınırı 1: iki işlem, iki `geri` | yalnız son işlem geri alınır; ikinci `geri` RET |
| 4 | kuyruk kapasitesi 1: iki `istek` | ikincisi RET (kuyruk dolu) |
| 5 | derste olan öğrenciyi `sil` | RET; kayıt ve defter değişmez |
| 6 | `kontenjan KOD K` küçültme ve büyütme | büyütmede bekleyenler sırayla aktarılır; geçmiş temizlenir |
| 7 | iki döngülü katalog | üç döngüsel bileşen; `donem` RET |
| 8 | AKTS'si W'yi aşan ders | "planlanamaz" işareti; ona bağlı dersler de |
| 9 | gün sınırı klikten küçük / büyük | YAPILAMAZ / EVET |
| 10 | yabancı numaralı kayıt dosyası | hiçbir kayıt yapılmaz; çıkış 1 |
| 11 | ara soyad → ekle → ara soyad → geri → ara soyad → yinele → ara soyad | yok / bulur / bulmaz / bulur |
| 12 | $10^6$ kayıt: yükle, ara no, oneri, kayit_uret, sinav ozet | süreler saniyeler mertebesinde; program geçerli |

**Tablo 14.26**
*Ölçümle ilgili kabul ölçütleri (yönerge § 7.3); son sütun v3.0 ölçüm raporunun referans makinesindeki değerdir, sizin sayınız değil*

| Ölçüt (v3.0 belgesi § 10) | Eşik | Referans makinede |
|---|---|---|
| $10^6$ kayıtta yinelenen tespiti: tüm çiftler (kestirim) / karma kümesi | ≥ 500 kat | 1 958 kat |
| $10^6$ kaydın toplu yüklenmesi (v2.5) | < 1 dakika | 1,75 s |
| $10^5$ öğrencide toplu ders kaydı: v2.7 yolu / v3.0 yolu | ≥ 50 kat | 139 kat |
| $10^6$ kayıtta öneri sorgusu: her sorguda sözlük / saklanan sözlük | ≥ 100 kat | 8 209 kat |
| n = 35 ders (W = 45): geri izleme / DP | ≥ $10^4$ kat | 458 374 kat |
| V = 16 000 (E = 4V): matrisle / listeyle Kahn | ≥ 50 kat | 256 kat |
| $10^6$ kayıtta numarayla arama (sorgu başına): doğrusal / karma | ≥ $10^4$ kat | 242 274 kat |
| `make olcum` toplam süresi | ≤ 6 dakika | 4 dk 6 s |

Bir eşik kendi makinenizde sağlanmıyorsa bu otomatik bir kayıp değildir: nedenini raporda ölçümle tartışın (ör. sanal makine, güç tasarrufu, önbellek boyutu, başka süreçler). Açıklanmayan sapma puan kaybettirir. Kestirimler (k) ölçüm gibi sunulmaz.

## B.4 Final projesi: rubrik düzeyleri ve sunum yönergesi (Bölüm 10.6–10.7)

**Tablo 14.27**
*Final projesi rubriğinin dört düzeyi (yönerge § 8; Yol Haritası Tablo 10.3)*

| Ölçüt (ağırlık) | Yetkin (%90–100) | Gelişmiş (%70–89) | Gelişmekte (%40–69) | Yetersiz (%0–39) |
|---|---|---|---|---|
| **İşlevsel doğruluk** (%25) | bütün menü işlevleri ve komutlar beklenen çıktıyı üretir; sınır durumları (boş veri, tek kayıt, döngülü graf, karşılanamayan kısıt, yabancı numara, hatalı argüman) doğru ve tek tip iletiyle işlenir; çıktı sözleşmesine tam uyum; v1.0 oturumları aynen | bir iki küçük sapma (ileti, çıkış kodu, tek bir sınır durumu); algoritmaların sonuçları doğru | bir modül eksik ya da bazı girdilerde yanlış (ör. önbellek eskiyor, dönem planında önkoşul aynı dönemde); bütünleştirme yarım (modüller ayrı programlarda) | birden çok modül eksik; yanlış sonuçlar ya da çökme |
| **Algoritma seçimi ve analiz** (%25) | her işlev için seçilen algoritma ve veri yapısı ölçümle ya da kuramla gerekçeli; kimlik kartları ve karmaşıklıklar doğru; değişmez ve ön/son koşul yorumları eksiksiz; NP-zor alt problemlerde sezgisel + alt sınır + küçük örnekte kesin çözüm; alternatifler tartışılmış | analiz doğru, gerekçelerde küçük boşluklar | karmaşıklık iddialarında hatalar; gerekçe yüzeysel; sezgisel çözüm "en iyi" diye sunulmuş | analiz yok ya da büyük ölçüde yanlış |
| **Test ve doğrulama** (%15) | elle izlenmiş birim testleri; farklı yoldan hesaplayan tohumlu kâhinler; gölge modelli stres testi ve $10^5$–$10^6$ kayıt; regresyon; en az üç kasıtlı hata ve küçük karşı örnek; hepsi `make test` ile otomatik | kâhin ve stres var; karşı örnek, sınır durumu ya da regresyon eksik | yalnızca birkaç elle test; kâhin sınanan fonksiyonu çağırıyor; stres testi yok | test yok ya da çalışmıyor |

| Ölçüt (ağırlık) | Yetkin (%90–100) | Gelişmiş (%70–89) | Gelişmekte (%40–69) | Yetersiz (%0–39) |
|---|---|---|---|---|
| **Ölçüm raporu** (%15) | şablonun bütün bölümleri; ilk ↔ son gerçekleştirim aynı düzenekte; $10^6$ ölçüm ve işaretli kestirimler; ortanca ve tekrar; okunur log–log grafikler; oran, eğim ve sayaçlarla kuram karşılaştırması; derleyici ↔ algoritma; sapmalar açıklanmış; makine bilgisi, sınırlılıklar, test kapsamı | eksik bir deney ya da zayıf yorum; biçim kusurları | tablolar var ama kuramla karşılaştırma ya da düzen bilgisi eksik; doğrusal eksen; tek ölçüm; etiketsiz kestirim | rapor yok; sayılar CSV ile tutarsız ya da ölçülmemiş |
| **Kod kalitesi ve bellek güvenliği** (%10) | sıfır uyarı; ASan/UBSan ve Valgrind temiz; modüler, sözleşmeli, okunur kod (Yol Haritası 6.5); tek tip hata bildirimi; yapılandırma; tek bellek sahipliği tablosu; bütün Makefile hedefleri | birkaç uyarı ya da biçim kusuru; bir hedef eksik | ASan/Valgrind raporu ya da sızıntı; dağınık modüller; hata bildirimi tutarsız | derleme sorunları, çok sayıda bellek hatası |
| **Sunum** (%10) | süreye uyar; mimari, algoritma seçimleri ve ölçüm bulguları açık ve sayılarla; canlı demo temiz bir kopyadan sorunsuz; sorulara kendi kodundan ve raporundan doğru ve akıcı yanıt | küçük aksaklıklar (süre, bir demo adımı); yanıtlar büyük ölçüde doğru | demo yarım; slaytlar kod dökümü; yanıtlar yüzeysel | sunum yapılmadı ya da proje açıklanamıyor |

**Puanın dağılımı (ölçüt içinde; yönerge § 8):**

- **İşlevsel doğruluk (25):** varsayılan projede: kayıtlar, geçmiş ve v1.0 regresyonu 4 · arama ve öneri 4 · listeler ve istatistik 3 · ders kaydı ve kuyruk 4 · önkoşul grafı ve dönem planı 4 · sınav programı 3 · ders seçimi 1 · tek tip hata bildirimi ve yapılandırma 2. Havuz projelerinde: yönerge § 2.2'deki asgari gereksinimler eşit ağırlıkla 20 · sınır durumları ve hata bildirimi 5.
- **Algoritma seçimi ve analiz (25):** kimlik kartları ve karmaşıklıklar 8 · değişmez ve sözleşme yorumları 4 · seçimlerin ölçümle ya da kuramla gerekçesi (alternatifleriyle) 8 · NP-zor alt problemlerde sezgisel/alt sınır/kesin yaklaşımı ve indirgemeler 5.
- **Test ve doğrulama (15):** birim testleri 3 · kâhin testleri 4 · stres testi (gölge model + büyük veri + sınır durumları) 4 · kasıtlı hatalar ve regresyon 2 · otomatik çalıştırma ve kapsam bilgisi 2.
- **Ölçüm raporu (15):** yöntem, ortam ve kestirim yöntemi 3 · tablolar ve grafikler 4 · kuramla karşılaştırma (oran, eğim, sayaç) 4 · derleyici ↔ algoritma ve tartışma 2 · sınırlılıklar, yeniden üretilebilirlik ve test kapsamı eki 2.
- **Kod kalitesi ve bellek güvenliği (10):** uyarısız derleme 2 · ASan/UBSan ve Valgrind temiz 3 · modülerlik, adlandırma, sözleşme yorumları, 80 sütun, tek tip hata ve yapılandırma 3 · Makefile hedefleri 2.
- **Sunum (10):** içerik ve yapı (mimari, seçimler, bulgular) 4 · canlı demo 3 · sorular 3.

**Sunum yönergesi** (yönerge § 9; Bölüm 10.7):

- **Süre:** öğrenci başına 15 dakika: **10 dakika sunum** (canlı demo dahil; demo en az 3 dakika) + **5 dakika soru**. Süre aşımında sunum durdurulur.
- **Slaytlar:** 10–12 slayt (PDF). Önerilen akış: problem ve kapsam (1) · mimari diyagramı (1–2) · algoritma seçimleri ve kimlik kartları (2) · doğrulama: test paketi, stres, bellek (1) · ölçüm bulguları: dönem kıyaslaması grafiği, kuramla karşılaştırma (2–3) · canlı demo (1) · öğrenilen dersler ve sınırlılıklar (1). Slaytta kod dökümü yok (en çok 10–15 satırlık bir kesit); her ölçüm sayısı raporunuzdaki CSV'den gelir. Başvuru içerik planı: v3.0 sunum taslağı.
- **Canlı demo:** sınıftaki bilgisayarda ya da kendi bilgisayarınızda, **temiz bir kopyadan**: `make temiz && make`, bir betik senaryosu (`sunum/demo.txt`: birkaç modülü uçtan uca gösteren 8–15

komut; en az bir hata durumu ve bir geri alma), büyük veride bir komut (ör. $10^5$–$10^6$ kayıtta arama ya da öneri) ve bir ölçüm noktası. Demo bozulursa yedek olarak demo betiğinin önceden kaydedilmiş çıktısını gösterebilirsiniz; ama önce canlı denenir ve aksaklık sunumda açıklanır.

- **Sorular:** öğretim üyesi (ve isteğe bağlı olarak akranlar) şu türlerden sorar: kendi kodunuzdan bir fonksiyonun açıklaması (değişmez, karmaşıklık, sahiplik); bir tasarım kararının gerekçesi; raporunuzdaki bir bulgunun açıklaması (ör. kuramdan sapma); "şu girdide ne olur?" türünden bir sınır durumu; demoda küçük bir değişiklik (ör. bir ayarı değiştirip komutu yeniden çalıştırma).

## B.5 Final hazırlığı: konu haritası, çalışma planı ve örnek sorular (Bölüm 12)

**Tablo 14.28**
*H1–H14 konu haritası (74 kimlik kartından programla; her haftanın ilk iki kartı ve en kötü durum karmaşıklığı; parantezde kalan kart sayısı)*

| Hafta | Konu | Algoritmalar — en kötü durum (kimlik kartlarından) | Strateji |
|---|---|---|---|
| H1 | Doğruluk ve Ölçüm | En büyük eleman Θ(n); Doğrusal arama Θ(n) (+1) | kaba kuvvet, azalt ve fethet |
| H2 | Asimptotik Analiz | Deneme bölmesiyle asallık O(√n); Eratosthenes kalburu O(N log log N) (+2) | kaba kuvvet, karma, böl ve fethet, dinamik programlama |
| H3 | Özyineleme ve Geri İzleme | Alt küme üretimi Θ(n·2ⁿ); Permütasyon üretimi Θ(n·n!) (+2) | kaba kuvvet, geri izleme |
| H4 | Arama Algoritmaları | Doğrusal arama Θ(n); İkili arama Θ(log n) (+3) | kaba kuvvet, azalt ve fethet |
| H5 | Temel Sıralama | Seçmeli sıralama Θ(n²); Kabarcık sıralaması Θ(n²) (+2) | artımlı yöntem |
| H6 | Böl ve Fethet | Birleştirmeli sıralama Θ(n log n); Hızlı sıralama Θ(n²) (+3) | böl ve fethet |
| H7 | Öbek ve Doğrusal Sıralama | Öbeğe ekleme O(log n); En büyüğü çıkarma Θ(log n) (+5) | veri yapısıyla çözüm, karma |
| H8 | Dinamik Dizi ve Bağlı Liste | Dinamik dizi O(n); Bağlı liste O(1) / Θ(n) (+4) | veri yapısıyla çözüm, böl ve fethet |
| H9 | Yığın ve Kuyruk | Yığın O(1)*; Kuyruk O(1) (+5) | veri yapısıyla çözüm |
| H10 | Karma Tabloları | Karma tablosu Θ(n); Ayrı zincirleme Θ(n) (+5) | karma, kaba kuvvet |
| H11 | Graflar: BFS ve DFS | Genişlik öncelikli arama Θ(V + E); Derinlik öncelikli arama Θ(V + E) (+3) | graf modelleme |
| H12 | Açgözlü Algoritmalar | Aralık çizelgeleme Θ(n log n); Kesirli sırt çantası Θ(n log n) (+6) | açgözlü, veri yapısıyla çözüm |
| H13 | Dinamik Programlama | Fibonacci Θ(n); Bozuk para Θ(n·S) (+3) | dinamik programlama |
| H14 | Sentez ve Final Projesi | Gezgin satıcı Θ(n!); Held–Karp Θ(n²·2ⁿ) (+2) | kaba kuvvet, dinamik programlama, sezgisel |

**Tablo 14.29**
*İki haftalık final çalışma planı (Bölüm 12.3)*

| Gün | Konu | Etkinlik |
|---|---|---|
| 1–3 | Modül I (H1–H3): ölçüm, büyüme sınıfları, özyineleme ağaçları, geri izleme | kimlik kartlarını kâğıda yeniden yazın; bankadan o haftanın beş sorusu |

| Gün | Konu | Etkinlik |
|---|---|---|
| 4–6 | Modül II (H4–H7): arama ve sıralama; değişmezler (lower_bound, Lomuto, öbek) | bir diziyi dört algoritmayla elle sıralayın, karşılaştırmaları sayın |
| 7–8 | Modül III (H8–H10): dinamik dizi, liste, yığın/kuyruk, karma; amortize analiz, yük faktörü | kod yazma: listeyi ters çevirme, dairesel kuyruk, doğrusal yoklama |
| 9–11 | Modül IV–V (H11–H14): BFS/DFS, Dijkstra, Kruskal, DP tabloları, gezgin satıcı, strateji, P/NP | her problemde önce “hangi strateji, neden?” |
| 12–14 | tekrar | bankanın tamamı süreyle (soru başına ≈ 2 dk); yanlışları türe göre gruplayıp Hata Avcısı ve alıştırmalara dönün; deneme sınavı |

**Bankadan örnek sorular.** Aşağıdaki iki soru final hazırlık soru bankasından seçilmiştir (yanıtlar web'de, soruyu yanıtladıktan sonra açıklamasıyla görünür):

**Soru 07 (H2 · İzleme).** `for (int i = 100; i > 0; i /= 3) say++;` döngüsü kaç kez döner?

C `quiz/H14_Q07_h2_ucte_bir.c · satır 13–16`

```c
    for (int i = 100; i > 0; i /= 3) {
        printf(" %d", i);
        say++;
    }
```

- **A)** 5 (i: 100, 33, 11, 3, 1)
- **B)** 4 (i: 100, 33, 11, 3)
- **C)** 7 ($\lceil \log_2 100 \rceil$)
- **D)** 34 ($\lceil 100/3 \rceil$)

**Soru 72 (H14 · Diller arası okuma).** Python'da `len(list(itertools.permutations([2, 1, 3])))` nedir? C++'ta `v = {2, 1, 3}` ile `do { … } while (std::next_permutation(v.begin(), v.end()));` kaç permütasyon üretir?

C++ `quiz/H14_Q72_h14_permutasyon.cpp · satır 17–21`

```cpp
    std::vector<int> v = {2, 1, 3};
    std::vector<std::vector<int>> p;
    do {
        p.push_back(v);
    } while (std::next_permutation(v.begin(), v.end()));
```

- **A)** İkisi de 6
- **B)** Python 4, C++ 6
- **C)** Python 6, C++ 4
- **D)** İkisi de 4